2027

Human Computer Interaction

By : Dr. Waleed Abd Elkhalik

2025/2026 2025/2026 2025/2026

Faculty of Computers and Informatics – Tanta University
Computer Science department.

[2026-2027]

رومانی صبحی رزق جرجس مطر

Course Specification

(2025)

1. Basic Information

Course Title (according to the bylaw)	**Human Computer Interaction**			
Course Code (according to the bylaw)	**CS331**			
Department/s participating in delivery of the course	**Computer Science**			
Number of credit hours/points of the course (according to the bylaw)	**Theoretical**	**Practical**	**Other (specify)**	**Total**
	2	**2**		
Course Type	اجباري			
Academic level at which the course is taught	الفرقة/المستوي الثالث			
Academic Program	**First Term 2026 / 2027**			
Faculty/Institute	**Faculty of Computers and Informatics**			
University/Academy	**Tanta University**			
Name of Course Coordinator	**Dr. Waleed Abd Elkhalik**			
Course Specification Approval Date	**9/25/2025**			
Course Specification Approval (Attach the decision/minutes of the department /committee/council ....)				

2. Course Overview (Brief summary of scientific content)

This course aims to introduce students to the following concepts and cognitive skills:

This course introduces the study of how people use computing systems and how to design systems that people can use effectively, efficiently and safely. It combines findings from cognitive psychology and human factors with practical design and evaluation methods, so that students can diagnose usability problems, justify design decisions with evidence, and build and test interface prototypes. Specifically, the course covers:

1. **People and Machines** – The relationship between people and computer systems, Norman's seven-stage interaction cycle, the gulfs of execution and evaluation, task allocation between human and machine, and the analysis of human error (slips, mistakes and violations).
2. **Human Factors and Psychology** – Perception, attention, working memory, learning and mental models, and the design consequences that follow from each.
3. **Motivation for Usability** – The components of usability and how to express usability as measurable, testable requirements.
4. **Principles of Interaction and Interface Design Issues** – Visibility, feedback, affordance, mapping, constraints and consistency, and Shneiderman's eight golden rules.
5. **Interaction Styles** – Command languages, menus, form fill-in, windows, icons and direct manipulation.
6. **Error Messages, Response Time and Feedback** – Writing messages that enable recovery, the response-time thresholds that govern perceived performance, and designing for waiting.
7. **Physical Interaction** – Input and output devices, Fitts's law, ergonomics, touch and gesture, and the choice of interaction techniques.
8. **The Design Process, User Models and Interface Evaluation** – User-centred design, task analysis, personas and scenarios, rapid prototyping, usability testing, heuristic evaluation and iterative refinement.
9. **Natural Language and Voice Interfaces** – Speech recognition, text-to-speech, and the design of interfaces without a screen.

2025/2026 2025/2026 2025/2026

3. Course Learning Outcomes CLOs

Matrix of course learning outcomes CLOs with program outcomes POs (NARS/ARS)

		Course Learning Outcomes Upon completion of the course, the student will be able to:	
Code	**Text**	**Code**	**Text**
A1	Explain the interaction cycle between people and computer systems (Norman's seven stages, the gulfs of execution and evaluation) and the human factors – perception, attention and memory – that govern it.		
A2			

		Course Learning Outcomes Upon completion of the course, the student will be able to:	
Code	**Text**	**Code**	**Text**
	Describe the components of usability, the principles of interaction (visibility, feedback, affordance, mapping, constraints, consistency) and Shneiderman's eight golden rules.		
A3	Describe the main interaction styles and devices: command languages, menus, forms, windows, icons, direct manipulation, physical input devices, and natural language and voice interfaces.		
B1	Analyze an interface failure by locating it in the interaction cycle, classifying the human error involved (slip, mistake or violation) and identifying the design principle it violates.		
B2	Evaluate interface designs quantitatively using usability measures, response-time thresholds and Fitts's law, and specify usability requirements that can be tested.		
C1	Apply the user-centred design process – task analysis, personas and scenarios – to derive the requirements of an interface.		
C2	Build low- and high-fidelity interface prototypes (paper and HTML/CSS/JavaScript) and refine them iteratively.		
C3	Conduct usability tests and heuristic evaluations, and report ranked findings with justified design changes.		
D1	Work effectively in a team and communicate design decisions and evaluation results clearly in written reports and presentations.		

2025/2026

4. Teaching and Learning Methods

1. Lectures
2. Discussion (Brainstorming) and case studies of real interface failures
3. Self-learning (reading and the weekly failure log)
4. Laboratory exercises (paper prototyping and HTML/CSS/JavaScript prototypes)
5. Team project (user-centred design, usability testing and iterative refinement)

Course Schedule

Number of the Week	Scientific content of the course (Course Topics)	Total Weekly Hours	Expected number of the Learning Hours			
			Theoretical teaching (lectures/discussion groups/ .....)	Training (Practical/Clinical/ .....)	Self-learning (Tasks/ Assignments/ Projects/ ...)	Other (to be determined)
1	Course overview; People and Machines: what HCI is, Norman's seven-stage interaction cycle, the gulfs of execution and evaluation, human error (slips, mistakes, violations).	3	2	2	Start the failure log; observe a person using an interface; form project groups.	
2	Human Factors I – Perception and Attention: the visual system, pre-attentive features, Gestalt grouping laws, colour and contrast, attention.	3	2	2	Lab: audit a screen for grouping, colour coding and contrast (WCAG).	
3	Human Factors II – Memory, Learning and Mental Models: working memory, recognition vs. recall, mental models, mode errors.	3	2	2	Lab: convert recall tasks into recognition; diagnose a mode error.	
4	Usability: motivation, the five components of usability, and measurable usability requirements.	3	2	2	Project: choose a real task, study its users and write usability goals.	
5	Principles of Interaction and Interface Design Issues: visibility, feedback, affordance, mapping, constraints, consistency; Shneiderman's eight golden rules.	3	2	2	Lab: principle-based critique of real interfaces.	
6	Command Languages, Menus and Form Fill-in: interaction styles, menu breadth vs. depth, form design.	3	2	2	Project: paper prototype of the main screens.	

2025/2026 2025/2026 2025/2026

7	Windows, Icons and Direct Manipulation (WIMP) + Midterm Exam (Weeks 1–6).	3	2	2	Project: heuristic evaluation of another group's prototype.	
8	Error Messages, Response Time and Feedback: message design, response-time thresholds (0.1 s, 1 s, 10 s), designing for waiting.	3	2	2	Lab: rewrite error messages; add progress feedback to a prototype.	
9	Physical Interaction: input devices, Fitts's law, ergonomics, touch and gesture, interaction techniques.	3	2	2	Lab: Fitts's law measurement and target-size audit.	
10	The Design Process and User Models: user-centred design, task analysis, personas and scenarios.	3	2	2	Project: contextual inquiry, personas and scenarios; start the working prototype (HTML/CSS/JS).	
11	Interface Evaluation: rapid prototyping, usability testing, heuristic evaluation, iterative refinement.	3	2	2	Project: usability test with five users; ranked findings.	
12	Natural Language and Voice Interfaces: speech recognition, text-to-speech, voice dialogue design.	3	2	2	Lab: script and test a voice dialogue.	
13	Iterative Refinement Workshop: revising the prototype from test findings and re-testing.	3	2	2	Project: revised prototype and re-test; submit the course sheet.	
14	Practical Exam: demonstration and evaluation of the project prototype.	3	2	2	Project presentation and final report.	
15	Course Review and Integration.	3	2	2	Revision using the weekly question banks.	

5. Methods of students' assessment

No.	Assessment Methods *	Assessment Timing (Week Number)	Marks/ Scores	Percentage of total course Marks
1	Midterm Written Exam (Semester work)	7	25	25%
2	Final Written Exam	16	50	50%
3	Final Practical Exam (project prototype and its evaluation)	14	10	10%
4	Assignments / Laboratory Activities / Term Project / Course Sheet	2–13	15	15%

* The methods mentioned are examples, the organization may add and/or delete

6. Learning Resources and Supportive Facilities *

Learning resources (books, scientific references, etc.) *	**The main (essential) reference for the course** (must be written in full according to the scientific documentation method)	Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N. and Diakopoulos, N. (2016). Designing the User Interface: Strategies for Effective Human-Computer Interaction, 6th ed. Boston: Pearson.
	Other References	Norman, D. A. (2013). The Design of Everyday Things, Revised and Expanded Edition. New York: Basic Books. Dix, A., Finlay, J., Abowd, G. D. and Beale, R. (2004). Human-Computer Interaction, 3rd ed. Harlow: Pearson Prentice Hall. Nielsen, J. (1993). Usability Engineering. San Francisco: Morgan Kaufmann. Rogers, Y., Sharp, H. and Preece, J. (2023). Interaction Design: Beyond Human-Computer Interaction, 6th ed. Hoboken: Wiley.
	Electronic Sources (Links must be added)	Nielsen Norman Group – UX research articles: https://www.nngroup.com/articles/ W3C Web Content Accessibility Guidelines (WCAG): https://www.w3.org/WAI/standards-guidelines/wcag/ Interaction Design Foundation – HCI literature: https://www.interaction-design.org/literature MDN Web Docs – HTML, CSS and JavaScript reference: https://developer.mozilla.org/
	Learning Platforms (Links must be added)	Course page on the Faculty e-learning platform: _____
	Other (to be mentioned)	CS331 course textbook, lecture slides, weekly question banks and course sheet prepared by the course coordinator.

Supportive facilities & equipment for teaching and learning *	**Devices/Instruments**	Data show projector; laboratory PCs; smartphones for mobile usability testing; stopwatch.
	Supplies	Paper, sticky notes and markers for paper prototyping; whiteboard.
	Electronic Programs	Web browser (Chrome / Edge) with developer tools; Visual Studio Code; Figma (free education plan) for wireframes.
	Skill Labs/ Simulators	Faculty computer laboratories, used for prototyping and usability-test sessions.
	Virtual Labs	Online accessibility checkers, e.g. WebAIM Contrast Checker: https://webaim.org/resources/contrastchecker/
	Other (to be mentioned)	Volunteer participants (students and staff) for usability testing.

*** The list mentioned is an example, the institution may add and/or delete depending on the nature of the course**

Name and Signature **Course Coordinator** ***Dr. Waleed Abd Elkhalik***	**Name and Signature** **Program Coordinator**

Human Computer Interaction

CS331 · A Twelve-Week Course for Third-Year Computer Science · Faculty of Computers and Information, Tanta University

Table of Contents

Preface 5
About this book 5
Course details 6
Course map 6
How to use this book 7
Notation and conventions 8
A note on evidence 8
A note for students of computer science 8
Acknowledgements and further reading 9
Week 1 — People and Machines: The Foundations of Interaction 10
2025/2026 2025/2026 2025/2026
1.1 What the subject is 10
1.2 The interaction cycle 11
1.3 The two gulfs 12
1.4 Allocating the task 13
1.5 Human error, properly divided 13
1.6 The disciplines HCI draws on 15
1.7 Context is part of the design 15
1.8 Why this matters 16
1.9 What you do with this in the lab 17
Self-check 18
Week 2 — Human Factors I: Perception and Attention 19
2.1 Why perception comes first 19
2.2 What the eye is good at 20
2.3 Pre-attentive processing 20
2.4 The Gestalt laws of grouping 21
2.5 Colour 23
2.6 Attention 24

رومانی صبحی رزق جرجس مطر

2.7 How people actually read 24
2.8 Six rules, and where each comes from 25
Self-check 27
Week 3 — Human Factors II: Memory, Learning and Mental Models 28
3.1 The three stores 28
3.2 Chunking 29
3.3 Recognition beats recall 29
3.4 Mental models 30
3.5 Errors that come from memory limits 31
3.6 Learning, and the two curves 32
3.7 Designing for memory 33
Self-check 35
Week 4 — Usability: Motivation, Definition and Measurement 36
4.1 The problem with “easy to use” 36
4.2 What usability means 36
4.3 The components conflict 37
4.4 Writing a usability requirement 38
4.5 The four metrics 39
4.6 Arguing for usability 41
4.7 Why early 42
4.8 Accessibility is part of usability 42
Self-check 44
Week 5 — Principles of Interaction and Interface Design Issues 45
5.1 Why principles, and what they are for 45
5.2 Visibility 45
5.3 Feedback 46
5.4 Affordance and signifiers 47
5.5 Mapping 47
5.6 Constraints 48
5.7 Consistency 49
5.8 Shneiderman’s eight golden rules 50
5.9 The aesthetic-usability effect 51
5.10 How to critique 52
Self-check 53

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Week 6 — Command Languages, Menus and Form Fill-in 54
6.1 The five interaction styles 54
6.2 Command languages 55
6.3 Menus 56
6.4 Forms 57
Self-check 61
Week 7 — Windows, Icons and Direct Manipulation 62
7.1 WIMP 62
7.2 Direct manipulation 63
7.3 Windows 64
7.4 Icons 65
7.5 Object then action 67
7.6 Drag and drop 67
Self-check 69
Week 8 — Error Messages, Response Time and Feedback 70
8.1 Response time 70
8.2 Perceived duration 71
8.3 Error messages 72
8.4 Severity 73
8.5 Prevent, accept, then report 74
8.6 Feedback beyond errors 75
Self-check 76
Week 9 — Physical Interaction: Devices, Fitts's Law and Interaction Techniques 77
9.1 Devices 77
9.2 Fitts's law 78
9.3 Target sizes 80
9.4 Reach 80
9.5 Gestures 81
9.6 Ergonomics 82
9.7 Choosing a technique 83
Self-check 84
Week 10 — The Design Process and User Models 86
10.1 User-centred design 86
10.2 Why your intuition is unreliable 87

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

10.3 Gathering requirements 88
10.4 Task analysis 89
10.5 Personas 90
10.6 Scenarios 90
10.7 Putting it together 91
Self-check 92
Week 11 — Interface Evaluation: Rapid Prototyping and Iterative Refinement 94
11.1 Prototyping 94
11.2 The iteration loop 95
11.3 Running a usability test 96
11.4 Heuristic evaluation 97
11.5 Severity 98
11.6 Reporting 99
11.7 Iterating 99
Self-check 100
Week 12 — Natural Language and Voice Interfaces, and Text-to-Speech 101
12.1 The stages of a voice interaction 101
12.2 Two kinds of error 102
12.3 When voice is the right choice 103
12.4 The invisibility problem 104
12.5 Confirmation 105
12.6 Writing for the ear 106
12.7 Text-to-speech and accessibility 106
12.8 The course, end to end 107
Self-check 109
Appendix A — The Design Checklists 110
A.1 Diagnosing a failure *(Ch. 1)* 110
A.2 Display and layout *(Ch. 2)* 110
A.3 Memory *(Ch. 3)* 110
A.4 Usability requirements *(Ch. 4)* 111
A.5 The principles *(Ch. 5)* 111
A.6 Menus and forms *(Ch. 6)* 111
A.7 Windows, icons, direct manipulation *(Ch. 7)* 112
A.8 Feedback and errors *(Ch. 8)* 112

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

A.9 Physical interaction *(Ch. 9)* 112
A.10 Design process *(Ch. 10)* 113
A.11 Evaluation *(Ch. 11)* 113
A.12 Voice and accessibility *(Ch. 12)* 113
Appendix B — Numbers, Formulas and Templates 115
B.1 The numbers worth knowing 115
B.2 Fitts's law 115
B.3 Problems found by *n* participants 116
B.4 The usability requirement template 116
B.5 The critique template 117
B.6 The usability finding template 117
B.7 Severity scale 117
B.8 Nielsen's ten heuristics 117
B.9 Shneiderman's eight golden rules 118
B.10 Norman's principles and stages 118
B.11 Error types and remedies 118
B.12 Choosing an interaction technique 119
B.13 Response-time and waiting indicators 119
B.14 Diagnostic quick-reference 119
Appendix C — Glossary 121

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Preface

About this book

This is the complete text for **CS331 Human Computer Interaction**, a third-year course in the Computer Science department at the Faculty of Computers and Information, Tanta University. It assumes the Algorithms Analysis and Design prerequisite and working programming ability, and nothing else. Laboratory work uses **HTML, CSS and JavaScript**, because a web page is the fastest way to put a working interface in front of a person — not because the course is about web development.

The book has twelve chapters, one for each teaching week, preceded by a course overview (Chapter 0). Each chapter can be read alone, but the course is a single continuous argument and reading in order is strongly recommended.

Key Idea

> The argument of the whole course, in one sentence:
>
> **An interface is a claim about a person — and the claim can be tested.**
>
> Chapters 1–3 establish what the person is like. Chapters 4–8 establish how to judge the claim. Chapters 9–12 establish how to produce one and find out whether it was right.

Course details

Code	CS331
Title	Human Computer Interaction
Level	Third year
Department	Computer Science
Faculty	Computers and Information, Tanta University
Weekly contact	2 hours lecture · 2 hours laboratory
Prerequisite	Algorithms Analysis and Design
Language of instruction	English
Laboratory tools	HTML · CSS · JavaScript · paper · a stopwatch

Assessment

2025/2026 2025/2026 2025/2026

Component	Weight
Student activities and laboratory work	15%
Midterm examination (Week 7)	25%
Practical examination (the project and its evaluation)	10%
Final examination	50%

Course map

Week	Chapter	What you can do afterwards
0	Course Overview	see the whole arc and how each week depends on the last
1	People and Machines	locate a failure in the interaction cycle; name its gulf and error type
2	Perception and Attention	use pre-attentive coding and Gestalt grouping deliberately
3	Memory and Mental Models	convert memory tasks into perception tasks; diagnose mode errors

Week	Chapter	What you can do afterwards
4	Usability	turn "easy to use" into a requirement that can fail a test
5	Principles of Interaction	critique an interface by naming a principle and a change
6	Command Languages, Menus and Forms	structure a menu; design a form people can finish
7	Windows, Icons and Direct Manipulation	judge when direct manipulation is the wrong choice
8	Errors, Response Time and Feedback	write a message that enables recovery; design for waiting
9	Physical Interaction	apply Fitts's law; specify target sizes; identify ergonomic risk
10	The Design Process and User Models	run a contextual inquiry; write personas and scenarios that work
11	Interface Evaluation	run a usability test, rank findings, and iterate
12 2025/2026	Natural Language and Voice Interfaces 2025/2026	design for a channel with no visibility; write for the ear 2025/2026

رومانی صبحی رزق جرجس مطر

How to use this book

Each chapter follows the same structure:

- **Learning outcomes** — what you should be able to *do*, not merely recognise.
- **Numbered sections** with worked examples, real cases and figures.
- **Call-out boxes**: *Key Idea* (the point to remember), *Common Mistake* (what goes wrong), *Definition*, *Note*, *Tip*.
- **Lab activity** — the two-hour practical for that week.
- **Summary** — a compressed revision list.
- **Self-check** — short questions whose full solutions are in that week's question bank.

> **Tip**
>
> **The single most effective way to study this material is to watch people use things.**
>
> Reading that users do not read is not the same as sitting beside someone who scrolls past the instruction you wrote and then asks you what to do. Ten minutes of silent observation teaches more than an hour of revision, and it is the only way to make the findings in Chapters 2 and 3 feel true rather than merely plausible.
>
> From Week 1, keep the **failure log** described in §0.9. By the midterm you should have thirty entries. They will be worth more than any set of notes, because they are yours.

Notation and conventions

- Sizes are given in **CSS pixels** unless stated otherwise.
- Contrast ratios follow the WCAG definition; the body-text threshold used throughout is **4.5:1**.
- Times are in seconds or milliseconds, and the response-time thresholds (0.1 s, 1 s, 10 s) are used consistently from Chapter 8 onwards.
- In Fitts's law, D is the distance to the target and W its width **along the direction of movement**.
- Code is HTML, CSS or JavaScript, written to run without a build step.
- Where a figure shows a comparison, **orange** marks what is changing or at risk, **green** the improved version, **red** a failure, and **grey** what is inactive.
- Quantities such as "about 8% of men" and "roughly 44 px" are given to convey an order of magnitude and a design consequence, not as specifications.

A note on evidence

This subject sits between engineering and psychology, and the quality of its claims varies. It is worth knowing which is which.

Well established, with decades of replication: the limits of working memory; the recognition/recall asymmetry; pre-attentive processing; Fitts's law; the response-time thresholds; the existence and general shape of the Gestalt grouping effects.

2025/2026 2025/2026 2025/2026

Well supported but contested in detail: the "five users find 85% of problems" figure, which depends on an assumption about problem frequency that varies by system; specific minimum target sizes, which differ between platform guidelines; the exact proportion of words read on a page.

Design guidance rather than finding: most of the specific rules in Chapters 5–8. They are compressed conclusions from the findings above, and they are reliable in ordinary cases and wrong in unusual ones.

> **Key Idea**
>
> Knowing which category a claim belongs to is part of the subject. A rule you can justify from a finding is one you can also defend, adapt, and knowingly break. A rule you have only memorised will be applied where it does not belong.
>
> When a rule and a real user disagree, **the user is right**, and the rule was being misapplied.

A note for students of computer science

It is tempting to treat this course as the soft one. Two things are worth saying against that.

First, the standard of evidence is the same as in your other subjects. You already know that an algorithm's cost is measured rather than asserted. This course applies exactly that standard to interfaces: "this design is better" is a claim, and a claim needs a completion rate, a time on task, an error count. Students who bring that habit from algorithms do well here, and students who expect to be graded on opinion do not.

Second, you are the people who will build these systems. The decisions that determine whether an interface is usable — what to show, what to require, what happens when something goes wrong, what the error message says — are made by developers, usually early, usually without anyone calling them design decisions at the time. They will be made by you whether or not you have thought about them.

The examples in this book include a ticket machine, an infusion pump, an alarm panel and an examination system. That range is deliberate. Once a task carries a consequence, the quality of its interface stops being a matter of taste.

Acknowledgements and further reading

The treatment of the interaction cycle, the two gulfs, affordances and everyday error follows Donald Norman's *The Design of Everyday Things*. The golden rules and the direct-manipulation criteria follow Ben Shneiderman's *Designing the User Interface*. The usability components, the discount-usability argument and heuristic evaluation follow Jakob Nielsen's *Usability Engineering*. The response-time thresholds are due to Robert Miller (1968) and were popularised by Nielsen. Fitts's law is from Paul Fitts (1954). The treatment of human error follows James Reason's *Human Error*. Card, Moran and Newell's *The Psychology of Human-Computer Interaction* is the origin of the information-processing model used in Chapter 2.

Students who wish to go further should read Norman first — it is short, it is about doors and cookers as much as computers, and it will change how you look at every object you use for the rest of your life.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Week 1 — People and Machines: The Foundations of Interaction

Learning Outcomes

By the end of this week you should be able to:

1. Define human-computer interaction and say what distinguishes it from software engineering.
2. Describe Norman's seven-stage interaction cycle and identify where a given failure occurs.
3. Explain the gulf of execution and the gulf of evaluation, and name what bridges each.
4. Allocate a task sensibly between a person and a machine, given what each is good at.
5. Distinguish slips, mistakes and violations, and give the appropriate remedy for each.
6. Explain why "user error" is a category of design problem rather than an explanation.
7. State the disciplines HCI draws on and what each contributes.

1.1 What the subject is

Definition

Human-computer interaction is the study of how people use computing systems, and the design of systems that people can use. It is concerned with the loop between the two: what the person intends, what they can do, what the system does, and what the person can tell about it afterwards.

The field has an origin worth knowing. During the Second World War, aircraft crashed for reasons that could not be explained by mechanical failure or by pilot incompetence. Skilled pilots retracted the landing gear after touchdown instead of the flaps — repeatedly, in some aircraft and not in others.

The investigation found the cause in the cockpit rather than in the pilot: the two levers were identical in shape and adjacent in position. Pilots changing aircraft type carried a habit across, and the design allowed the habit to be fatal. The remedy was to give each control a distinctive shape — a wheel for the gear, a flap-shaped paddle for the flaps.

Key Idea

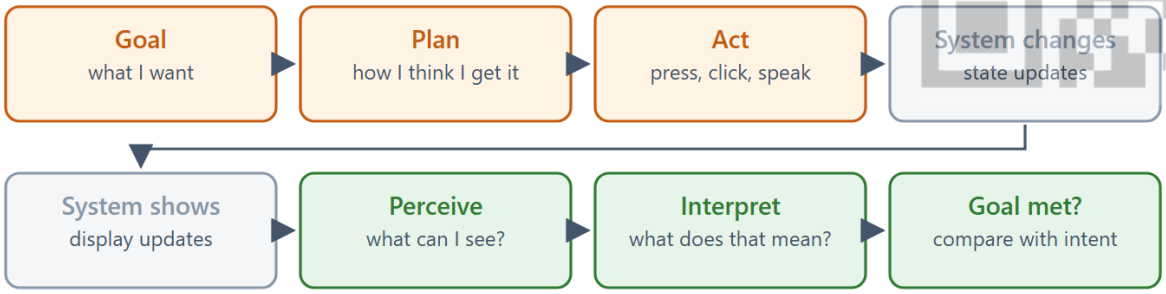
That investigation established the principle this course is built on:

when a competent person repeatedly makes the same error, the error is a property of the design, not of the person.

Everything else follows. If the cause were carelessness, there would be nothing to study and nothing to fix.

رومانی صبحی رزق جرجس مطر

1.2 The interaction cycle



Norman's seven stages: three of doing, three of evaluating, around one machine. Every stage can fail.

Norman's seven stages: three of doing, three of evaluating.

Donald Norman's model breaks an interaction into seven stages, and its value is diagnostic: given a failure, it tells you *where* to look.

Stage	The person asks
1. Form the goal	what do I want?
2. Form the intention	what will achieve it?
3. Specify the action	which control does that?
4. Execute	*(do it)*
5. Perceive the state	what can I see now?
6. Interpret	what does that mean?
7. Evaluate	did I get what I wanted?

2025/2026 2025/2026 2025/2026

Worked Example — Locating a failure in the cycle

Case. A student wants to drop a course. They open the university portal, click **My Courses**, look at the list, click **Edit**, see no way to remove a course, and give up.

Where did it fail? Not at stage 1 — the goal was clear. Not at stage 4 — every click worked. It failed at **stage 3**: the person could not find an action that corresponded to their intention.

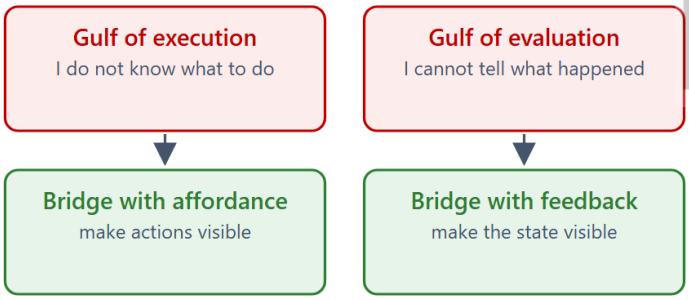
What that tells you. The fix is not a better confirmation dialog, nor a faster page, nor a redesign of the list. It is to make the "drop" action visible at the point where a student looks for it. Identifying the stage identified the class of fix.

A different case, same symptom. A student clicks **Drop**, the page reloads, the course is still listed, and they click **Drop** three more times. Here every stage up to 4 succeeded; the failure is at **stage 5–6** — the system gave no perceivable feedback that the request was received. The fix is a status message, not a navigation change.

Two failures that look identical from the help desk ("the drop function does not work") are different problems with different remedies. The cycle is what separates them.

رومانی صبحی رزق جرجس مطر

1.3 The two gulfs



Two gaps between what a person intends and what a machine offers. Design is the work of narrowing them.

Two gaps between intention and machine.

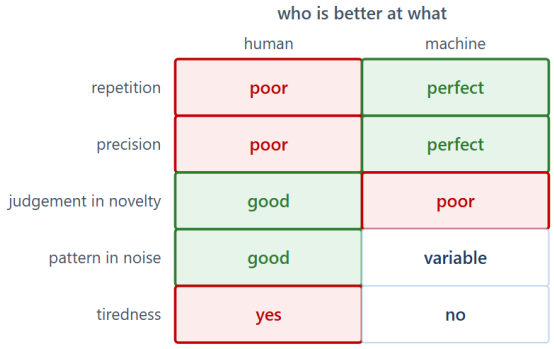
> **Definition**
>
> The **gulf of execution** is the gap between what a person wants to do and what the system lets them do — the difficulty of finding and performing the right action. *(Stages 2–4.)*
>
> The **gulf of evaluation** is the gap between the state the system is in and the person's ability to perceive and interpret it. *(Stages 5–7.)*

Every interface problem in this course sits in one gulf or the other, and the bridges are different:

- **Execution is bridged by visibility, affordance, mapping and constraints** — make the possible actions apparent and the right one easy.
- **Evaluation is bridged by feedback** — make the state of the system apparent without the person having to work it out.

> **Common Mistake**
>
> Teams overwhelmingly attend to the execution gulf, because it is the one users complain about. The evaluation gulf produces quieter failures: the person does the right thing, cannot tell that it worked, and does it again.
>
> Duplicate orders, double payments and repeated submissions are almost always evaluation-gulf failures. Nobody reports them as usability problems; they arrive as data-integrity bugs.

1.4 Allocating the task



Allocate the task to whichever side is suited to it. Most bad systems give the human the machine's job.

Neither side is better at everything.

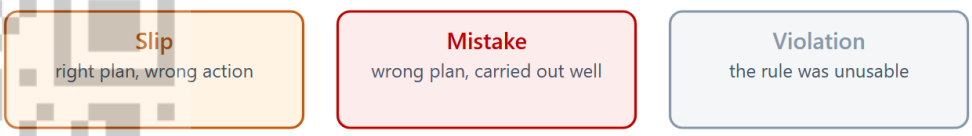
A system design decides, implicitly or explicitly, which part of a task the person does and which part the machine does. Making that decision explicitly is one of the oldest contributions of human factors.

Humans are better at	Machines are better at
judgement in unfamiliar situations	exact repetition, indefinitely
detecting patterns in noisy data	precise calculation and storage
improvising when a rule does not fit	monitoring without attention lapses
understanding intent and context	responding in milliseconds

2025/2026 2025/2026 2025/2026

> **Key Idea**
>
> **Most bad systems give the human the machine's job.**
>
> Copying a number from one screen to another, checking that two lists match, remembering a code between steps, watching an unchanging display for a rare event — these are tasks at which people are poor and machines are excellent, and a system that assigns them to a person will generate errors indefinitely.
>
> When you find yourself designing a step that requires a person to be accurate and tireless, you have found a defect, not a training problem.

1.5 Human error, properly divided



Three kinds of failure with three different fixes. Calling all of them "user error" hides the difference.

Three failures, three remedies.

رومانى صبحى رزق جرجس مطر



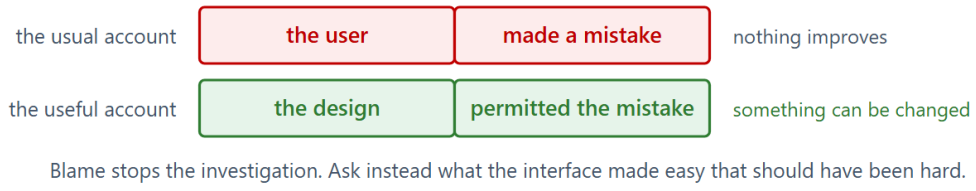
"User error" is not an explanation. It is a label placed on a failure at the point where investigation stopped. Useful analysis divides it:

> **Definition**
>
> A **slip** is a failure to execute a correct plan — the right intention carried out wrongly. Typing the wrong adjacent key; clicking Delete when you meant Save.
>
> A **mistake** is a failure of the plan itself — the wrong intention, executed perfectly. Confidently using the wrong function because you misunderstood what it does.
>
> A **violation** is a deliberate departure from the prescribed procedure, usually because the procedure obstructs the actual work.

The remedies are genuinely different, which is why the distinction matters:

Failure	Cause	Remedy
Slip	attention, similarity, proximity	separate similar controls; distinguish them; make the action reversible
Mistake	a wrong mental model	improve feedback and labelling so the correct model forms (Week 3)
Violation 2025/2026	an unusable procedure 2025/2026	fix the procedure, not the person 2025/2026

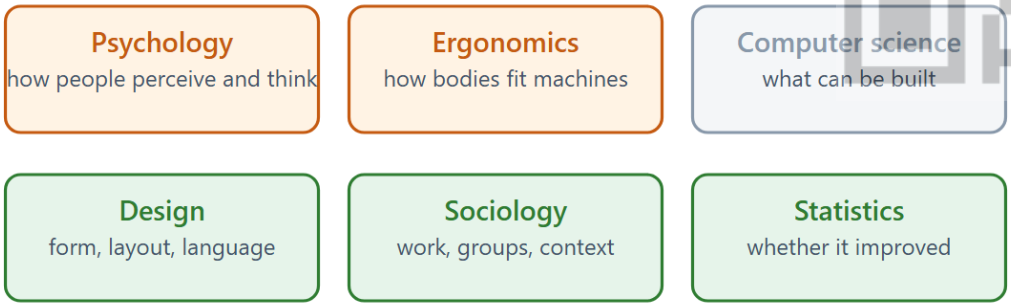
> **Common Mistake**
>
> **Violations are the most misdiagnosed.** When staff routinely bypass a safety step — sharing a password, leaving a door wedged, skipping a confirmation — the reflex is disciplinary. The finding, almost always, is that following the rule made the job impossible or absurdly slow.
>
> Treat a widespread violation as a report that the design is wrong. A rule that is never followed is not a control, however firmly it is written down.



Blame ends the investigation; design continues it.

رومانی صبحی رزق جرجس مطر

1.6 The disciplines HCI draws on



HCI is not a branch of programming. It borrows its methods from the sciences of people.

Where the methods come from.

Discipline	Contributes
Cognitive psychology	perception, attention, memory, learning — Weeks 2–3
Ergonomics / human factors	physical fit, reach, posture, device design — Week 9
Computer science	what can be built, and at what cost
Design	layout, typography, language, form
Sociology and anthropology	how work is really organised, and by whom
Statistics	whether a measured improvement is real — Week 11

2025/2026 2025/2026 2025/2026

> **Note**
>
> This is why the subject cannot be reduced to a list of rules. Rules are the compressed output of findings from these fields; when a rule and a real user disagree, the user is right and the rule was being misapplied.
>
> A student who learns only the rules will apply them where they do not belong. A student who learns why they exist will know when to depart from them.

1.7 Context is part of the design



The same screen is excellent in an office and dangerous in a moving vehicle. Context is part of the design.

The same screen, three contexts.

رومانی صبحی رزق جرجس مطر

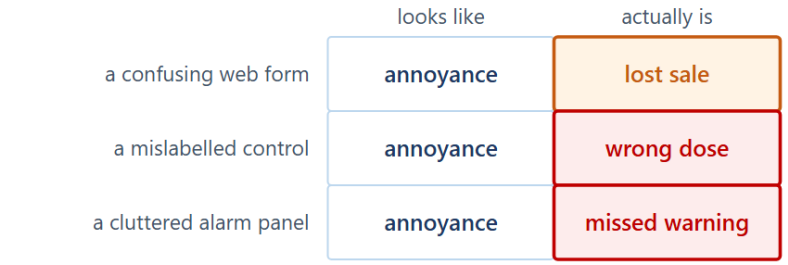
An interface is not good or bad in the abstract. It is good or bad **for a particular person, doing a particular task, in a particular situation.**

- **Who.** A first-time user and a clerk who performs the task 300 times a day need almost opposite things. So does a user with low vision, a tremor, or one free hand.
- **Where.** A desk, a factory floor, bright sunlight, a moving vehicle, a hospital ward at night.
- **Under what conditions.** Time pressure, noise, interruption, stress, fatigue — all of which reduce the capacity available for anything other than the task.

> **Key Idea**
>
> Stress and time pressure do not merely make people slower. They narrow attention, reduce working memory, and push behaviour towards well-learned habits — which is precisely when a familiar habit captures an unfamiliar action.
>
> An interface that is fine in a usability laboratory can therefore be dangerous in the situation it was built for. This is why Week 10 insists on studying the real context rather than imagining it.

1.8 Why this matters

2025/20



Interface quality is not a matter of taste once the task carries a consequence.

Once a task carries a consequence, interface quality stops being taste.

Three well-documented classes of accident, all of them interface failures:

Infusion pumps. A nurse enters a dose. A keying slip — a decimal point in the wrong place — delivers ten times the intended amount. Pumps now enforce dose limits per drug, which is a *constraint* (Week 5) rather than a warning.

Alarm flooding. In a plant or an intensive-care unit, dozens of alarms sound at once during an incident. Staff silence them to think, and the one alarm that mattered is silenced with the rest. The failure is in prioritisation and display, not in attention.

Ambiguous confirmation. A user cannot tell whether a payment succeeded, so they submit again. The bank records two payments. Nobody involved made a mistake in any ordinary sense — the evaluation gulf was not bridged.

> **Note**
>
> Notice what these have in common. In each case the system worked exactly as specified, the person was competent, and the outcome was still wrong. That gap — between a correct

implementation and a bad outcome — is the territory of this subject, and it is not covered by any other course in your degree.

1.9 What you do with this in the lab

The analysis you will produce all term has three parts, and this week establishes all three:

1. **Where in the cycle did it fail?** Stage 3 and stage 6 failures look identical to a help desk and need opposite fixes.
2. **Which gulf?** Execution failures need visibility; evaluation failures need feedback.
3. **Slip, mistake or violation?** Each has its own remedy, and none of them is "train the user".

Tip

Write your failure-log entries in this form from now on:

Ticket machine, Tanta station. I wanted a return ticket. I pressed "Tickets", then looked for "return" and could not find it — it was inside "Single" as a checkbox. ***Stage 3, execution gulf.*** *Fix: show return as a first-class option at the same level as single.*

Three sentences. By Week 5 you will add the principle name, and by Week 11 you will be testing whether your proposed fix actually worked.

Lab Activity

Lab 1 (2 hours)

1. **The cycle.** In pairs, take turns performing an unfamiliar task on a phone or website (suggestions: change a privacy setting, cancel a subscription, find a refund policy). The observer says nothing and records where in the seven stages each hesitation occurred.

2. Write up three failures, naming the stage and the gulf for each.

3. **Task allocation.** Take a paper form used at the Faculty. List every step; mark each as suited to a human or a machine. Identify at least two steps currently done by a person that should not be.

4. **Error classification.** Collect five real errors from your group's own experience this week. Classify each as slip, mistake or violation, and give the remedy that follows.

5. **The two doors.** Photograph two doors on campus — one you can use correctly on sight, one you cannot. Explain the difference in terms of the execution gulf. *(Keep these; we return to them in Week 5.)*

6. **Project.** Form groups of three or four. Bring three candidate tasks with a one-sentence description of who performs each and how often.

Summary

• **HCI studies the loop** between person and system; it exists because competent people fail systematically when a design permits it.

• **Norman's seven stages** — goal, intention, action, execution, perception, interpretation, evaluation — locate *where* a failure happened, which determines the class of fix.

• The **gulf of execution** ("I do not know what to do") is bridged by visibility, affordance, mapping and constraints; the **gulf of evaluation** ("I cannot tell what happened") is bridged by feedback.

رومانى صبحى رزق جرجس مطر



- Evaluation-gulf failures are quieter and produce duplicate submissions rather than complaints.
- **Allocate tasks** to the side suited to them. Most bad systems give the human the machine's job — accuracy without attention.
- **Slips** (wrong execution), **mistakes** (wrong plan) and **violations** (unusable rule) have three different remedies, none of which is blame.
- HCI borrows from **psychology, ergonomics, design, sociology and statistics**; the rules are compressed findings, not axioms.
- An interface is good **for a specified who, task and context** — never in the abstract.

Self-check

1. Define HCI in one sentence, and say what distinguishes it from software engineering.
2. List the seven stages of the interaction cycle in order.
3. A user clicks Submit three times because nothing appears to happen. Which stage failed, which gulf, and what is the fix?
4. Give a task humans do better than machines, and one machines do better, with a reason for each.
5. Distinguish a slip from a mistake with an example of each.
6. Why is a widespread violation a design finding rather than a discipline problem?
7. Why can an interface that performs well in a laboratory fail in its real setting?
8. What exactly is wrong with the explanation "the user made an error"?

(Full solutions are in the Week 1 question bank.)

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Week 2 — Human Factors I: Perception and Attention

Learning Outcomes

By the end of this week you should be able to:

1. Describe the human information-processing pipeline and identify the limit at each stage.
2. Explain pre-attentive processing and use it deliberately in a display.
3. Apply the Gestalt grouping laws to diagnose and improve a layout.
4. Explain why colour must never be the sole carrier of meaning, and what to add.
5. State how people actually read on screens and write accordingly.
6. Explain why attention is a scarce resource and judge whether an interruption is justified.
7. Derive six concrete display rules from the perceptual findings behind them.

2.1 Why perception comes first

An interface communicates only through the senses. Whatever the system knows, the person receives it as light on a retina or pressure waves in an ear — and those channels have measurable, well-documented limits.



A rough model, and a useful one: every stage has a limit, and every limit shows up in an interface.

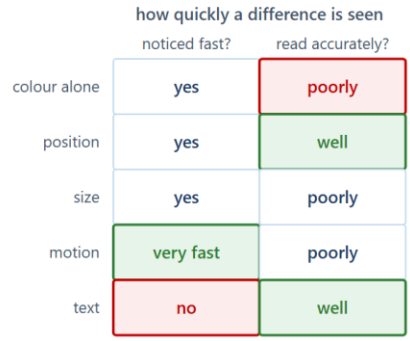
A rough model, and a useful one.

Key Idea

Every rule in this week is a *consequence* of a property of the visual system, not a matter of style. That is what distinguishes the advice in this course from the advice in a design blog: you can say why it is true, and you can say when it does not apply.

رومانى صبحى رزق جرجس مطر

2.2 What the eye is good at



Colour and motion attract attention; position and text carry the value. Use each for the job it can do.

Different visual properties do different jobs.

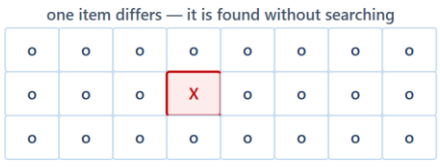
The visual system is not a camera. It is a system tuned by evolution for particular tasks, and it is excellent at some things and poor at others.

Property	Noticed quickly?	Judged accurately?
Position	yes	**yes**
Length	yes	yes
Motion	extremely	no
Colour	yes	**no**
Area / volume	yes	poorly
Text	no	yes

2025/2026 2025/2026 2025/2026

> **Note**
>
> **The practical consequence for any display of data.** Position and length are the only channels people read *accurately*, which is why a bar chart communicates quantity better than a pie chart and far better than a colour scale.
>
> Colour and motion are the channels that *attract* attention without being read accurately. Use them to say “look here”, never to say “this value is 27”.

2.3 Pre-attentive processing



Pre-attentive features are seen in under 200 ms, whatever the number of distractors. Use one, never five.

One item differs; it is found without searching.

Definition

A **pre-attentive** feature is one that is detected in under about 200 milliseconds, before conscious attention is directed — and crucially, in a time that does *not* increase with the number of surrounding items.

Colour, orientation, size, curvature, motion and enclosure are all pre-attentive. Searching for a red item among grey ones takes the same time whether there are ten distractors or a hundred. Searching for the *word* "error" among other words does not: that is a serial search, and its cost grows with the list.

Key Idea

This is the mechanism behind every effective status display, and its rule is severe: **the effect only works when one feature is unique.**

A dashboard where five things are red, two flash and three are bold has no pre-attentive channel left. Every emphasis you add reduces the value of the others. The designer's discipline is to decide what is *genuinely* the most urgent item and to spend the effect there alone.

Worked Example — Redesigning an alarm list

Before. A monitoring screen lists 40 servers. Each row shows a status word (OK / WARN / FAIL). All rows are the same size, same weight, same colour. Finding the failed server requires reading 40 rows — a serial search, several seconds, and error-prone under pressure.

A first fix. Colour the FAIL rows red. Now the search is pre-attentive: the eye lands on the red row regardless of list length.

Why it degrades. Six months later, WARN rows are amber, rows with old data are grey, rows recently changed are bold, and the row under the cursor is highlighted. Five visual codes compete, and the search is serial again — slower than the original, because now there is more to filter.

The disciplined version. One pre-attentive code for the single most important distinction (failed / not failed). Everything else is carried by **position** — sort failures to the top — and by **text** for the detail. Position is pre-attentive too, and it does not compete for the same channel as colour.

The general lesson. Emphasis is a budget, not a feature. Spending it in five places is the same as not spending it at all.

2.4 The Gestalt laws of grouping



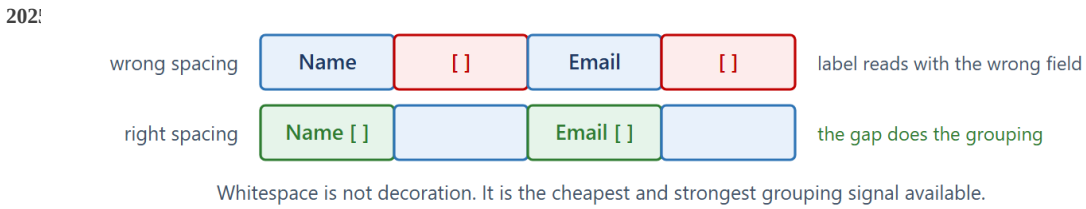
People group things before they read them. A layout makes a claim about structure whether you intended it or not.

People group before they read.

The visual system organises a scene into objects and groups *before* any conscious reading occurs. It does so by a small set of principles identified in the 1920s, which have proved remarkably durable.

Law	Statement	In an interface
Proximity	things that are near are related	spacing between form fields and their labels
Similarity	things that look alike are related	all primary buttons share one style
Enclosure	a border binds what it contains	a card groups related controls
Continuity	the eye follows smooth lines	aligned fields read as one column
Closure	incomplete shapes are completed	a partial border still reads as a box
Common fate	things moving together are one	items animating as a group

> **Key Idea**
>
> **Proximity is the strongest and the cheapest.** It overrides labels: if a label sits nearer to the wrong field than to its own, users will read it with the wrong field no matter what it says.
>
> Whitespace is therefore not decoration. It is the primary structural signal in a layout, and it is free.



The gap does the grouping.

> **Common Mistake**
>
> The most common form layout error is uniform spacing: every element separated by the same gap. This tells the eye that everything is equally related — which is never true — so the user must read every label to recover the structure.
>
> The fix costs nothing: reduce the gap *within* a group and increase it *between* groups. A form with four fields in two logical pairs should show two pairs, not four equidistant rows.
>
> Designers reach for boxes and dividing lines to solve this. Those work, but they add visual noise. Try spacing first; add a border only if spacing alone is not enough.

2.5 Colour



About 8% of men cannot reliably distinguish red from green.

Colour is powerful for attracting attention and poor for conveying value. Three constraints govern its use.

Colour is not read accurately. People cannot reliably order more than about five or six colours by any dimension, and cannot place a colour on a scale. A colour-coded quantity is an approximation at best.

Colour is not universally perceived. Roughly 8% of men and 0.5% of women have some form of colour-vision deficiency, most commonly red–green. A red/green status indicator is invisible as a *distinction* to a substantial fraction of any user population.

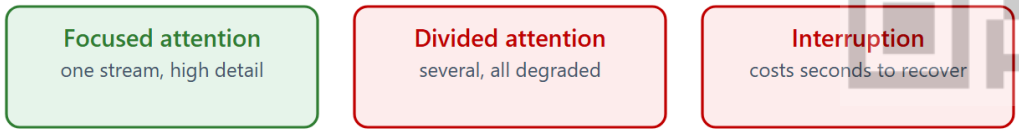
Colour is not stable. It changes with the display, ambient light, viewing angle and calibration. A distinction that is clear on your monitor may vanish on a projector or in sunlight.

2025/2026 2025/2026 2025/2026

> **Key Idea**
>
> **Never encode meaning in colour alone.** Add a second, redundant channel — a shape, an icon, a position, or a word.
>
> The cost is close to zero and the benefit is not confined to colour-blind users: redundant coding also survives a bad projector, a printed copy in greyscale, and bright sunlight on a phone.
>
> **Note**
>
> **Contrast is a separate question from colour**, and a more consequential one. Low-contrast grey text on a white background is currently fashionable and is measurably harder to read, especially for older users — and the population of every long-lived system gets older.
>
> The accessibility guidelines give a concrete threshold: a contrast ratio of at least 4.5:1 for body text. It is checkable by a tool in seconds, and unlike most design questions it has a definite answer.

رومانی صبحی رزق جرجس مطر

2.6 Attention



Attention is a scarce resource you are spending on the user's behalf. Every alert is a withdrawal.

Attention is a scarce resource, and you are spending it.

> **Definition**
>
> **Focused attention** is the allocation of processing to one stream of information. It is largely single-channel: genuine parallel attention to two demanding tasks is not available.

What is commonly called multitasking is rapid switching, and switching has a cost. After an interruption, returning to a complex task takes seconds at best — substantially longer if the interrupted task involved holding intermediate results in memory, which Week 3 explains.

> **Common Mistake**
>
> **Every notification, modal dialog and alert is a withdrawal from the user's attention**, and the interface makes that withdrawal on your instruction.
>
> The test before adding one is not "is this information useful?" — almost everything is potentially useful. It is: **is this worth interrupting a person mid-task for, right now?**
>
> Applied honestly, that test removes most notifications. Systems that fail it train users to dismiss alerts without reading — at which point the genuinely urgent alert is dismissed with the rest. This is the same mechanism as alarm fatigue in Week 1, in a smaller and more familiar setting.

Change blindness is worth knowing about because it contradicts intuition directly: people routinely fail to notice substantial changes to a display if the change coincides with a visual interruption such as a page refresh, a scroll or a blink. A message that appears at the top of a page after a reload may never be seen at all. If a change matters, it must be shown *where the person is looking* and, often, must persist.

2.7 How people actually read



On a screen, people scan in an F-shaped path and read perhaps a quarter of the words. Put the meaning first.

Reading a screen is not reading a page.

Eye-tracking studies are consistent on this point: people scan screens rather than read them. The typical pattern is an F shape — the first line or two read across, then progressively shorter scans, then a vertical run down the left edge. Estimates of the proportion of words actually read on a typical page cluster around 20–30%.

> **Key Idea**
>
> Since the first words of each line and heading receive most of the attention, **put the meaning first**.
>
> Not: *"In order to be able to proceed with your application, it is necessary that you first upload your identification document."*
>
> But: *"Upload your ID to continue."*
>
> Shorter is a side effect. The substance is that the operative word is now in the position that is actually read.

Practical consequences:

- **Headings must be informative**, because for many readers they are the entire content. "Next steps" is read; "Introduction" is skipped.
- **Front-load links and buttons.** "Download the timetable", not "Click here to download the timetable".
- **Prefer lists to paragraphs** for anything procedural.
- **Do not centre body text.** A ragged left edge destroys the vertical scan.

2.8 Six rules, and where each comes from



Six rules, each of which is a direct consequence of how the visual system works — not a matter of style.

Every rule traces to a perceptual fact.

Rule	The finding behind it
Use **one** accent colour for the primary action	pre-attentive effects work only when unique
Group by **spacing** before boxes and lines	proximity is the strongest grouping law
Front-load text	people scan in an F pattern and read ~25%
Code **redundantly** — colour *and* shape	colour vision deficiency; unstable rendering
Interrupt **only** for genuine urgency	attention is single-channel; switching costs
Ensure **contrast** (4.5:1 for body text)	acuity declines with age and with conditions

> **Tip**

When you are asked in an examination or a design review to justify a layout decision, name the finding, not the fashion. “Grouped by proximity so the label is not read with the wrong field” is an argument. “It looks cleaner” is not, and cannot be evaluated by anyone else.

Lab Activity

Lab 2 (2 hours)

1. **Pre-attentive search.** Build two HTML pages showing 60 items. On the first, mark one item with a single unique colour. On the second, use four colours, two weights and an underline. Time five people finding a specified item on each. Report the difference.

2. **Gestalt.** Take a form with six fields from any real site. Photograph or screenshot it, then redraw it changing *only the spacing* to group the fields logically. No new boxes, lines or colours.

3. **Colour audit.** Find an interface that uses red/green status indicators. Convert a screenshot to greyscale (or use a colour-blindness simulator). Report what information is lost, and add a redundant code that recovers it.

4. **Contrast.** Measure the contrast ratio of body text on three websites with a checker. Report which pass 4.5:1. Comment on any that fail.

5. **Reading.** Take a paragraph of instructions from a university page and rewrite it front-loaded, as a list, in under half the words. Give both to two people and ask each to state the required action — time them.

6. **Change blindness.** Show someone a page, have them look away, change one non-trivial element, and ask what changed. Record the result for five people.

7. **Project.** Produce a one-page description of your chosen task and its users. Identify the single most important item on the main screen — this is what will receive your one pre-attentive code.

Summary

• An interface reaches the user only through the senses, and every stage of the perceptual pipeline has a **measurable limit**.

• **Position and length** are read accurately; **colour and motion** attract attention but are not read accurately. Use each for the job it can do.

• **Pre-attentive** features are found in under 200 ms regardless of the number of distractors — but **only while unique**. Emphasis is a budget.

• The **Gestalt laws** govern grouping; **proximity is the strongest** and overrides labels. Whitespace is a structural signal, not decoration.

• **Never encode meaning in colour alone** — about 8% of men have a red–green deficiency, and colour is unstable across displays. Add shape, icon, position or a word.

• **Contrast** has a concrete threshold: 4.5:1 for body text.

• **Attention is single-channel** and switching is costly; every alert is a withdrawal. **Change blindness** means changes are missed unless shown where the person is looking.

• People **scan in an F pattern** and read perhaps a quarter of the words: **front-load the meaning**.

رومانى صبحى رزق جرجس مطر

Self-check

1. Which visual properties are judged accurately, and which merely attract attention?
2. Define a pre-attentive feature. Why does its benefit disappear when five are used at once?
3. State the proximity law and give an interface failure it explains.
4. Why is a red/green status indicator inadequate, and what is the minimal fix?
5. What contrast ratio is required for body text, and why does it matter more over time?
6. What is change blindness, and what does it imply about where to show a confirmation?
7. What is the test for whether a notification is justified?
8. Rewrite front-loaded: "In order to complete your registration, it will be necessary for you to pay the fees."

(Full solutions are in the Week 2 question bank.)

رومانی صبحی رزق جرجس مطر

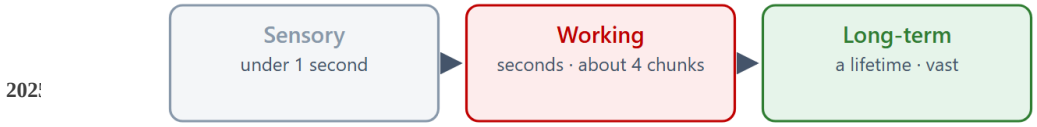
Week 3 — Human Factors II: Memory, Learning and Mental Models

Learning Outcomes

By the end of this week you should be able to:

1. Describe the three memory stores and state the capacity and duration of each.
2. Explain chunking and apply it to codes, numbers and procedures.
3. Explain why recognition is easier than recall, and redesign an interface accordingly.
4. Define a mental model and explain how a user forms one from the system image.
5. Diagnose errors caused by a wrong mental model and propose the correct remedy.
6. Explain the trade-off between learnability and efficiency, and decide which a product needs.
7. Design one interface that serves both novices and experts.

3.1 The three stores



Working memory is the bottleneck of every interface. It is small, brief, and destroyed by interruption.

Capacity and duration differ by orders of magnitude.

Definition

Sensory memory holds raw sensory input for well under a second. You have no conscious access to it and cannot design for it directly.

Working memory (short-term memory) holds the information currently in use. Its capacity is small — around four chunks by current estimates, traditionally quoted as seven — and its contents decay within seconds without rehearsal.

Long-term memory is effectively unlimited in capacity and duration. Retrieval from it is unreliable and slow; recognition is far more reliable than recall.

Key Idea

Working memory is the bottleneck of every interface. It is small, it is brief, and it is destroyed by interruption.

Every time an interface asks a person to carry something from one screen to the next — a code, a total, a decision made three steps ago — it is spending the scarcest resource the user has, and doing so where a machine could have carried it for free.

3.2 Chunking



The same information, twice, with very different costs.

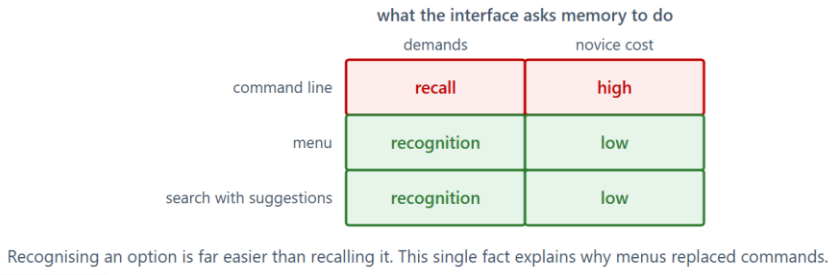
Capacity is measured in **chunks**, not items, and a chunk is whatever the person has learned to treat as one unit. `1 0 6 6` is four chunks for most people and one chunk for a historian; `+20 100 123 4567` is easier to hold than `201001234567` even though it has more characters.

This gives the designer a free improvement, because chunking is something the *display* can do on the user's behalf:

Instead of	Display as
`EG380019284736`	`EG38 0019 2847 36`
`01001234567`	`0100 123 4567`
a 14-step procedure	three named stages of four or five steps

> **Tip**
>
> Chunk the **display** but accept the **unchunked input**. A user typing a card number should be able to type it with spaces, without spaces, or with hyphens, and the field should normalise it silently.
>
> Rejecting `4111 1111 1111 1111` because of the spaces the interface itself displayed is a specific and common form of self-inflicted error, and it is usually one line of code to fix.

3.3 Recognition beats recall



What the interface asks memory to do.

Retrieving an item from long-term memory unaided (**recall**) is slow and unreliable. Confirming that a presented item is the right one (**recognition**) is fast and comparatively accurate. This asymmetry is one of the most robust findings in cognitive psychology, and it explains a great deal of interface history.

> **Key Idea**
>
> **The menu replaced the command line because recognition beats recall.**

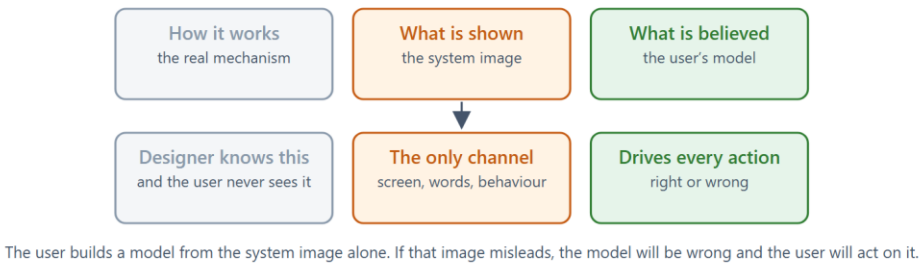
> A command line requires you to recall the command name, its options and their syntax. A menu shows you the options and asks only that you recognise the one you want. That single change is what made computers usable by people who had not been trained on them.

Applications of the principle:

- **Offer, do not ask.** A list, a dropdown or a set of suggestions beats a blank field wherever the set of valid answers is known.
- **Show recent and frequent items.** Most choices repeat; showing the last five converts recall into recognition for the common case.
- **Autocomplete** is recognition applied to an open-ended field.
- **Keep the context visible.** A confirmation screen that shows what you are confirming does not require you to remember it.

> **Common Mistake**
>
> **Placeholder text used as the only label** is a memory defect disguised as a clean design. The hint vanishes the moment the user types, so anyone who is interrupted, or who returns to check a field, must recall what the field was for — and they cannot, because the interface deleted the answer.
>
> It also fails on contrast (Week 2) and on screen readers. Use a persistent label; use placeholders only for an example of the *format*.

3.4 Mental models



The user's model is built from the system image alone.

> **Definition**
>
> A **mental model** is a person's internal explanation of how something works. It need not be correct or complete; it only needs to generate predictions, and the person will act on those predictions.

Three things must be distinguished:

- **The design model** — how the system actually works, known to the engineers.
- **The system image** — everything the system shows: the screen, the labels, the behaviour, the documentation.
- **The user's model** — what the user comes to believe.

The user never sees the design model. **They build their model entirely from the system image**, so if that image is misleading, the resulting model will be wrong and the user will act confidently on it.

Worked Example — The thermostat, and why it matters in software

The wrong model. Many people believe a thermostat is a valve: set it to 30° and the room heats *faster* than at 22°. They therefore set it far above the temperature they want, then open a window when it becomes too hot.

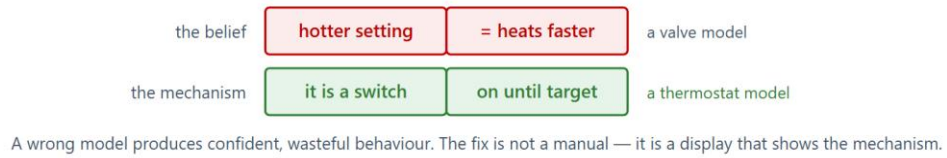
The mechanism. A thermostat is a switch with a target. The heater runs at full output until the target is reached and then stops. Setting 30° does not heat faster; it heats *longer*, past the point the person wanted.

Why the belief persists. Nothing in the interface contradicts it. A dial marked only with numbers is consistent with both models, and the wrong one is more intuitive — most controls in daily life *are* valves (taps, volume, throttle).

The fix is a display, not a manual. Show current temperature, target temperature and whether the heater is running. Now the system image supports one model only, and the wrong belief cannot survive contact with it.

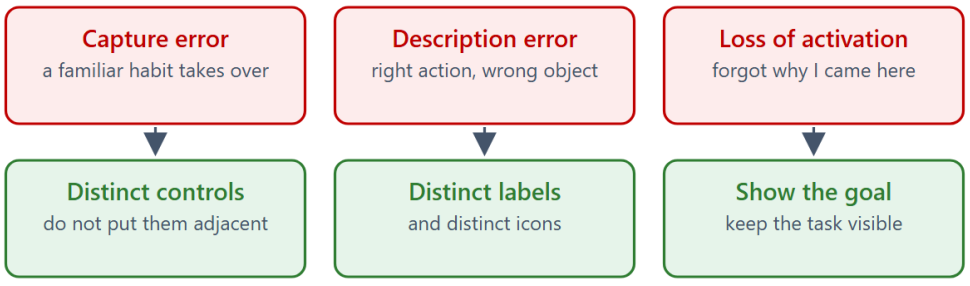
The software version. A user believes that closing an application's window quits it, or that a file in a synced folder is safe as soon as it appears there, or that a "draft saved" message means saved to the server. Each belief is reasonable, unsupported by the interface, and produces a predictable class of data loss. In every case the remedy is to show the state, not to document it.

رومانى صبحى رزق جرجس مطر



A wrong model produces confident, wasteful behaviour.

3.5 Errors that come from memory limits



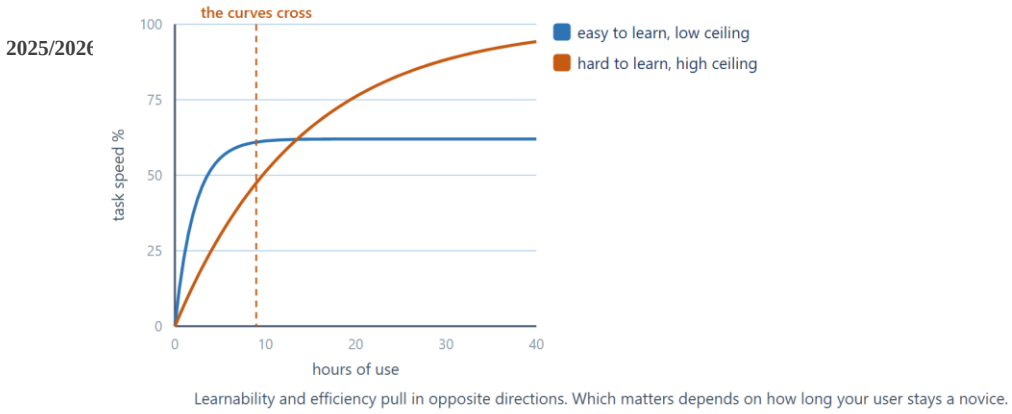
Each has a known remedy.

Error	What happens	Remedy
Capture error	a frequent habit takes over a similar, less frequent action	separate similar sequences; do not place them adjacent
Description error	the right action on the wrong	make targets visually and

Error	What happens	Remedy
	object	spatially distinct
Loss of activation	arriving somewhere and forgetting why	keep the goal visible; show the task in progress
Mode error	the correct action for the *other* mode	show the mode prominently, or remove modes

> **Key Idea**
>
> **Mode errors deserve separate attention** because they are caused by a design decision that is always optional.
>
> A mode is a state in which the same action produces a different result. The classic case is a text editor where typing either inserts characters or executes commands depending on an invisible state — and the classic outcome is a user who types a paragraph into a command interpreter.
>
> Three remedies, in order: **remove the mode**; if it must exist, **display it unmissably**; if it is dangerous, **make it spring-loaded** so it requires continuous effort to remain in (holding a key rather than toggling one).

3.6 Learning, and the two curves



Learnability and efficiency pull against each other.

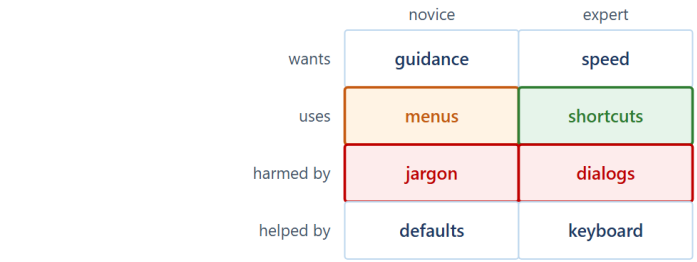
An interface that is easy to learn is not automatically efficient once learned, and the most efficient interfaces are frequently hard to learn.

- **Menus** are discoverable, and slow: each use costs navigation.
- **Keyboard shortcuts** are fast, and invisible: each must be learned separately.
- **A command language** is very fast for an expert and impassable for a novice.

> **Note**
>
> **Which matters depends on the ratio of novices to experts, and on how long a user remains a novice.**
>
> A ticket machine's users are *always* novices — each person uses it a few times a year, and there is no path to expertise. Optimise entirely for learnability.

A radiology workstation's users perform the same task 200 times a day for years. A week of training is affordable; two extra seconds per case is not. Optimise for efficiency.

A code editor sits in between, which is why editors offer menus *and* shortcuts *and* a command palette. Most general software is in this position.



One interface must serve both. The usual answer is a visible path for novices with a fast path beside it.

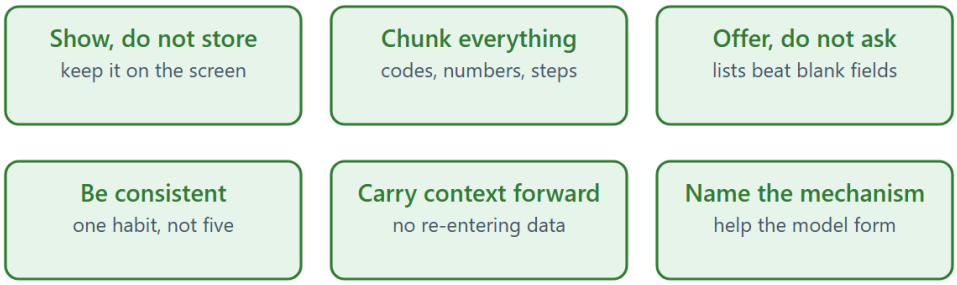
The same interface must serve both.

Key Idea

The standard resolution is a visible path with a fast path beside it. The menu item exists and is discoverable; the shortcut is printed next to it, so that using the slow path teaches the fast one.

That last detail is what makes the pattern work. A shortcut that is never displayed is a shortcut most users will never acquire, because nothing in their daily use ever mentions it.

3.7 Designing for memory



Six rules that each convert a memory task into a perception task — which is where humans are strong.

Six rules, each converting a memory task into a perception task.

Rule	What it prevents
Show, do not store — keep needed information on screen	working-memory decay and interruption loss
Chunk everything — codes, numbers, procedures	exceeding capacity
Offer, do not ask — lists and suggestions over blank fields	unreliable recall

Rule	What it prevents
Be consistent — one habit, not five	capture and description errors
Carry context forward — never ask twice for the same data	transcription slips
Show the mechanism — display state, not just outcomes	wrong mental models

Tip

All six are instances of one strategy: **convert a memory task into a perception task.** People are unreliable at holding information and excellent at seeing it.

When you are stuck on a design problem, ask what the user is currently required to remember, and put it on the screen. It resolves a surprising proportion of cases.

Lab Activity

Lab 3 (2 hours)

1. **Capacity.** Read aloud sequences of random digits of increasing length to five people and record where each fails. Then repeat with the digits chunked in threes. Report the difference in the mean span.

2. **Recognition versus recall.** Build two versions of a country-selection field: a blank text input, and a searchable list. Time ten entries on each and count errors.

3. **Chunking.** Take three real identifiers used at the Faculty (student number, course code, national ID). Propose a chunked display format for each and implement the normalising input handler in JavaScript.

4. **Mental models.** Ask five people to explain what happens when they delete a file on a phone, and where it goes. Record the models. Identify which parts of the system image produced each belief.

5. **Mode errors.** Find an interface with a mode (caps lock, an edit/preview toggle, a drawing tool selection). Document one error it causes and propose the three remedies in order.

6. **Placeholder audit.** Find three forms that use placeholder text as the only label. Fill in half of each form, then describe what a returning user can no longer see.

7. **Project.** List everything your chosen task currently requires the user to remember between steps. For each item, state how the interface will carry it instead.

Summary

• Three stores: **sensory** (<1 s), **working** (~4 chunks, seconds, destroyed by interruption), **long-term** (vast, unreliable retrieval).

• **Working memory is the bottleneck.** Anything the interface asks the user to carry between screens is spending the scarcest resource they have.

• Capacity is measured in **chunks**; chunk the display, and accept unchunked input.

• **Recognition beats recall** — the reason menus replaced command lines. Offer options; show recent and frequent items; keep context visible.

رومانی صبحی رزق جرجس مطر



• **Placeholder text as the only label** is a memory defect: the hint vanishes exactly when it is needed.

• A **mental model** is built from the **system image** alone. A misleading image produces a confident, wrong model — fixed by showing state, not by documentation.

• **Capture, description, loss-of-activation and mode errors** each have a specific design remedy.

• **Learnability and efficiency conflict.** Which dominates depends on whether users ever stop being novices. The usual resolution is a **visible path that teaches the fast path**.

رومانی صبحی رزق جرجس مطر

Self-check

1. Give the approximate capacity and duration of working memory.
2. What is a chunk? Why does chunking a display improve accuracy at no cost?
3. State the recognition/recall asymmetry and explain what it implies about blank text fields.
4. Distinguish the design model, the system image and the user's model. Which does the user see?
5. Give an example of a wrong mental model in software and the display that would prevent it.
6. What is a mode error? Give the three remedies in order of preference.
7. Why is a ticket machine optimised differently from a radiology workstation?
8. Name the strategy that unifies all six memory rules.

2025/2026 2025/2026 2025/2026

(Full solutions are in the Week 3 question bank.)

Week 4 — Usability: Motivation, Definition and Measurement

Learning Outcomes

By the end of this week you should be able to:

1. Define usability and name its five components.
2. Explain why the components conflict and decide which a given product must favour.
3. Turn a vague aspiration into a usability requirement that can pass or fail.
4. Choose and collect the four standard usability metrics.
5. Argue for usability in the terms an organisation actually responds to.
6. Explain why accessibility is part of usability rather than an addition to it.
7. Write the usability goals for your project.

4.1 The problem with “easy to use”

Every specification contains a sentence like *“the system shall be easy to use”*. It is worthless, and it is worth being precise about why.

It is worthless because it **cannot fail**. No test can be run against it, no build can be rejected for violating it, and two reasonable people can disagree about whether it has been met with no way to settle the disagreement. In practice, “easy to use” is decided at the end by whoever is most senior in the room.

Key Idea

Compare with a requirement you already know how to write:

“The search shall return results within 200 ms for a database of 10^6 records.”

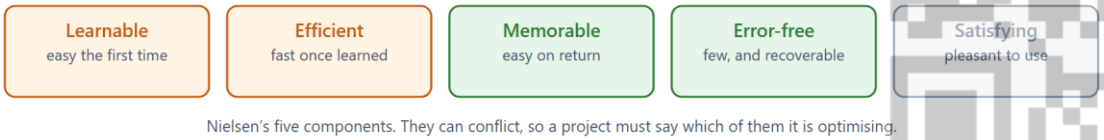
This can be measured, it can fail, and when it fails everyone agrees that it has. This week is about writing requirements of that kind for **usability**.

4.2 What usability means

Definition

Usability is the extent to which a specified user can achieve a specified goal, in a specified context, with **effectiveness**, **efficiency** and **satisfaction**. *(This is the substance of the ISO 9241 definition.)*

Note the three occurrences of “specified”. Usability is not a property of a system. It is a property of a **system, a user, a goal and a context together.**



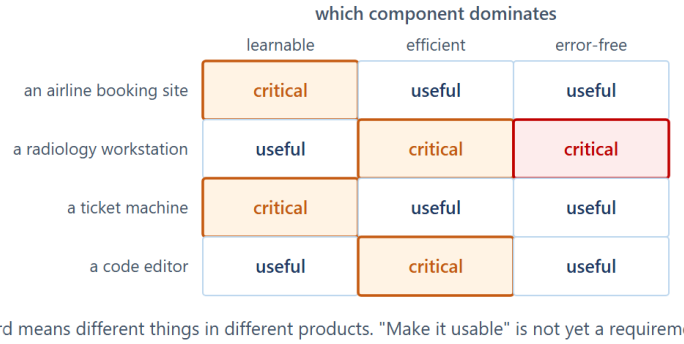
Nielsen's five components.

The most widely used breakdown is Jakob Nielsen's:

Component	Question it answers
Learnability	how easy is it the first time?
Efficiency	how fast once learned?
Memorability	how easy on return after not using it?
Errors	how many, how severe, how recoverable?
Satisfaction	how pleasant is it to use?

> **Note**
>
> **Memorability is the component teams forget**, and it matters more than it appears. Most systems have a core of functions used daily and a long tail used once a month — filing a report, changing a setting, requesting leave.
>
> For that tail, every user is effectively a first-time user every time, and learnability rather than efficiency governs. A system optimised entirely around its daily functions will be painful in exactly the places where users are least confident.

4.3 The components conflict



The same word means different things in different products.

You cannot maximise all five. Increasing learnability usually means more guidance on screen, which slows an expert. Reducing errors usually means confirmations, which cost time. Satisfaction can be raised by animation that reduces efficiency.

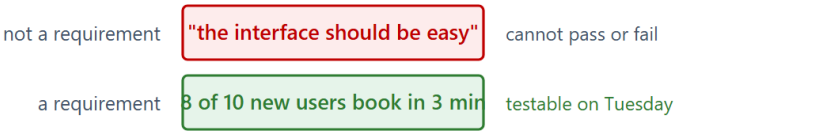
> **Key Idea**

So **“make it usable” is not yet a requirement.** A usable ticket machine and a usable radiology workstation are different objects, optimised for different components, and a design that is excellent as one would be a failure as the other.

The first act of usability work is therefore to decide **which components dominate for this product and this user**, and to write that decision down where the team can see it.

Product	Dominant components	Why
Ticket machine	learnability, errors	every user is a novice; a wrong ticket is costly
Radiology workstation	efficiency, errors	hundreds of cases daily; a missed finding is severe
Tax filing site	learnability, memorability	used once a year, by everyone, under anxiety
Code editor	efficiency, memorability	daily expert use, enormous function set

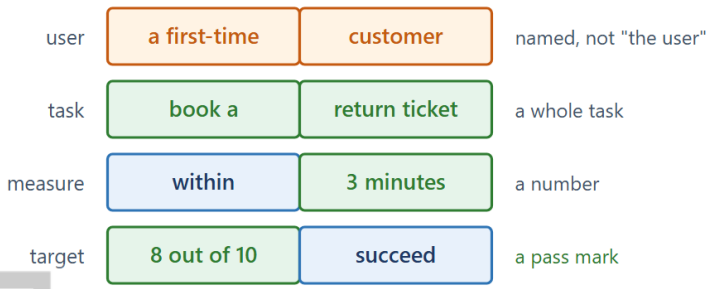
4.4 Writing a usability requirement



A usability goal names the user, the task, the measure and the target. Anything less cannot be verified.

The difference between an aspiration and a requirement.

A usability requirement has four parts, and omitting any one of them makes it untestable.



Fill the four rows and you have a requirement an engineer can build against and a tester can check.

Four rows, and you have something an engineer can build against.

Definition

User — which user, described specifically enough to recruit five of them. **Task** — a complete task with a definite end. **Measure** — what will be counted or timed. **Target** — the value that counts as passing, and for what proportion of users.

Worked Example — Turning three aspirations into requirements

Aspiration 1: *"The booking site should be easy to use."*

Requirement. Eight of ten first-time users, who have not seen the site before, shall complete a return booking between two named cities within four minutes, unaided, with no more than one error.

Aspiration 2: *"Registration should be quick."*

Requirement. A returning student who has registered at least once before shall complete registration for five courses within three minutes, measured from login to confirmation screen, for at least nine of ten participants.

Aspiration 3: *"The error messages should be helpful."*

Requirement. After encountering a validation error, eight of ten participants shall correct the input on their first subsequent attempt, without asking for help and without consulting documentation.

What changed in each case. The user is now specific enough to recruit; the task has a definite end; the measure is a number someone can collect on a Tuesday afternoon; and the target says what counts as passing.

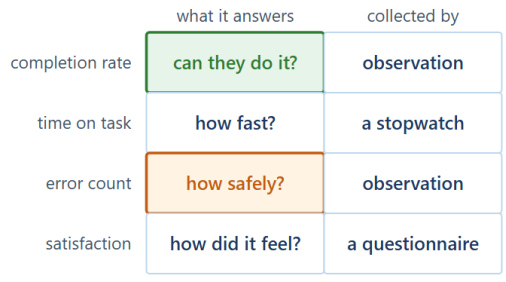
And note the third one. "Helpful" felt like the least measurable of the three and turned out to be the most precisely testable, because the *purpose* of an error message is recovery, and recovery is observable. Whenever a quality seems unmeasurable, ask what it is *for*.

Common Mistake

Do not set targets you have not baselined. "Complete in 90 seconds" sounds rigorous and may be impossible, or trivial — you do not know until you have measured the current system or a competitor.

Measure first, then set the target as an improvement on the measurement. A target invented in a meeting is a guess wearing the costume of a requirement.

4.5 The four metrics



Completion rate first: an interface nobody can finish has no meaningful time or satisfaction score.

What each metric answers, and how it is collected.

Metric	Answers	How
Completion rate	can they do it at all?	observation: finished unaided,

Metric	Answers	How
		yes or no
Time on task	how long?	stopwatch, from a defined start to a defined end
Error count	how safely?	observation, against a pre-agreed definition of "error"
Satisfaction	how did it feel?	a short standard questionnaire after each task

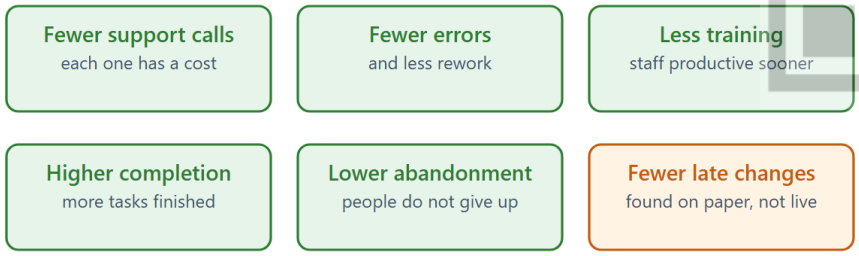
> **Key Idea**
>
> **Completion rate is the first metric, always.** An interface that people cannot finish has no meaningful time-on-task — the fast times will be the people who gave up early, and the average will look excellent.
>
> Report completion first, then time *among those who completed*, and say how many did not.
>
> **Note**
>
> **Two practical rules for collecting these.**
>
> *Define "error" before you start.* Otherwise you will classify inconsistently as the session proceeds, and worse, more leniently for designs you like. Write the list: wrong field, wrong button, needed a hint, used the back button to recover.
>
> *Define the start and end of the timer before you start.* "From login to the confirmation screen" is reproducible. "How long the booking took" is not.

Satisfaction is usually measured with a short standard questionnaire rather than a home-made one, because a standard instrument has known properties and lets you compare against other systems. The System Usability Scale (SUS) — ten statements, five-point agreement, producing a score out of 100 — is the usual choice, and it takes about a minute to complete.

> **Common Mistake**
>
> **Satisfaction and performance frequently disagree**, and the disagreement is informative rather than a problem with the data.
>
> Users often rate an attractive interface highly while performing worse on it — the aesthetic-usability effect of Week 5. If your satisfaction scores are good and your completion rate is poor, **believe the completion rate.** People are reporting how they felt, and they are not well placed to know how they performed.

رومانی صبحی رزق جرجس مطر

4.6 Arguing for usability



Usability is argued to management in these terms, not in terms of taste. Every row is a number someone tracks.

The argument as an organisation hears it.

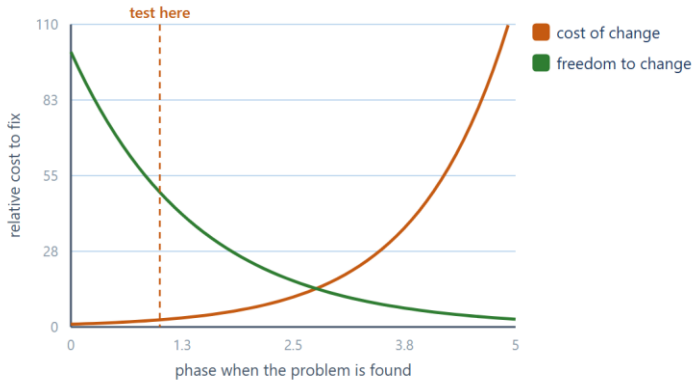
Usability work competes for budget with features, and “it would be nicer” loses that argument every time. The case has to be made in quantities the organisation already tracks.

Argument	The quantity
Fewer support calls	cost per call × call volume
Fewer user errors	rework, corrections, refunds
Less training required	staff hours before productivity
Higher task completion	conversions, applications completed
Lower abandonment	users who start and do not finish
Fewer late changes	the cost curve of §4.7

> **Tip**
>
> **The most persuasive single move is to show the recording.** Five minutes of video of a real customer failing to complete a purchase changes more minds in a meeting than any table of numbers.
>
> Combine the two: the recording to establish that the problem is real, the numbers to establish that it is expensive. Neither works as well alone.

2025/2026 2025/2026 2025/2026

4.7 Why early



Early on, change is cheap and freedom is high. Both reverse as the project proceeds — which is why testing is early.

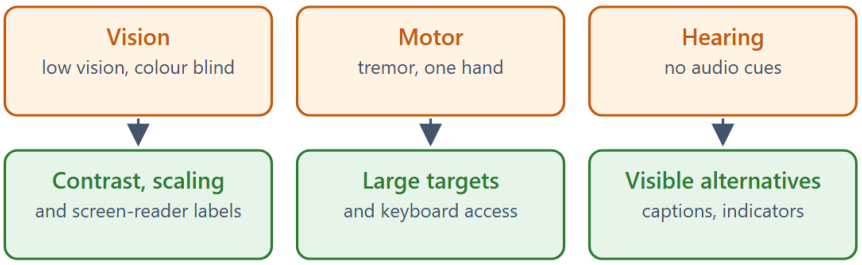
Cost rises as freedom falls.

The cost of changing a design rises steeply through a project, while the freedom to change it falls. A problem found on a paper sketch costs minutes; the same problem found after release costs a rewrite, a support burden, and the users who left without complaining.

This is the entire argument for the methods in Week 11 — prototyping, testing with five users, iterating — and it is why they appear at the sketch stage rather than at the end.

2025/2026 2025/2026 2025/2026

4.8 Accessibility is part of usability



Designing for impairment improves the product for everyone — bright sunlight is temporary low vision.

Designing for impairment improves the product generally.

> **Definition**
>
> **Accessibility** is the extent to which a system can be used by people with disabilities — of vision, hearing, motor control or cognition.

It is not a separate concern bolted on at the end. It is usability, for a user population that was always present and was simply not specified.

Impairment	Design response	Who else benefits
Low vision	contrast ≥ 4.5:1, scalable text	anyone outdoors, or over forty
Colour blindness	redundant coding	greyscale printing, bad

Impairment	Design response	Who else benefits
		projectors
Motor difficulty	large targets, keyboard access, no timing requirement	phone users on a moving bus
Hearing	captions, visible alternatives to sound	noisy or silent environments
Cognitive	plain language, simple structure	everyone under time pressure

Key Idea

Every row's third column is the point. **A design that works for a user with a tremor also works for a commuter on a moving bus**; a design that works at 4.5:1 contrast also works in sunlight.

Accessibility work is not charity and it is not compliance. It is the discipline of not making unstated assumptions about your user — which is the same discipline as the rest of this course.

Lab Activity

Lab 4 (2 hours)

1. **Rewrite aspirations.** Take five vague statements ("intuitive", "user-friendly", "clean", "modern", "simple") and rewrite each as a usability requirement with all four parts.

2. **Baseline.** Choose a real task on an existing system. Measure completion rate, time on task and error count with five people. Report the numbers, including how many failed.

3. Using your baseline, set a defensible target for a redesign, and justify the number.

4. **SUS.** Administer the System Usability Scale after the same task and compute the score. Compare it with your completion rate and comment on any disagreement.

5. **The business case.** For one problem you observed, estimate its annual cost using stated assumptions (call volume, staff cost, abandonment rate). State the assumptions explicitly — the estimate is only as good as they are.

6. **Accessibility audit.** Attempt one task using the keyboard only, with no mouse or touch. Then attempt it with a screen reader enabled. Record what is impossible and what is merely slow.

7. **Project.** Write three usability goals for your chosen task, each with user, task, measure and target. These are what you will test against in Week 11.

Summary

• *"Easy to use"* is not a requirement because it **cannot fail**. A usability requirement can.

• **Usability** = a specified user achieving a specified goal in a specified context, with effectiveness, efficiency and satisfaction. It is a property of a **system, user, goal and context together**.

• **Five components:** learnability, efficiency, memorability, errors, satisfaction. They **conflict**, so a product must state which dominate.

• **Memorability** is the forgotten component, and it governs the long tail of rarely-used functions.

• A requirement needs four parts: **user, task, measure, target**. Missing any makes it untestable.

• **Baseline before setting a target**, or the number is a guess.

رومانی صبحی رزق جرجس مطر

- **Completion rate first.** Fast times from people who gave up will flatter a bad interface.
- **Believe performance over satisfaction** when they disagree.
- Argue for usability in **costs the organisation already tracks**, and show a recording alongside the numbers.
- **Accessibility is usability** for users who were always present; every accommodation benefits a wider population.

Self-check

1. Why is “the system shall be easy to use” not a requirement?
2. Name the five components of usability, and explain which the tax filing site of §4.3 must favour.
3. State the four parts of a usability requirement and write one for a login screen.
4. Why must completion rate be reported before time on task?
5. Why should a target be set after a baseline measurement rather than before?
6. Your satisfaction score is 82/100 and your completion rate is 55%. Which do you believe, and why?
7. Give three arguments for usability that a finance director would accept.
8. Give an accessibility accommodation and name a non-disabled group that benefits from it.

(Full solutions are in the Week 4 question bank.)

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Week 5 — Principles of Interaction and Interface Design Issues

Learning Outcomes

By the end of this week you should be able to:

1. State and apply the six core design principles: visibility, feedback, affordance, mapping, constraints and consistency.
2. Distinguish an affordance from a signifier, and explain why the distinction matters in software.
3. Distinguish the three kinds of consistency and say which to favour when they conflict.
4. Use constraints to prevent errors rather than reporting them.
5. State Shneiderman's eight golden rules and relate them to Norman's principles.
6. Explain the aesthetic-usability effect and the risk it creates in testing.
7. Critique an interface by naming a principle and a specific change.

5.1 Why principles, and what they are for

Weeks 1–3 supplied the findings; Week 4 supplied the measurements. This week supplies the working vocabulary between them.

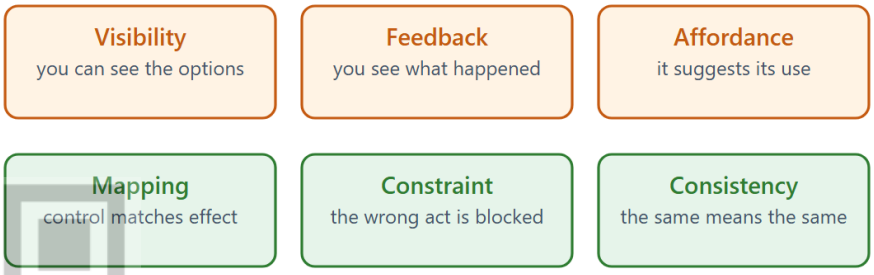
2025/2026 2025/2026 2025/2026

Key Idea

A principle is a compressed finding. Its value is that it lets you **diagnose before you test** and **argue without appealing to taste**.

"I don't like the layout" cannot be evaluated by anyone else. "The primary action is not visible on the first screen, so users cannot form an intention" names a specific defect, predicts a specific failure, and can be checked.

Principles do not replace testing. They generate hypotheses cheaply, and testing decides them.



Six principles that between them account for most everyday interface failures — and most of the fixes.

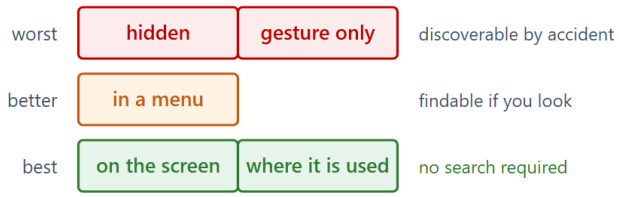
Six principles that account for most everyday interface failure.

5.2 Visibility

Definition

رومانى صبحى رزق جرجس مطر

> **Visibility**: the actions that are possible, and the current state of the system, should be apparent without the user having to search, remember or experiment.



Every hidden feature is a feature most users do not have. Visibility is the cheapest form of documentation.

Every hidden feature is a feature most users do not have.

There is a hierarchy. In descending order of reliability:

1. **On screen, at the point of use** — no search required.
2. **In a labelled menu** — findable if the user thinks to look.
3. **Behind a gesture or a long-press** — discoverable essentially by accident.

> **Common Mistake**
>
> Hidden gestures are the most over-used pattern in modern mobile design. A swipe that reveals Delete is invisible: nothing on the screen indicates it exists, and a user who does not already know is not going to guess.
>
> The defence offered is "it keeps the interface clean". The cost is that the feature effectively does not exist for most users — and the ones who discover it do so by accident, which means they also trigger it by accident.
>
> A gesture is acceptable as an **accelerator** for an action that is also visible elsewhere. It is not acceptable as the only route.

5.3 Feedback

> **Definition**
>
> **Feedback**: every action should produce a perceptible response indicating what happened, within a time appropriate to the action.

This is the bridge across the evaluation gulf of Week 1, and its absence produces the quiet failures — duplicate submissions, repeated clicks, users who cannot tell whether they have finished.

Good feedback has four properties:

Property	Meaning
Immediate	within ~100 ms for direct manipulation (Week 8)
Located	where the user is looking, not elsewhere on the page
Specific	"Saved to your account at 14:32", not "Done"
Proportionate	a small action gets a small signal; a destructive

Property	Meaning
	one gets a clear one

> **Tip**
>
> Feedback must also cover the **in-between** states. An action that takes three seconds needs a signal that it has started, not only one that it has finished — otherwise the user spends those three seconds believing nothing happened, and clicks again.

5.4 Affordance and signifiers



The distinction, and where doors go wrong.

> **Definition**
>
> An **affordance** is a relationship between an object and a user that determines what actions are possible. A door affords pulling whether or not anyone realises it.
>
> A **signifier** is the perceptible indication of that possibility: a handle, a flat plate, a label saying PUSH.

The distinction matters because **designers control the signifier**, not the affordance. The classic failing door — a vertical handle on a door that opens away from you — has the affordance of pushing and the signifier of pulling. The signifier wins, and people pull.

In software the affordances are almost entirely conventional: a rectangle is not naturally clickable. Everything therefore rests on signifiers, and the modern tendency towards flat, borderless design has weakened them:

- borderless buttons that look like text;
- text that is coloured but not underlined, and may or may not be a link;
- controls that appear only on hover — invisible on a touch screen, which has no hover.

> **Key Idea**
>
> **If a user must experiment to discover whether something is clickable, the signifier has failed.** The test is direct: show a screenshot to someone for three seconds and ask what they can click. Do it before the next design review — the results are usually sobering.

5.5 Mapping

> **Definition**

Mapping is the relationship between a control and its effect. A **natural mapping** exploits spatial or cultural analogies so that the relationship needs no learning.

which knob controls which burner?

	back L	back R	front L	front R
the four burners				
knobs in a row — bad	1	2	3	4
knobs in a square — good	back L	back R	front L	front R

Numbered knobs in a line must be memorised. Knobs laid out as the burners are need no learning at all.

Four burners, four knobs.

The cooker is the standard example because the contrast is so stark. Four burners in a square with four knobs in a row requires a label and, forever after, a moment's thought. Four knobs arranged in a square matching the burners requires nothing: the mapping is the layout.

Software examples of the same idea:

- a scrollbar whose thumb position maps to position in the document;
- a volume slider where up is louder;
- form fields in the order the information appears on the paper document being copied;
- a seat-selection map shaped like the actual aircraft or theatre.

2025/2026 2025/2026 2025/2026

Note

Mapping failures survive for years because each individual encounter costs only a second of thought, so nobody reports them. They accumulate as a general sense that a system is tiring to use, with no specific complaint attached.

5.6 Constraints

	example	in software
physical	plug only fits one way	disabled button
logical	one piece left, one hole	greyed-out date
cultural	red means stop	red delete button
semantic	helmet faces forward	end after start

A constraint is better than a warning: it removes the error rather than reporting it afterwards.

A constraint removes the error; a warning reports it.

Definition

رومانى صبحى رزق جرجس مطر

A **constraint** limits the actions that are possible, so that an incorrect action cannot be performed at all.

Kind	Physical world	Software
Physical	a plug that fits one way only	a disabled button
Logical	one piece left, one hole left	an option greyed out because it does not apply
Cultural	red means stop	destructive actions in red
Semantic	a helmet's visor must face forward	an end date that cannot precede a start date

Key Idea

A constraint is better than a warning, because a warning is a request for the user to be careful and a constraint removes the need.

Ranked by strength:

1. **Make the error impossible** — a date picker that cannot select a past date.
2. **Make it hard and reversible** — allow it, but offer undo.
3. **Confirm it** — a dialog, which relies on the user reading it, and they will not.
4. **Report it afterwards** — an error message, the weakest option.

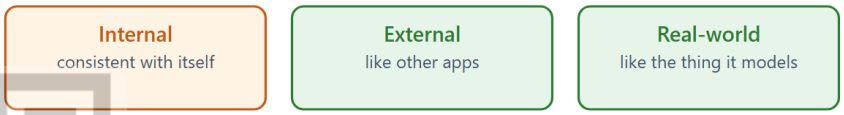
Most teams reach for 3. Prefer 1 wherever the rule is genuinely absolute.

Common Mistake

Do not over-constrain. A form that rejects a valid foreign telephone number, a name with an apostrophe, or a postal code in an unexpected format has used a constraint to enforce an assumption rather than a rule.

The test: is this restriction a **fact about the domain** (an end date cannot precede a start date) or a **convenience for the developer** (all phone numbers have eleven digits)? Only the first justifies a constraint.

5.7 Consistency



External consistency is why you should not invent a new gesture for saving. Habits transfer; cleverness does not.

Three kinds, and they can conflict.

Definition

Internal consistency — the system behaves the same way throughout itself. **External consistency** — it behaves like other systems the user knows. **Real-world consistency** — it matches the domain it models.

Consistency matters because it lets a user transfer knowledge rather than learning each screen separately. An inconsistent system must be learned as a set of special cases.

Key Idea

When internal and external consistency conflict, external usually wins, and students consistently guess the other way.

A user spends a few hours a month in your application and thousands of hours in everything else. Their habits are formed elsewhere. Inventing a new gesture for "save", a new position for "close", or a new icon for "print" means fighting those habits every time, for a benefit that is almost always aesthetic.

Standards exist to be used. Deviating from one requires a reason you could defend to a user, not a preference.

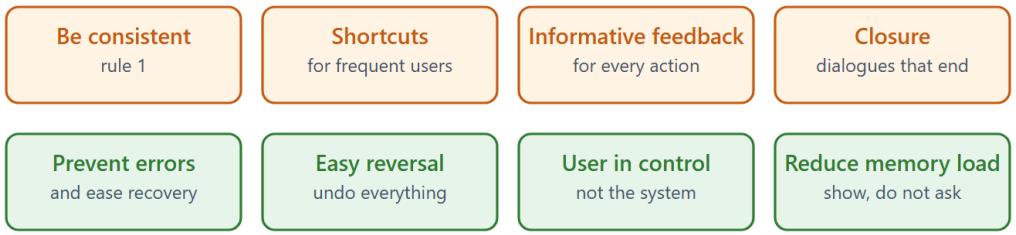
Note

The legitimate exception is when consistency would preserve a genuine defect — when the convention is itself bad, or when the situation is genuinely different and treating it the same would mislead. Deliberate inconsistency is sometimes correct: making a destructive action look *different* from the routine actions beside it is an inconsistency that prevents errors.

The rule is therefore: be consistent by default, and be able to state the reason whenever you are not.

5.8 Shneiderman's eight golden rules

2025/2026 2025/2026 2025/2026



Shneiderman's eight rules overlap Norman's six. Where two independent lists agree, the advice is probably sound.

Two independent lists that largely agree.

Rule	Relates to
1. Strive for consistency	consistency
2. Offer shortcuts for frequent users	learnability/efficiency (Week 3)
3. Offer informative feedback	feedback
4. Design dialogues to yield closure	feedback — an action must have an end
5. Prevent errors and offer simple recovery	constraints
6. Permit easy reversal of actions	*(new — see below)*
7. Keep the user in control	*(new — see below)*

Rule	Relates to
8. Reduce short-term memory load	memory (Week 3)

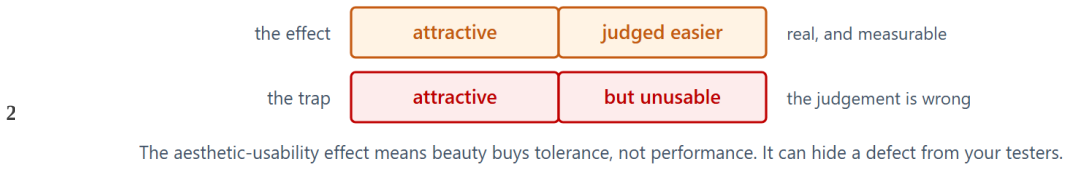
Two rules add something Norman's list does not state directly:

Reversibility (rule 6). If every action can be undone, exploration becomes safe, which is how people learn an interface. An irreversible action must be confirmed; a reversible one should not be, because unnecessary confirmations train users to dismiss all of them.

User control (rule 7). The user should initiate actions, not merely respond to the system. Interfaces that seize focus, auto-play, auto-advance a carousel or move a control as the user reaches for it violate this — and the last of those converts a user's correct action into an error.

> **Tip**
>
> That two independently derived lists overlap so heavily is itself evidence. Where Norman and Shneiderman agree, the advice is well supported. Where a guideline appears on neither list and comes from a single blog post, treat it as a hypothesis.

5.9 The aesthetic-usability effect



Beauty buys tolerance, not performance.

> **Definition**
>
> The **aesthetic-usability effect**: people perceive attractive interfaces as easier to use, and report higher satisfaction, independently of their measured performance.

The effect is real and has a legitimate use: an attractive interface earns patience, and patient users persist through difficulties rather than abandoning.

It also creates a specific risk in evaluation. **A polished prototype will be rated well even when participants are failing on it.** If you rely on satisfaction scores or on what participants say, a beautiful bad design will pass.

> **Key Idea**
>
> This is the strongest argument for measuring **completion rate and errors** (Week 4) rather than asking people how it felt — and one more reason to test **rough sketches**, where the effect is weakest and the criticism is frankest.

5.10 How to critique

	violates	fix
no sign the file saved	feedback	show a confirmation
delete next to save	constraint	separate and confirm
a new icon for print	consistency	use the standard one
settings behind a swipe	visibility	add a visible control

This is what a critique looks like in this course: name the principle, then name the change. Never "I do not like it".

What a critique looks like in this course.

Every critique you produce from now on has three parts:

1. **Observation** — what specifically happens, stated neutrally.
2. **Principle** — which principle it violates, named.
3. **Change** — a specific alteration, not a direction.

> *Observation:* after pressing Save, nothing on the screen changes. *Principle:* feedback — the evaluation gulf is unbridged. *Change:* show "Saved 14:32" beside the button, persisting until the next edit.

Compare: *"the save experience feels unpolished"*, which names nothing, predicts nothing and cannot be acted on.

2025/2026 2025/2026 2025/2026

> **Lab Activity**
>
> **Lab 5 (2 hours)**
>
> 1. **Door audit.** Return to the two doors you photographed in Week 1. Explain each in terms of affordance and signifier, and design a signifier that would fix the failing one.
>
> 2. **Clickability test.** Show three people a screenshot of a modern website for three seconds each and ask them to mark everything they believe is clickable. Compare with reality and report the errors in both directions.
>
> 3. **Constraints.** Take a form with a date range, a quantity and a telephone number. For each field, list the constraint you would apply and classify it as a genuine domain rule or a developer convenience.
>
> 4. **Consistency.** Find one place where an application you use deviates from the platform convention. Judge whether the deviation is defensible, and say why.
>
> 5. **Principle critique.** Critique one screen of a real system using the three-part format. Produce five findings, each naming a principle and a specific change.
>
> 6. **Reversibility.** Find an action in a real system that is irreversible and should not be, and one that is confirmed and should not be. Explain both.
>
> 7. **Project.** Apply the three-part critique to your own paper prototype, done by another group. Exchange findings.
>
> **Summary**

رومانی صبحی رزق جرجس مطر



• A **principle is a compressed finding**. It lets you diagnose cheaply and argue without appealing to taste — but testing decides.

• **Visibility**: on screen at the point of use > in a menu > behind a gesture. A hidden gesture is a feature most users do not have.

• **Feedback** must be immediate, located where the user is looking, specific, and proportionate — including for in-between states.

• An **affordance** is what is possible; a **signifier** is what shows it. Designers control the signifier, and flat design has weakened them.

• **Natural mapping** removes learning: knobs laid out like the burners. Mapping failures go unreported and accumulate as fatigue.

• **Constraints** beat warnings. Ranked: make it impossible > allow with undo > confirm > report afterwards. Do not constrain a developer convenience.

• Three kinds of **consistency**; when internal and external conflict, **external usually wins** — habits are formed elsewhere.

• **Shneiderman's eight rules** overlap Norman's and add **reversibility** and **user control**.

• The **aesthetic-usability effect** means a beautiful bad design passes a satisfaction-based test. Measure completion and errors.

• A critique is **observation + principle + change**.

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

Self-check

1. State the six principles and give one interface failure for each.
2. Distinguish affordance from signifier. Which does a designer control?
3. Why has flat design made signifiers harder, and what is the three-second test?
4. What is a natural mapping? Why do mapping failures rarely get reported?
5. Rank the four ways of handling an invalid action, strongest first.
6. When is it right to break external consistency?
7. What do Shneiderman's rules add that Norman's six do not state?
8. Why does the aesthetic-usability effect make satisfaction scores unreliable?

(Full solutions are in the Week 5 question bank.)

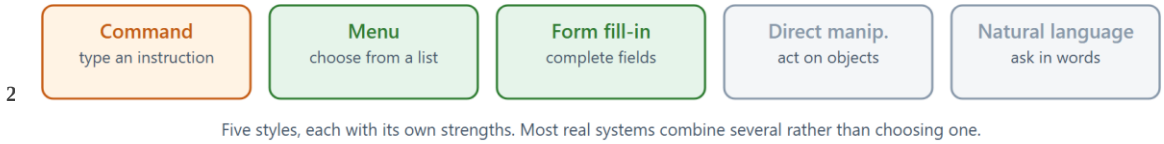
Week 6 — Command Languages, Menus and Form Fill-in

Learning Outcomes

By the end of this week you should be able to:

1. Name the five interaction styles and state where each is appropriate.
2. Explain why command languages persist despite their poor learnability.
3. Apply the breadth-versus-depth result to a menu structure.
4. Order and group menu items defensibly, including the placement of destructive items.
5. Design a form using one-column layout, grouping and forgiving input.
6. Choose the right moment to validate, and write a message that enables recovery.
7. Justify every field in a form, or remove it.

6.1 The five interaction styles



Five styles. Most real systems combine several.

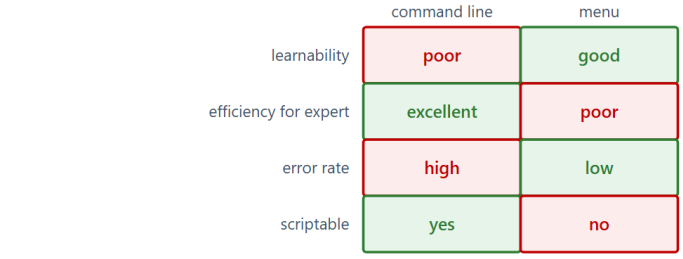
Style	The user	Strength	Weakness
Command language	types an instruction	fast, precise, scriptable	must be recalled; high error rate
Menu selection	chooses from a list	recognition; discoverable	slow; scales badly
Form fill-in	completes fields	good for structured data entry	tedious; rigid
Direct manipulation	acts on visible objects	immediate, learnable *(Week 7)*	hard for abstract or bulk operations
Natural language	asks in words	no syntax to learn *(Week 12)*	ambiguous; capability is invisible

Key Idea

The question is never "which style is best". It is **which style suits this task, for this user, at this frequency** — and most systems need more than one.

> A file manager offers direct manipulation for the common case *and* a command line for the bulk case, because dragging 4,000 files into folders by hand is absurd and so is typing a command to open one document.

6.2 Command languages



The command line did not die because it wins the bottom two rows — and those matter to professionals.

Why the command line did not die.

A command language requires the user to **recall** a name, its options and its syntax. By the recognition/recall asymmetry of Week 3, this is the hardest demand any interface can make, and command languages are correspondingly difficult to learn and error-prone.

They survive, and in professional settings they dominate, for four reasons:

1. **Speed for an expert.** A short command beats any amount of navigation.
2. **Precision.** `rm *.tmp` expresses exactly what is meant, with no ambiguity about which items are selected.
3. **Composition.** Commands combine — the output of one becomes the input of another. No menu system offers an equivalent.
4. **Automation.** A command can be recorded, scripted, scheduled and repeated. A sequence of clicks cannot.

> **Note**
>
> Points 3 and 4 are the decisive ones, and they are frequently omitted when this comparison is taught. They are not about speed at all: they are about the interface being **programmable**, which converts a repeated manual task into a task the machine performs — the task-allocation principle of Week 1.

Design rules for a command language, where you must build one:

- **Consistent, predictable naming.** A verb-object pattern throughout, not `delete file` beside `mv source dest`.
- **Consistent option syntax**, and long forms as well as short (`--recursive` as well as `-r`) so scripts are readable.
- **Abbreviation with feedback** — accept a prefix, and echo the full command.
- **Discoverability**, since recall is the weakness: `help`, completion on Tab, and a listing of available commands.
- **Confirmation for the irreversible**, and a dry-run option for anything destructive and bulk.

6.3 Menus

Menus replaced command languages for general use because they convert recall into recognition. The design questions that remain are about **structure**.

Breadth versus depth

three ways to organise the same 64 items

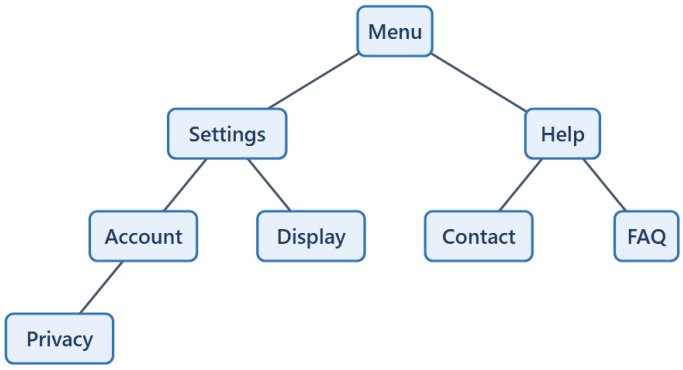
	levels	per level	decisions	reachable
broad and shallow 8 x 8	2	8	2	64
middle 4 x 4 x 4	3	4	3	64
narrow and deep 2^6	6	2	6	64

Every level is a decision, and every decision is a chance to take a wrong turn. Prefer breadth to depth.

Three ways to organise the same 64 items.

Every level of a menu is a decision, and every decision is an opportunity to choose wrongly. A wrong choice is expensive: the user must recognise the error, back out, and try another branch — and often they do not recognise it, and conclude the function does not exist.

2025/20

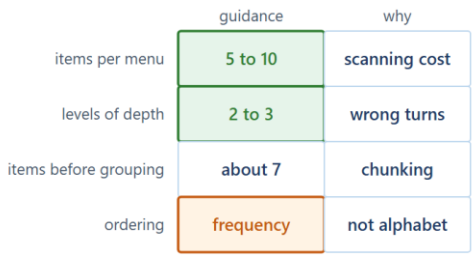


Four levels to reach Privacy. A user who guesses wrong at level 2 must back out and start again.

Four levels to reach one item.

> **Key Idea**
>
> **Breadth generally beats depth.** The research consistently favours broader, shallower structures over narrow, deep ones for the same content.
>
> The reason is that scanning a list of eight items is fast and reliable — recognition — while deciding which of two abstract categories contains your target is slow and often wrong, and the cost of being wrong is a whole branch of wasted navigation.

رومانی صبحی رزق جرجس مطر

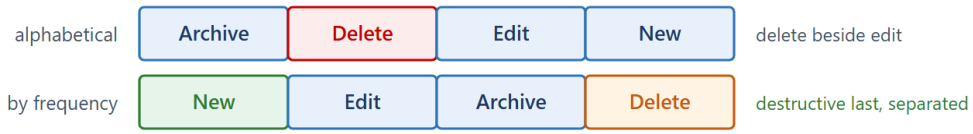


Guidance, not law. Broad and shallow generally beats narrow and deep — but measure it on your own content.

Guidance, not law.

Question	Working guidance
Items per menu	5–10 before grouping becomes necessary
Depth	2–3 levels for most systems
Grouping	group at about 7 items, with separators or headings
Ordering	by frequency or by task sequence, not alphabetically

> **Common Mistake**
>
> **Alphabetical ordering is the default choice and usually the wrong one.** It is appropriate only when the user already knows the exact word they are looking for — a country list, a contact list, a list of course codes.
>
> For a menu of *actions*, the user knows what they want to *do*, not what it is called. Alphabetical order scatters related actions and, worse, places items adjacent by accident: `Delete` next to `Edit` is an alphabetical coincidence and a capture-error hazard (Week 3).



Alphabetical ordering is defensible only when the user already knows the exact word they are looking for.

Ordering changes which errors are possible.

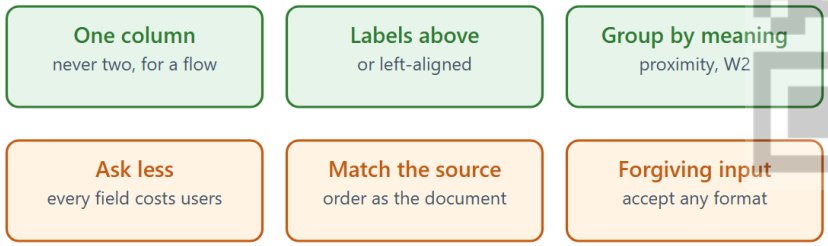
Where destructive items go

Three rules, each of which prevents a specific error:

- **Never adjacent to a frequent action.** `Delete` beside `Edit` invites a slip.
- **Separated visually**, at the end of the menu, below a separator.
- **Styled differently** — this is the legitimate inconsistency of Week 5.

6.4 Forms

Form fill-in is the dominant style for structured data entry, and it is where most users of most systems spend most of their time. It is also where the most easily avoided losses occur.



The strongest improvement to most forms is deleting fields. The second strongest is accepting what people type.

Six rules for a form.

Layout

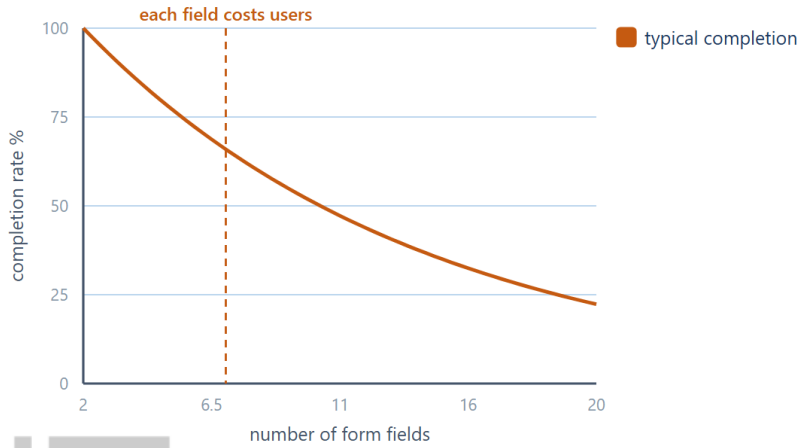
One column. A multi-column form has no unambiguous reading order: users skip fields, or complete them in an order the validation did not anticipate. The only common exception is a pair of genuinely parallel short fields (city and postal code) placed on one line.

Labels above or left-aligned, never inside the field. Placeholder-as-label disappears on typing (Week 3), fails contrast (Week 2) and fails screen readers.

Group by meaning, using spacing (Week 2, proximity), not by adding boxes.

Match the order of the source document. If users are copying from a paper form, follow the paper's order — even if a different order seems more logical. They are transcribing, and every reordering is a place to lose their position. 2025/2026 2025/2026 2025/2026

Ask for less



Every additional field loses some proportion of users. Ask only for what you will actually use.

Every field costs some proportion of users.

> **Key Idea**
>
> **The single strongest improvement to most forms is deleting fields.**
>
> For each field, ask: what will we do with this? If the answer is "it might be useful later", delete it. A field that is never used has a cost paid by every user and a benefit paid to nobody.

> The second strongest improvement is accepting whatever format people type.

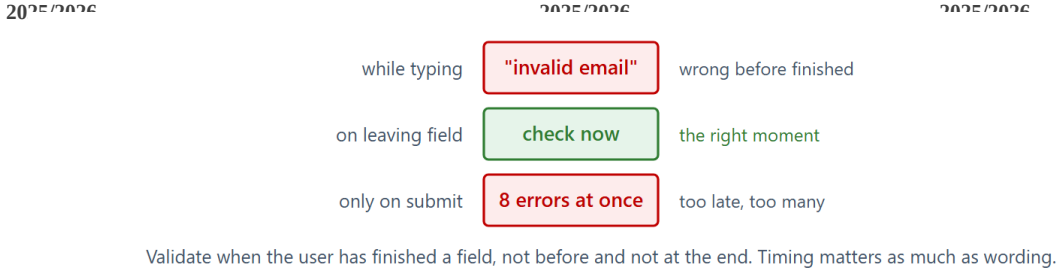
Forgiving input

A form should accept what a reasonable person would write and normalise it silently:

The user types	Accept and normalise
`0100 123 4567`, `01001234567`, `+20 100 123 4567`	one stored format
`AHMED`, `ahmed`, `Ahmed`	trim and case-normalise
`01/05/2026`, `1 May 2026`	one date, with the interpretation shown back
a card number with spaces	strip them

> **Common Mistake**
>
> Rejecting input the system could have interpreted is the most common self-inflicted error in form design. It is particularly bad when the interface itself displayed the value in the format it then rejects.
>
> Where ambiguity is real — `01/05/2026` is 1 May or 5 January depending on the reader — do not guess silently. Interpret, then **show the interpretation back** ("1 May 2026") so the user can correct it.

Validation timing

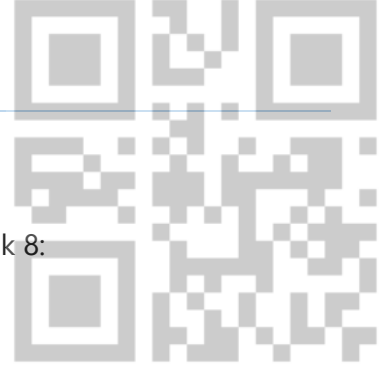


Validate when the user has finished a field, not before and not at the end. Timing matters as much as wording.

The right moment is when the user has finished the field.

When	Effect
While typing	tells the user they are wrong before they have finished — irritating and untrue
On leaving the field	**correct** — they have finished, and the context is fresh
Only on submit	eight errors at once, far from the fields, after the effort is spent

There is one exception in each direction: *while typing* is right for a live strength or availability check that is genuinely useful in progress (password strength, username availability), and *on submit* is unavoidable for checks that require the whole form (dates that must be consistent with each other).



Error messages

A validation message must do three things — this is developed fully in Week 8:

1. **Say what is wrong**, specifically.
2. **Say how to fix it.**
3. **Be located at the field concerned**, not at the top of the page.

Not: *"Invalid input."* But: *"Enter the date as 05/2026 — we found 052026."*

Lab Activity

Lab 6 (2 hours)

1. **Menu restructure.** Take a real application menu with more than 30 items. Record its current structure as a tree, count the levels, and restructure it to be broader and shallower. Justify each grouping.

2. **Card sort.** Write the 30 items on cards and ask five people to group them and name the groups. Compare their groupings with yours and with the original. *(This is a real technique — you will use it again in Week 10.)*

3. **Ordering.** Find a menu ordered alphabetically. Reorder by frequency or task sequence and identify any destructive item that was previously adjacent to a frequent one.

4. **Form redesign.** Take a real registration form. For each field, state what the organisation does with the data. Delete every field you cannot justify, and report how many remain.

5. **Forgiving input.** Implement a phone and date field that accept at least four formats each and show the normalised interpretation back to the user.

6. **Validation timing.** Build one form validating on each of the three timings. Test with five people and record irritation and error-recovery time.

7. **Project.** Produce the paper prototype of your main screens. Bring it next week for another group to critique.

Summary

• **Five styles**: command, menu, form fill-in, direct manipulation, natural language. The question is which suits this task and user — most systems need several.

• **Command languages** demand recall and are error-prone, but win on speed, precision, **composition** and **automation**. The last two make the interface programmable.

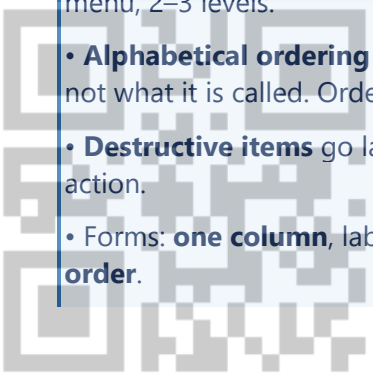
• **Breadth beats depth**: every level is a decision and every decision can be wrong. 5–10 items per menu, 2–3 levels.

• **Alphabetical ordering is usually wrong** for actions — the user knows what they want to do, not what it is called. Order by frequency or task sequence.

• **Destructive items** go last, separated and styled differently — never adjacent to a frequent action.

• Forms: **one column**, labels above or left, group by spacing, **match the source document's order**.

رومانى صبحى رزق جرجس مطر



• **Deleting fields is the strongest improvement** to most forms; accepting any reasonable format is the second.

• **Validate on leaving the field** — not while typing, not only on submit. Show the interpretation back when input is ambiguous.

Self-check

1. Name the five interaction styles with one appropriate use of each.
2. Give two advantages of a command language that have nothing to do with speed.
3. State the breadth-versus-depth result and explain why a wrong turn is expensive.
4. When is alphabetical ordering appropriate?
5. Give three rules for the placement of a destructive menu item.
6. Why should a form match the order of the paper document it replaces?
7. When should a field be validated, and what are the two exceptions?
8. A form asks for a fax number “in case we need it”. What is your response?

(Full solutions are in the Week 6 question bank.)

رومانى صبحى رزق جرجس مطر

Week 7 — Windows, Icons and Direct Manipulation

Learning Outcomes

By the end of this week you should be able to:

1. Describe the WIMP interface and explain why it has survived so long.
2. State the criteria for direct manipulation and identify where it fails.
3. Choose between overlapping, tiled and full-screen window arrangements for a task.
4. Judge when an icon will be understood, and when it must be labelled.
5. Explain the object-action order and why it is preferred.
6. Identify the accessibility problems of drag and drop and provide alternatives.

Note

The midterm examination falls this week and covers Weeks 1–6. This week's material is examinable in the final, not the midterm.

7.1 WIMP



WIMP, unchanged in essentials since 1973. Its durability is evidence, not inertia.

Four elements, essentially unchanged since 1973.

Definition

WIMP — Windows, Icons, Menus, Pointer. The interface paradigm developed at Xerox PARC in the early 1970s, commercialised through the 1980s, and dominant ever since.

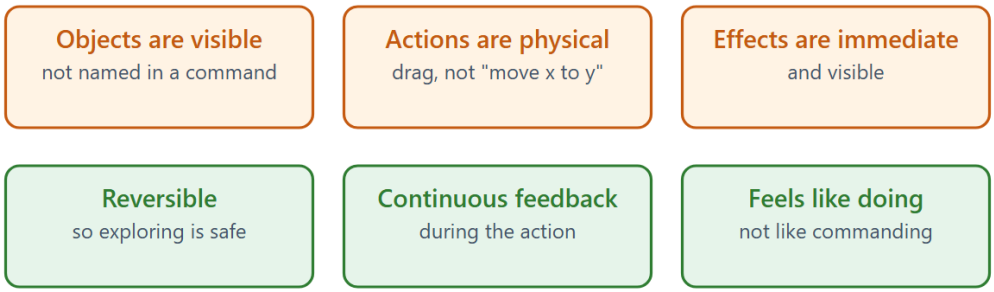
Its durability is worth taking seriously rather than treating as inertia. WIMP succeeded because each element solves a problem identified in Weeks 2–3:

Element	Solves
Windows	several tasks at once, without losing any of them
Icons	objects made visible, so they can be recognised rather than named
Menus	available commands displayed — recognition over recall
Pointer	select what you can see, removing the need to describe it

Key Idea

Every element converts a **memory task into a perception task** — the strategy of Week 3. That is why the paradigm has outlasted every attempt to replace it, and why a proposal to abandon part of it should be examined carefully.

7.2 Direct manipulation



Shneiderman's criteria. The last row is what people mean when they say an interface feels direct.

Shneiderman's criteria.

Definition

Direct manipulation is an interaction style in which the objects of interest are continuously visible, actions are physical and incremental rather than named, effects are immediately visible, and operations are reversible.

2025/2026 2025/2026 2025/2026

The term is used loosely to mean "clicking on things". Shneiderman's criteria are more specific, and the difference matters:

1. **Continuous representation** of the objects of interest.
2. **Physical actions** — dragging, pointing, resizing — rather than typed command names.
3. **Rapid, incremental, reversible** operations whose effect is **immediately visible**.
4. **Layered learning** — a novice can begin without understanding everything.

Point 3 is what distinguishes it. A drag that shows the object moving under the cursor is direct. A drag that shows nothing until the mouse is released is not — it has the gesture without the property that makes the gesture work.

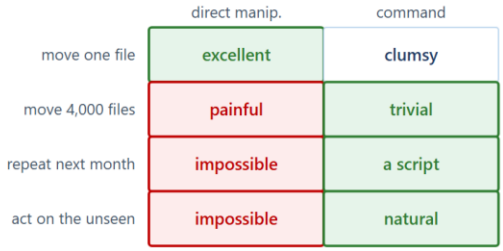
Note

Why it feels different. Direct manipulation collapses the interaction cycle of Week 1. There is no separate stage of specifying an action and then interpreting a result: the action *is* the result, and evaluation happens continuously during execution.

Both gulfs narrow at once, which is why these interfaces are learned so quickly and why people describe them as "feeling like doing" rather than commanding.

رومانی صبحی رزق جرجس مطر

Where it fails



Direct manipulation requires the object to be visible and the action to be performed once. Both fail at scale.

Visible, and once. Both conditions fail at scale.

Direct manipulation requires that the object be **visible** and that the action be performed **once**. Neither holds for:

- **Bulk operations.** Moving 4,000 files by dragging is absurd; `mv *.jpg photos/` is trivial.
- **Repetition.** A drag cannot be recorded, parameterised or scheduled.
- **Objects not on screen** — everything matching a criterion, everything modified since a date, everything not yet processed.
- **Abstract operations** — "apply this rule to future arrivals" has no object to point at.

> **Key Idea**
>
> This is the Week 6 argument restated: **direct manipulation and command languages are complementary, not competing.** A serious tool offers both, and users move between them according to the size and repetition of the task.
>
> A system offering only direct manipulation forces the human to do the machine's job whenever the task is large — which is the task-allocation defect of Week 1.

7.3 Windows



The arrangement should follow the task. Most systems impose one arrangement and let the user fight it.

The arrangement should follow the task.

Arrangement	Strength	Weakness	Suits
Overlapping	flexible; unlimited windows	windows get lost behind others	browsing many loosely related things

Arrangement	Strength	Weakness	Suits
Tiled	everything visible; no hunting	less space each; rigid	comparing two documents
Full screen	maximum space; fewest distractions	no comparison possible	one sustained task

> **Common Mistake**
>
> **Window management is work the system has delegated to the user.** Arranging, resizing, finding and switching windows is not part of anyone's task — it is overhead imposed by the interface.
>
> The measure of a good arrangement is how little of it the user must do. If your users spend time tidying windows before they can work, the default arrangement is wrong.

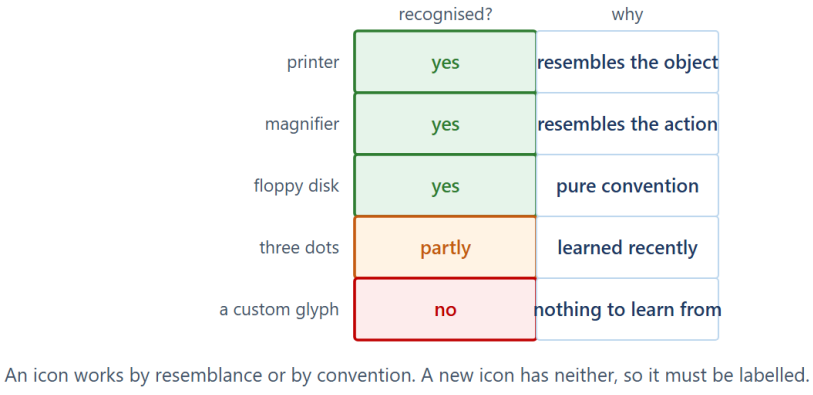
Two specific defects worth naming:

Lost windows. In an overlapping system a window can be entirely hidden, and the user has no indication it exists. This is a **visibility** failure (Week 5), and it is why taskbars, window lists and application switchers exist at all.

Dialogue boxes that steal focus. A modal dialog appearing while the user is typing captures keystrokes intended for something else, and the user may dismiss it without reading — a **mode error** (Week 3) with a violation of **user control** (Shneiderman 7).

2025/2026 2025/2026 2025/2026

7.4 Icons



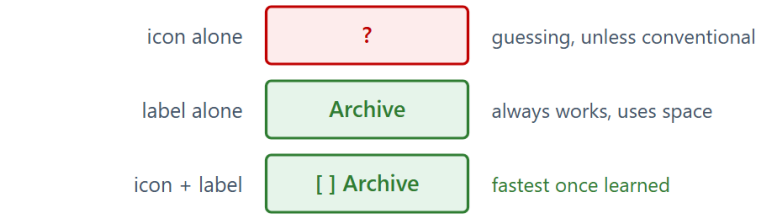
An icon works by resemblance or by convention.

An icon can be understood in only two ways: because it **resembles** the thing it represents, or because the user has **learned the convention**. There is no third route, and a new icon has neither.

Icon	Understood because
a printer	resemblance — it looks like a printer
a magnifying glass	resemblance to the *action* of examining

Icon	Understood because
a floppy disk for Save	**pure convention** — most users have never seen one
three dots for “more”	a recently learned convention, not yet universal
a novel custom glyph	neither — it must be labelled

> **Key Idea**
>
> **The floppy disk icon is the clearest evidence that convention beats resemblance.** It depicts an object most current users have never encountered, and it is understood perfectly — while a beautifully drawn novel icon for the same function would not be.
>
> The design conclusion follows directly: **use the conventional icon**, even if it is anachronistic. Replacing it costs recognition and buys nothing a user cares about.



2025/2026 2025/2026 2025/2026

Icon plus label beats either alone.

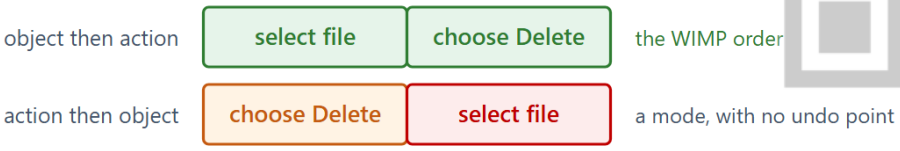
The rule for icons:

- **Conventional icon, universally known** (print, search, close, back) — the icon alone is acceptable.
- **Anything else** — icon **and** label. The label makes it usable on first encounter; the icon makes it faster to locate afterwards.
- **A toolbar of unlabelled custom icons** is a guessing game, and users will use three of the twelve.

> **Note**
>
> **Icons do not transfer across cultures as reliably as designers assume.** A mailbox, a shopping trolley, a thumbs-up gesture and a hand signal all carry different meanings in different places, and some are offensive in some of them. Where an icon must work internationally, test it, or label it.

رومانی صبحی رزق جرجس مطر

7.5 Object then action



Selecting first lets the user change their mind and lets the system offer only valid actions.

Selecting first is the WIMP order, and the safer one.

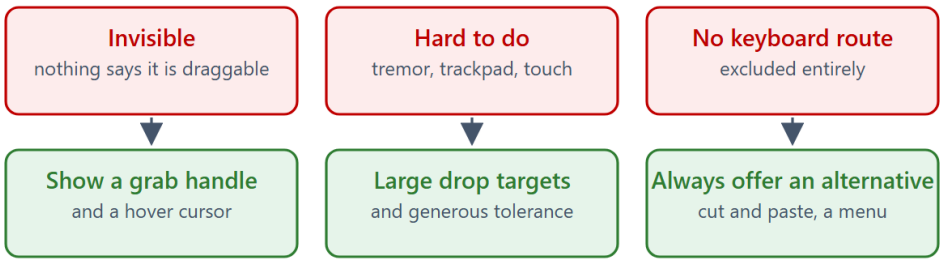
WIMP interfaces use **object-action** order: select the thing, then choose what to do to it. The alternative — choose the action, then apply it — is used by drawing tools ("select the eraser, then erase") and has two costs.

Object-action is preferred because:

1. **The system can offer only valid actions** for the selected object, which is a constraint (Week 5) rather than an error message.
2. **The user can change their mind** at no cost before committing.
3. **There is no mode.** Action-first creates a mode, with all the failures of Week 3 — and the eraser that is still selected ten minutes later is the canonical example.

> **Tip**
>
> Where action-first is genuinely appropriate — a drawing tool, a measuring tool — apply the mode remedies from Week 3: change the **cursor** so the mode is visible at the point of attention, and consider making it spring-loaded.

7.6 Drag and drop



Drag and drop is the least accessible common interaction. It must never be the only route to a function.

The least accessible common interaction.

Drag and drop is the most direct of manipulations and has three serious weaknesses:

1. **It is invisible.** Nothing about an item usually indicates that it can be dragged, or where it may be dropped. This is a signifier failure.

2. **It is physically demanding.** Press, hold, move accurately, release — difficult with a tremor, with a trackpad, with one hand, and on a small touch screen where the finger obscures the target.
3. **It has no keyboard equivalent by default**, so keyboard and screen-reader users are excluded entirely.

> **Key Idea**
>
> **Drag and drop must never be the only route to a function.**
>
> Provide it as an accelerator, and always offer an alternative: cut and paste, a "move to…" menu item, or arrow buttons for reordering. The alternative is also what makes the function reachable by keyboard.

Where it is used, mitigate:

- **Show a grab handle** and change the cursor on hover, so the affordance is signified.
- **Highlight valid drop targets** as soon as the drag begins — feedback during the action, not after it.
- **Make targets large and tolerant** of imprecision.
- **Animate the return** if a drop fails, so the user sees what happened rather than finding the item gone.

> **Lab Activity**
>
> **Lab 7 (2 hours)** — *shortened this week because of the midterm.*
>
> 1. **Icon test.** Collect twelve icons from real applications, without labels. Show them to five people and ask what each does. Report the recognition rate, and classify each icon as resemblance, convention or neither.
>
> 2. **Direct manipulation audit.** Find a task in a real system that requires repeating the same direct manipulation more than ten times. Describe the command-style alternative that should exist.
>
> 3. **Drag and drop.** Find an interface using drag and drop. Attempt the same task using only the keyboard. Report whether it is possible, and design the alternative route.
>
> 4. **Project.** Exchange paper prototypes with another group and produce a written critique using the Week 5 three-part format — at least eight findings, ranked by severity. This critique is assessed.
>
> **Summary**
>
> • **WIMP** — Windows, Icons, Menus, Pointer — has survived because each element converts a **memory task into a perception task**.
>
> • **Direct manipulation** requires continuous representation, physical actions, and **rapid, incremental, reversible** operations with immediately visible effects. It collapses both gulfs of Week 1.
>
> • It **fails for bulk, repetition, unseen objects and abstract operations** — so it complements command languages rather than replacing them.

رومانی صبحی رزق جرجس مطر

• **Window arrangement should follow the task**: overlapping for browsing, tiled for comparison, full screen for a single sustained task. Window management is overhead the system has delegated to the user.

• An icon is understood by **resemblance or by convention** — never by novelty. The floppy disk proves convention beats resemblance.

• Use **icon plus label** for anything not universally conventional.

• **Object-action order** avoids a mode, permits changing your mind, and lets the system offer only valid actions.

• **Drag and drop** is invisible, physically demanding and excludes keyboard users. Never make it the only route.

Self-check

1. What do the four letters of WIMP stand for, and what problem does each solve?
2. Give Shneiderman's criteria for direct manipulation. Which one distinguishes it from merely clicking?
3. Give three situations in which direct manipulation is the wrong choice.
4. When is a tiled arrangement better than overlapping windows?
5. Why is the floppy-disk Save icon still understood, and what does that prove?
6. State the rule for when an icon may appear without a label.
7. Give two reasons object-action order is preferred to action-object.
8. Why must drag and drop never be the only way to perform a function?

(Full solutions are in the Week 7 question bank.)

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Week 8 — Error Messages, Response Time and Feedback

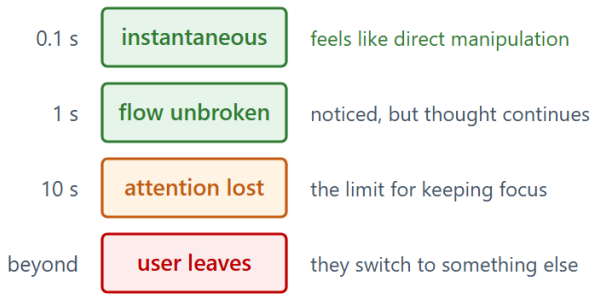
Learning Outcomes

By the end of this week you should be able to:

1. State the three response-time thresholds and what each governs.
2. Choose the correct waiting indicator for a given delay.
3. Distinguish perceived from actual duration and improve the former.
4. Write an error message containing all four required parts.
5. Match the severity of a response to the severity of the problem.
6. Argue that prevention and acceptance should be tried before any message is written.

8.1 Response time

The times at which a system feels fast, slow or broken are properties of human perception, not of hardware. They were established in the 1960s and have not changed since, because people have not changed.



Three thresholds that have held since the 1960s. They are properties of people, not of hardware.

Three thresholds.

Definition

0.1 second — the limit for feeling that the system is reacting *instantaneously*. Below this, the user perceives their own action as causing the result directly.

1 second — the limit for the user's *flow of thought* to remain unbroken. The delay is noticed, but no special feedback is needed beyond the eventual result.

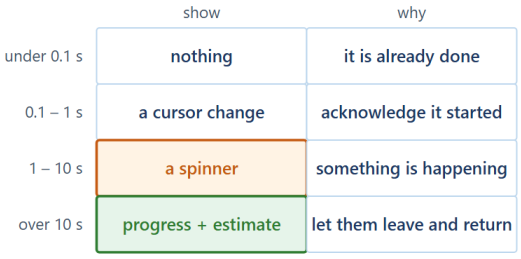
10 seconds — the limit for keeping *attention* on the task. Beyond this the user will begin doing something else, and must be given something that lets them leave and return.

Key Idea

The 0.1-second threshold is what makes direct manipulation (Week 7) work. If dragging an object showed it moving 300 ms later, the sense of manipulating the thing directly would disappear entirely — you would be issuing a command and watching a result.

This is why animation duration matters. A transition of 200–300 ms reads as responsive; 500 ms reads as sluggish, however elegant it looks to the designer who is watching it in isolation rather than performing it for the hundredth time.

What to show, and when



Match the indicator to the wait. A spinner for a two-second task; a real progress bar for a two-minute one.

Match the indicator to the wait.

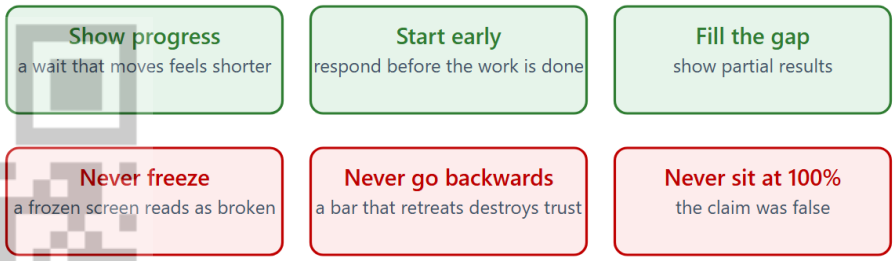
Delay	Show
under 0.1 s	nothing — it is already done
0.1–1 s	a cursor change, or a pressed state — acknowledge the action
1–10 s	a busy indicator — something is happening
over 10 s	a **progress bar with an estimate**, so the user can leave and return

Common Mistake

An indeterminate spinner is not adequate beyond about ten seconds. It says "something is happening" and nothing else: the user cannot judge whether to wait, so they wait anyway, then reload, then submit again.

If the operation has measurable phases, show them by name. "Uploading 3 of 7" tells the user far more than any animation, and it costs almost nothing.

8.2 Perceived duration



Perceived duration is not actual duration. Both are worth engineering, and the cheaper one is perception.

Perceived and actual duration are different quantities.

The user's experience of waiting depends on more than the elapsed time.

Things that make a wait feel shorter:

- **Visible progress.** A wait that is demonstrably advancing feels shorter than an identical wait that is not.
- **Early acknowledgement.** Responding to the *action* immediately, even before the work is complete, removes the anxious interval in which the user does not know whether anything happened.
- **Partial results.** Showing the first items while the rest load lets the user begin reading, which occupies the wait with the task rather than with waiting.
- **An explanation.** "Checking availability with 12 airlines" converts an unexplained delay into a justified one.

Things that make it feel longer, or break trust entirely:

- **A frozen interface.** An unresponsive screen reads as broken, and the user's next action is to force-quit or reload.
- **A progress bar that goes backwards**, or jumps, or stalls.
- **A bar that reaches 100% and stays there.** The claim was false, and the user now has no reason to believe the next one.

> **Key Idea**
>
> **Perceived duration is cheaper to improve than actual duration.** Making an operation genuinely twice as fast may be weeks of engineering; making it *feel* acceptable is often an afternoon.
>
> That is not an argument against optimisation. It is an argument for doing the cheap thing first, and for not shipping a fast operation with no feedback — which users experience as slower than a slow one with good feedback.

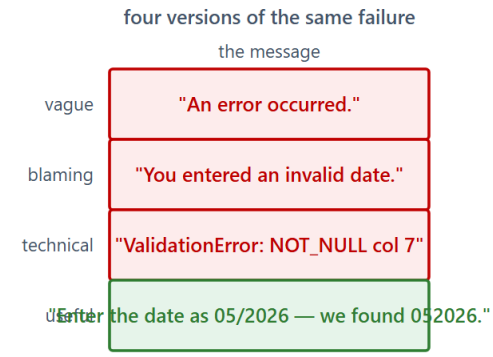
8.3 Error messages



Four parts. A message missing the third is a report; a message with all four is a recovery route.

Four parts.

> **Definition**
>
> A useful error message states **what happened**, **why** in terms the user understands, and **what to do next** — and it appears **where the problem is**.



Four versions of the same failure.

Version	Failure
"An error occurred."	says nothing at all
"You entered an invalid date."	blames the user; gives no format
"ValidationError: NOT_NULL col 7"	the implementation's view, not the user's
"Enter the date as 05/2026 — we found 052026."	**all four parts**

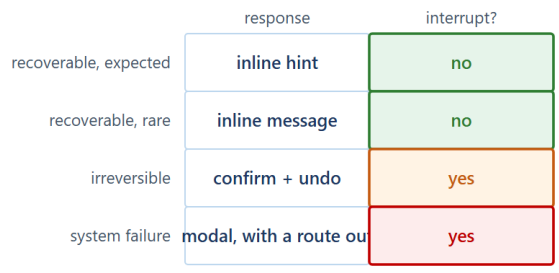
Note what the good message does: it states the expected format, quotes what was received, and sits beside the field. The user can act on it without thinking.

2025/2026 2025/2026 2025/2026

> **Tip**
>
> **Three rules for the wording.**
>
> *Never blame.* "You entered an invalid date" makes the user the subject of the failure. "Enter the date as MM/YYYY" is shorter and assigns nothing.
>
> *Never apologise at length.* "We are very sorry, an unexpected error has occurred" occupies the space where the instruction should be. Say what to do.
>
> *Never use the implementation's vocabulary.* Exception names, column numbers and status codes belong in the log. A reference code the user can quote to support is useful; the stack trace is not.

8.4 Severity



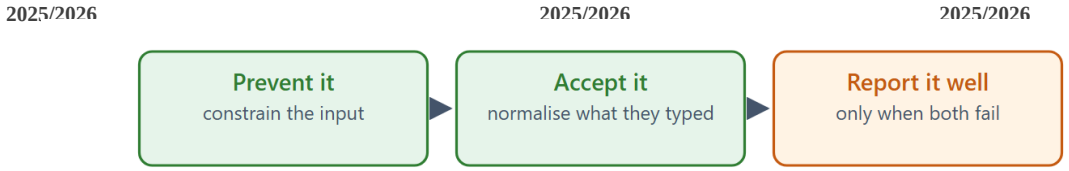


Interrupt in proportion to consequence.

Situation	Response	Interrupt?
Recoverable and expected (a mistyped field)	inline hint beside the field	**no**
Recoverable and rare (a failed upload)	inline message with a retry	**no**
Irreversible (deleting an account)	confirm, and offer undo afterwards	**yes**
System failure (the service is down)	a clear statement with a route out	**yes**

> **Common Mistake**
>
> **A modal dialog for a typo trains users to dismiss modal dialogs.**
>
> Every unnecessary interruption reduces the attention paid to the next one. A system that interrupts for trivia has disabled its own alerting channel by the time something serious happens — the same mechanism as alarm fatigue (Week 1) and over-notification (Week 2).
>
> The test is the one from Week 2: not "is this important to the system?" but **"is this worth interrupting a person mid-task for?"**

8.5 Prevent, accept, then report



Three stages, in order.

> **Key Idea**
>
> **Every error message you write is an opportunity you had to prevent the error or accept the input, and did not take.**
>
> 1. **Prevent it.** A date picker that cannot offer an invalid date produces no invalid dates. A disabled action cannot be taken. This is the constraint principle of Week 5.
>
> 2. **Accept it.** `052026`, `05 2026` and `5/2026` are all unambiguous. Normalise them and show the interpretation back (Week 6).
>
> 3. **Report it** — only when the input genuinely cannot be interpreted, and then with all four parts.

This ordering is the practical summary of much of the course. It is also a useful review question to ask of any interface: **count the error messages, and ask how many of them exist because prevention or acceptance was not attempted.**

8.6 Feedback beyond errors

Feedback is not only for failure. Three cases are routinely omitted:

Confirming success. An action that succeeds must say so, specifically and persistently. "Saved" that fades in two seconds is designed to be missed (Week 2, change blindness); "Saved to your account at 14:32" that remains until the next edit is not.

Showing state. The system knows whether the document is synced, whether the filter is active, whether you are in edit mode. Displaying that state costs a few pixels and prevents an entire class of mistaken action (Week 3).

Acknowledging the start. An action that takes time must show that it has begun, not merely that it has finished. Otherwise the interval belongs to the user's doubt, and doubt produces a second click.

Lab Activity

Lab 8 (2 hours)

1. **Threshold measurement.** Add artificial delays of 0.05, 0.3, 1, 3 and 12 seconds to a button in a test page. Ask five people to rate each as instant / quick / slow / broken. Compare with the published thresholds.

2. **Waiting indicators.** For the 3-second and 12-second delays, build three variants: nothing, a spinner, and a labelled progress bar. Measure how many participants click a second time on each.

3. **Message rewriting.** Collect eight real error messages from systems you use. Classify each as vague, blaming, technical or useful, and rewrite the failures with all four parts.

4. **Prevention audit.** For each of your eight messages, state whether the error could have been prevented by a constraint or accepted by normalisation. Report the proportion that could.

5. **Severity.** Find one interface that uses a modal dialog for something trivial, and one that uses an inline message for something serious. Correct both.

6. **Perceived duration.** Build two versions of a 6-second operation: one with a bare spinner, one with named phases ("Checking availability… Reserving… Confirming"). Ask five people to estimate how long each took.

7. **Project.** Write every error and confirmation message for your prototype. Each must have all four parts, and you must state for each why prevention and acceptance were not possible.

Summary

• **0.1 s** — feels instantaneous, and is what makes direct manipulation work. **1 s** — flow of thought unbroken. **10 s** — the limit of attention.

• Match the indicator to the wait: nothing, a cursor change, a busy indicator, then a **progress bar with an estimate**.

• A **spinner is inadequate beyond ten seconds**; name the phases instead.

• **Perceived duration** is a separate quantity and is cheaper to improve than actual duration — but never ship a fast operation with no feedback.

• Never **freeze**, never let a bar **go backwards**, never **sit at 100%**.

رومانی صبحی رزق جرجس مطر



- An error message needs four parts: **what happened, why, what to do, and where**. Never blame, never bury the instruction in an apology, never use the implementation's vocabulary.
- **Interrupt in proportion to consequence.** A modal for a typo disables the channel for the serious case.
- **Prevent, then accept, then report.** Every message is an opportunity not taken.
- Feedback is also needed for **success, state and starting** — not only for failure.

Self-check

1. State the three response-time thresholds and what each governs.
2. Why does the 0.1-second threshold matter for direct manipulation?
3. What should be shown for a 3-second operation? For a 3-minute one?
4. Give three ways to make a wait feel shorter and three that break trust.
5. List the four parts of a useful error message.
6. Rewrite: "Error: invalid input in field 3."
7. When is a modal dialog the correct response, and what does overusing it cost?
8. State the prevent-accept-report ordering and give an example of each.

(Full solutions are in the Week 8 question bank.)

رومانی صبحی رزق جرجس مطر

Week 9 — Physical Interaction: Devices, Fitts's Law and Interaction Techniques

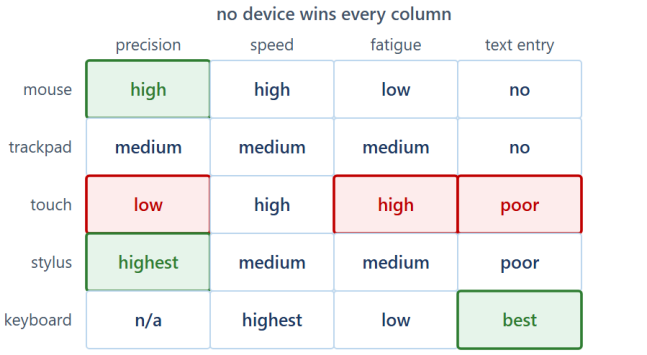
Learning Outcomes

By the end of this week you should be able to:

1. Compare the common input devices and choose one for a task.
2. State Fitts's law, apply it, and derive design consequences from it.
3. Specify minimum target sizes for pointer and touch, and justify them.
4. Account for one-handed reach when placing controls on a phone.
5. Judge which gestures are safe to rely on and which must duplicate a visible control.
6. Identify the ergonomic risks created by an interaction design.
7. Choose an interaction technique from the structure of the choice.

9.1 Devices

2025/20



Choose the device for the task, and expect several to be in use. Most systems are used on more than one.

No device wins every column.

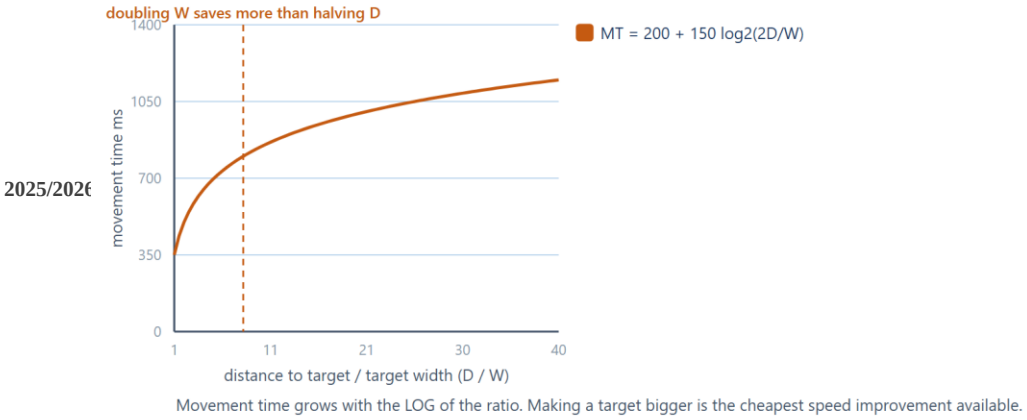
Device	Precision	Fatigue	Good for	Poor for
Mouse	high	low	sustained precise pointing	text; mobile use
Trackpad	medium	medium	portable pointing, gestures	fine dragging
Touch	low	high (arm raised)	large targets; casual use	precision; long sessions
Stylus	highest	medium	drawing, handwriting, annotation	typing
Keyboard	—	low	text; commands;	spatial tasks

Device	Precision	Fatigue	Good for	Poor for
			accessibility	

> **Key Idea**
>
> The important observation is not which device is best but that **most systems are used on more than one**. The same web application is used with a mouse on a desk, with a thumb on a phone, and with a keyboard by someone who cannot use a pointer.
>
> A design that assumes one device will fail on the others — and a hover-only control (Week 5) is the standard example, because touch has no hover at all.

Touch is the least precise of the pointing devices, for two reasons that compound: the finger contact area is roughly a centimetre across, and the finger **obscures the target** at the moment of contact. The pointer has neither problem, which is why a target adequate for a mouse is not adequate for a finger.

9.2 Fitts's law



Movement time grows with the logarithm of the ratio.

> **Definition**
>
> **Fitts's law.** The time to move to a target depends on the distance to it and its size:
>
> $$MT = a + b\log_2\left(\frac{2D}{W}\right)$$
>
> where D is the distance to the target, W its width along the direction of movement, and a, b are empirical constants for the device and the user.

Two properties of that formula do all the work.

The relationship is logarithmic, not linear. Doubling the distance does not double the time — it adds a fixed increment. Distance therefore matters less than intuition suggests.

Width appears in the same ratio as distance, so making a target twice as wide has the same effect as bringing it twice as close. **Size is usually the cheaper change**, since layout constrains distance far more than it constrains size.

Worked Example — Applying the law

The problem. A toolbar button is 16 px wide and about 400 px from where the cursor usually rests. Two proposals: move it to 200 px away, or make it 32 px wide. Which helps more?

Take $a = 200$ ms and $b = 150$ ms, typical for a mouse.

Current: $D/W = 400/16 = 25$, so $MT = 200 + 150\log_2(50) = 200 + 150(5.64) = \mathbf{1046}$ ms.

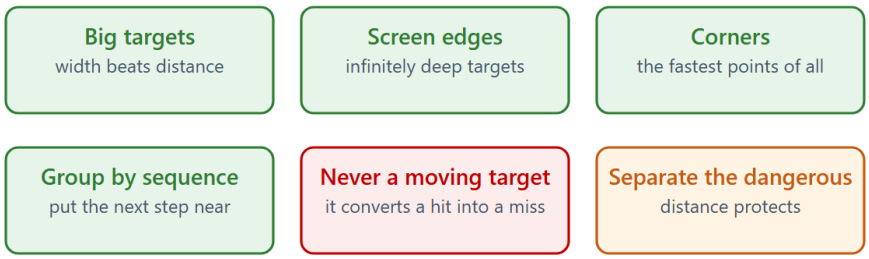
Halve the distance: $D/W = 200/16 = 12.5$, so $MT = 200 + 150\log_2(25) = 200 + 150(4.64) = \mathbf{896}$ ms.

Double the width: $D/W = 400/32 = 12.5$ — **the same ratio**, so **896 ms**.

The finding. They are identical, because only the ratio matters. But doubling the width of one button is a five-minute change, while halving the distance means rearranging the toolbar and affects every other control.

The general rule: when you want a control to be faster, make it bigger before you try to move it.

What follows from the law



The one case where distance is wanted: a destructive control should be far from the routine one beside it.

Six consequences.

Screen edges and corners are special. The pointer stops at the edge of the display, so an edge target is effectively infinitely deep in that direction: you can move towards it as fast as you like without overshooting. A corner is bounded in *two* directions and is the fastest target on the screen. This is why the menu bar at the very top of the screen and the corner Start button are faster than equivalent controls a few pixels inside.

Never move a target as the user reaches for it. A dialog that appears under the cursor, a list that reorders while loading, a button that shifts when an advert loads — each converts a correct action into a wrong one. The user did everything right.

Distance is a safety feature. The one case where you *want* a large D is a destructive control. Separating Delete from Save is Fitts's law used deliberately: the extra distance is the cost of the protection.

9.3 Target sizes

	minimum size	note
mouse pointer	~24 px	precise, visible
finger, phone	~44 px	the finger hides it
finger, moving	~60 px	bus, walking
thumb, one hand	~44 px	reach matters too

The pointer is precise and visible; the finger is neither. Touch targets must be larger than they look necessary.

The pointer is precise and visible; the finger is neither.

Situation	Minimum
Mouse pointer	about 24 px
Finger, phone at rest	about 44 px
Finger, in motion (walking, on a bus)	about 60 px
Spacing between adjacent touch targets	at least 8 px

> **Common Mistake**
>
> **The visual size and the touch target need not be the same.** A 16 px icon can have a 44 px invisible touch area around it. Designers frequently believe a large target means a large-looking control, and reject the guidance for that reason.
>
> The spacing requirement matters as much as the size: two adjacent 44 px targets with no gap will still produce wrong hits, because the finger's contact area straddles the boundary.

9.4 Reach

one-handed reach on a large phone (right hand)

	left	centre	right
top	hard	hard	stretch
middle	stretch	easy	easy
bottom	stretch	easy	easy

Primary actions belong in the lower half. A destructive action in the easy zone will be pressed by accident.

One-handed reach on a large phone.

A large phone held in one hand cannot be reached everywhere by the thumb. The lower and centre regions are comfortable; the top corners require a stretch or a change of grip.

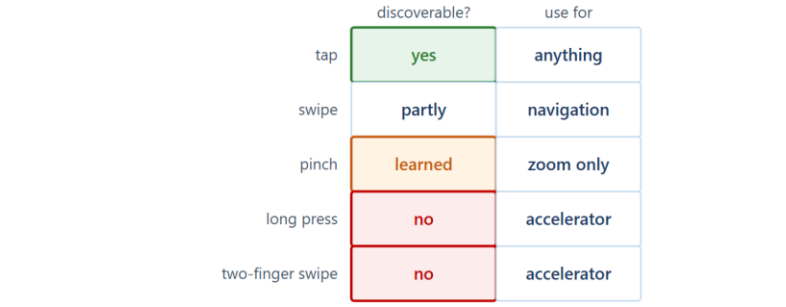
Two consequences:

- **Primary actions belong in the lower half** of a phone screen — which is why mobile navigation bars migrated from the top to the bottom.

- **A destructive action in the easy zone will be pressed by accident.** Place it where a deliberate stretch is required, which is Fitts's law used for safety.

> **Note**
>
> Do not over-apply this. Not everyone holds a phone the same way, holds it in the same hand, or holds it in one hand at all. Treat reach as one input to layout among several, and confirm it by observing your own users rather than by assuming a single grip.

9.5 Gestures



Only tap is reliably discoverable. Everything below it must duplicate a visible control, never replace one.

Only tap is reliably discoverable.

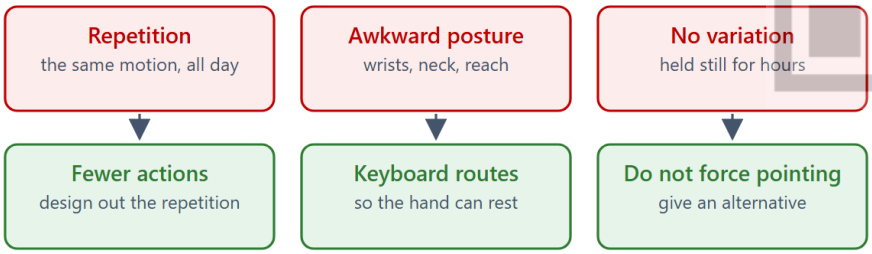
Gestures are efficient once known and invisible until then. The rule from Week 5 applies with full force here:

> **Key Idea**
>
> **Every gesture below "tap" must duplicate a visible control, never replace one.**
>
> A swipe to delete is a good accelerator and a bad only-route. A long press that is the sole means of reaching a function means most users do not have that function.
>
> Two further reasons, beyond discoverability: gestures are **hard or impossible for users with motor impairments**, and they are **unavailable to keyboard and screen-reader users** unless a parallel route exists.

Where gestures are used:

- **Follow platform conventions** exactly. A gesture that means one thing system-wide and another in your application is worse than no gesture.
- **Provide a visible hint** — a partially revealed action, a handle, a first-run cue.
- **Make them forgiving.** Generous thresholds, and an animated return when a gesture is abandoned so the user sees that nothing happened.

9.6 Ergonomics



Injury comes from repetition and posture. A design that needs 900 precise clicks a day is a health question.

Injury comes from repetition and posture.

Interaction design has physical consequences for anyone who uses a system all day. The risk factors are well established: **repetition**, **awkward posture**, **force**, and **absence of variation**.

> **Key Idea**
>
> A design that requires 900 precise clicks a day is not merely inefficient; it is a health question, and it is created by the software rather than by the hardware.
>
> The strongest ergonomic improvement available to a designer is usually **removing the repetition** — bulk operations, keyboard routes, defaults, and rules — rather than adjusting the furniture.

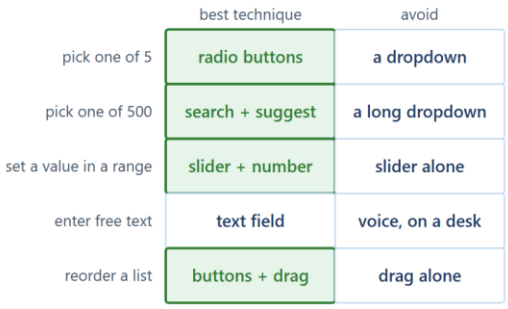
2025/2026 2025/2026 2025/2026

Specific measures:

- **Provide keyboard equivalents** for frequent actions, so the hand can leave the mouse. This is also the accessibility requirement of Week 4.
- **Do not require sustained precise pointing.** A task that needs 3 px accuracy for an hour is a poorly chosen technique.
- **Do not require dragging where clicking would do**, and never require a drag held over a long distance.
- **Beware "gorilla arm."** A vertical touch screen at arm's length is exhausting within minutes, which is why touch has not replaced the mouse on the desktop.

رومانى صبحى رزق جرجس مطر

9.7 Choosing a technique



The technique follows from the size of the choice set and the precision required — not from what looks modern.

The technique follows from the structure of the choice.

The choice	Use	Avoid
one of 2–5	radio buttons, or segmented control — all visible	a dropdown, which hides the options
one of 6–20	a dropdown	radio buttons, which occupy the screen
one of hundreds	search with suggestions	any list, which becomes a serial search
several of a few	checkboxes	a multi-select list, which is easy to mis-operate
a value in a range	slider **and** a number field	a slider alone, which cannot be set precisely
a date	a picker **and** a typed field	a picker alone, which is slow for a known date
free text	a text field	voice, at a desk
an ordering	up/down buttons **and** drag	drag alone

2025/2026 2025/2026 2025/2026

Tip

Notice how often the answer is **both**: slider and number, picker and typed entry, drag and buttons. Each pair offers an approximate method that is fast and a precise method that is reliable, and it also provides the keyboard route.

When you are unsure, offering both is usually right — and it is what a serious tool does.

Lab Activity

Lab 9 (2 hours)

1. **Fitts's law measurement.** Build a page with a start button and a target of variable size and distance. Record movement time over at least five combinations, with five participants. Plot MT against $\log_2(2D/W)$ and fit a line. Report a and b.

2. **Repeat with touch** on a phone, and compare the constants.

3. **Edges.** Time clicking a 4 px strip at the very edge of the window against an identical strip 20 px inside. Explain the difference.

4. **Target sizes.** Build a page with targets of 20, 32, 44 and 60 px. Measure the error rate for each with a finger, standing, and again while walking slowly.

5. **Reach.** Ask five people to hold a phone one-handed and tap targets in the nine regions. Record which require a grip change.

6. **Gesture audit.** List every gesture in an application you use. For each, state whether a visible control performs the same function.

7. **Project.** Specify the minimum target size for your prototype, place your primary and destructive actions with Fitts's law in mind, and state the keyboard route for every action.

Summary

• No device wins every column, and **most systems are used on several** — so a hover-only control excludes touch users.

• **Touch is the least precise** pointing device: a wide contact area, and the finger hides the target.

• **Fitts's law:** $MT = a + b\log_2(2D/W)$. The relationship is **logarithmic**, and **width trades against distance** — so enlarging a target is usually the cheaper improvement.

• **Edges and corners** are the fastest targets, because the pointer cannot overshoot them.

• **Never move a target** as the user reaches for it; **distance is a safety feature** for destructive controls.

• Minimum sizes: **24 px pointer, 44 px finger, 60 px in motion**, with at least 8 px between targets. The touch area need not equal the visual size.

• **Primary actions belong in the lower half** of a phone screen; destructive ones outside the easy reach zone.

• **Only tap is reliably discoverable.** Every other gesture must duplicate a visible control.

• **Ergonomics is a design output:** the best remedy is removing repetition, not adjusting furniture.

• The technique follows from **the size of the choice set and the precision required** — and the answer is often **both** an approximate and a precise method.

Self-check

1. Why is touch less precise than a mouse? Give two distinct reasons.
2. State Fitts's law and explain what the logarithm implies.
3. A button is 400 px away and 16 px wide. Compare halving the distance with doubling the width.
4. Why are screen corners the fastest targets?
5. Give the minimum target size for a finger, and explain why visual size may differ.
6. Where should a destructive action be placed on a phone, and why?
7. What is the rule for gestures below "tap"?

رومانی صبحی رزق جرجس مطر

8. Give two ergonomic remedies that are properties of the software, not the furniture.

(Full solutions are in the Week 9 question bank.)

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

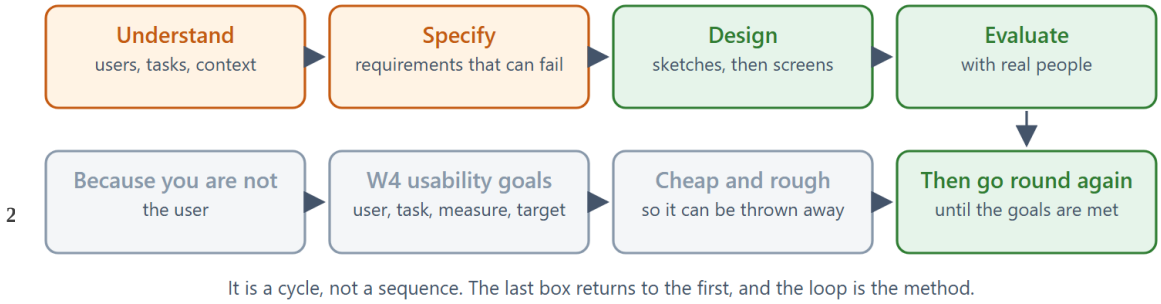
Week 10 — The Design Process and User Models

Learning Outcomes

By the end of this week you should be able to:

1. Describe the user-centred design cycle and explain why it is a cycle.
2. Explain why a designer's intuition about users is unreliable, specifically.
3. Choose a method for gathering requirements and state what each method cannot tell you.
4. Perform a hierarchical task analysis on a real goal.
5. Write a persona that can settle a design argument, and say what makes one useless.
6. Write scenarios that become usability test tasks.

10.1 User-centred design

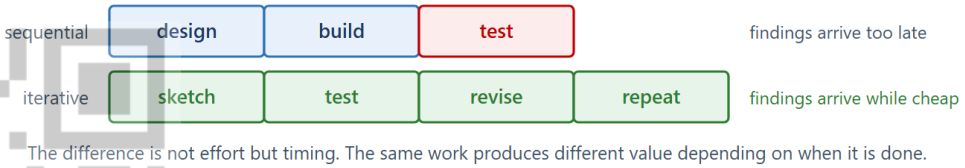


A cycle, not a sequence.

Definition

User-centred design is an approach in which the intended users, their tasks and their context are studied first and repeatedly, requirements are specified in terms that can be tested, designs are evaluated with real users, and the process repeats until the requirements are met.

The four activities are familiar from earlier weeks — Week 4 supplied the requirements, Week 5 the design vocabulary, and Week 11 will supply the evaluation. What this week adds is the **order**, and the insistence that it loops.



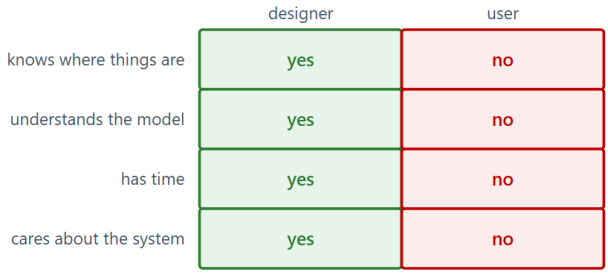
The difference is timing, not effort.

Key Idea

The same work — designing, building, testing — produces completely different value depending on **when** the testing happens.

> Test at the end and you learn what is wrong when it is most expensive to change and you have least freedom to change it (Week 4). Test at the sketch stage and the same finding costs a rubber.
>
> Iteration is not repetition caused by failure. It is the method.

10.2 Why your intuition is unreliable



Four ways in which you are unlike your user. Every one of them makes your intuition optimistic.

Four ways you differ from your user.

This is the hardest idea in the course to accept, because it cannot be checked from the inside.

	You	Your user
Knows where every feature is	yes — you put it there	no
Understands the underlying model	yes	no, and will invent one (Week 3)
Has unlimited time for the interface	yes, it is your job	no, they have a task
Cares about the system itself	yes	no, they care about the outcome

> **Common Mistake**
>
> Every one of these differences makes your judgement **optimistic in the same direction**. You will consistently find your own interface clearer, faster and more obvious than any user will.
>
> This is not a failure of skill or attention. It is a structural consequence of knowing the answer, and it does not diminish with experience — experienced designers are simply more aware of it, and therefore test.

There is a second, related trap: **you are not designing for people like you.** Computer science students are unusually tolerant of complexity, unusually willing to experiment, and unusually able to recover from errors. A design that your classmates can use tells you very little about whether a registrar's clerk or a first-year student can.

رومانى صبحى رزق جرجس مطر
2025/2026
2025/2026
2025/2026

10.3 Gathering requirements



People are unreliable reporters of their own behaviour. Watch as well as ask, and reconcile the difference.

Each method tells you something different, and misses something.

Method	Tells you	Cannot tell you
Interviews	what people believe and remember	what they actually do
Observation	what they actually do	why they did it
Existing data (logs, support tickets)	what happened, at scale	the reasons, or the near-misses
The client	what the organisation wants	how the work is really performed

2025/2026 2025/2026 2025/2026

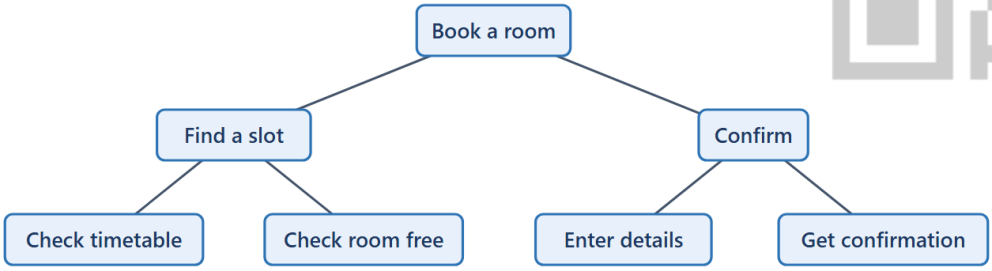
> **Key Idea**
>
> **People are unreliable reporters of their own behaviour** — not dishonest, but they remember the intended process rather than the actual one, omit the workarounds they no longer notice, and describe what they believe they should do.
>
> So **watch as well as ask**, and where the two disagree, treat the discrepancy as the finding. The gap between the described procedure and the observed one is usually where the design problem is.

Contextual inquiry — observing the work in its real setting and asking about it *there*, as it happens — combines both and is the most productive single technique. Two rules make it work:

1. **Watch first, ask afterwards.** Questions during the task change the task.
2. **Ask about the exception.** "What do you do when the code is missing?" reveals the workarounds, and the workarounds are where the requirements hide.

رومانی صبحی رزق جرجس مطر

10.4 Task analysis



Hierarchical task analysis: break the goal into sub-tasks until each is a single observable action.

Decompose until each leaf is a single observable action.

> **Definition**
>
> **Hierarchical task analysis** breaks a goal into sub-tasks, and those into smaller sub-tasks, until each leaf is a single observable action. A **plan** at each level states the order and the conditions.

Example — "book a laboratory".

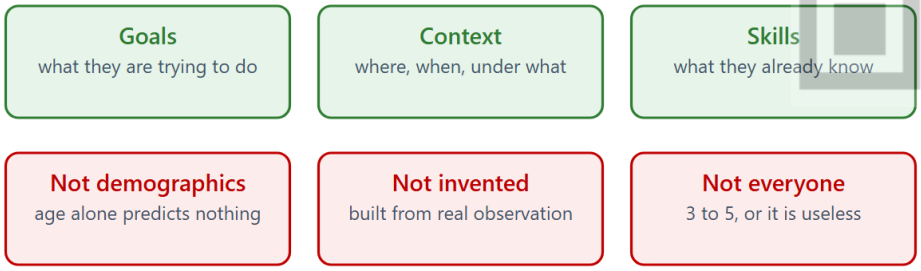
```
0. Book a laboratory                    plan: 1, then 2, then 3
   1. Decide what is needed             plan: 1.1 then 1.2
      1.1 Choose a date and time
      1.2 Determine the equipment required
   2. Find an available room            plan: 2.1, then 2.2; if none, back to 1.1
      2.1 Filter rooms by equipment
      2.2 Check availability at that time
   3. Confirm the booking               plan: 3.1 then 3.2
      3.1 Enter purpose and attendee count
      3.2 Receive and record confirmation
```

> **Key Idea**
>
> Two things make task analysis worth the effort.
>
> **It reveals steps nobody mentioned.** Step 2's plan — *if none available, return to 1.1* — is a loop that every real user performs and that no requirements document contains. An interface that does not support it forces users to start again from the beginning.
>
> **It separates the task from the current system.** "Log in, click Rooms, select from dropdown" describes an implementation. "Find an available room" describes the goal, and leaves you free to satisfy it differently.

> **Tip**
>
> The commonest mistake is to analyse the *existing interface* rather than the *task*. If your analysis contains verbs like "click", "scroll" or "select from the list", you are describing a system, not a goal. Rewrite each step to say what the user is trying to achieve.

رومانی صبحی رزق جرجس مطر

10.5 Personas



A persona is a decision-making tool. If it cannot settle an argument about a feature, it is decoration.

What a persona contains, and what it must not be.

> **Definition**
>
> A **persona** is a concrete description of a representative user — their goals, context, skills and constraints — constructed from real observation and used to make design decisions.

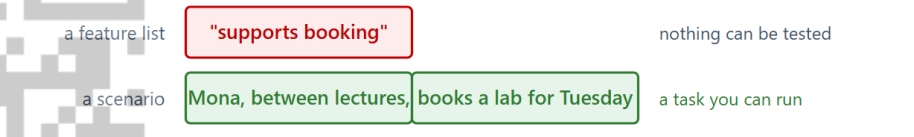
A usable persona:

> **Mona, 20, second-year student.** Uses the portal three or four times a term, almost entirely on a phone, usually between lectures with a few minutes to spare. Confident with apps, unfamiliar with academic regulations and unsure which deadlines apply to her. Wants to finish the task and get on with her day. *Will abandon a form that asks for information she does not have with her.*

Why that last sentence matters: it is a prediction that can settle an argument. Asked "should we require the national ID number on this form?", the persona answers.

> **Common Mistake**
>
> **Three ways personas go wrong.**
>
> *Demographics instead of behaviour.* "Ahmed, 34, married, likes football" predicts nothing about any design decision. Age and marital status are not design inputs; goals, context and prior knowledge are.
>
> *Invented rather than observed.* A persona assembled from assumptions is your own intuition wearing a photograph — and §10.2 explains why that is the least reliable input available.
>
> *Too many.* Beyond three to five, every feature is justified by some persona and the tool stops constraining anything. The value of a persona set is what it **excludes**.

10.6 Scenarios



A scenario names a person, a situation and a goal — which makes it something a usability test can use directly.

A scenario is a task a test can run.

Definition

A **scenario** is a short narrative describing a specific person, in a specific situation, achieving a specific goal — with no reference to the interface.

Mona is between lectures with seven minutes. She needs to book the networks laboratory for her group project on Tuesday afternoon, and she does not know which rooms have the equipment she needs. She has her phone and no documents with her.

Why the "no reference to the interface" clause matters. A scenario written as *"Mona clicks Rooms, filters by equipment, and selects Tuesday"* has assumed the design, so it can no longer be used to evaluate it — and it certainly cannot be given to a test participant, because it contains the answer.

Key Idea

Scenarios become the tasks in the usability test. This is why they are written before the design and in the user's vocabulary: next week you will hand them to participants almost verbatim.

That continuity is the practical reason the artefacts of this week are worth producing. Each one feeds the next, and the last one is what you test.



Each artefact feeds the next, and the scenarios become the tasks in the usability test of Week 11.

Each artefact feeds the next.

2025/2026

2025/2026

10.7 Putting it together

The sequence for your project, and for any real one:

1. **Observe** three to five real users performing the task as it is done today.
2. **Task-analyse** the goal, separating it from the current system.
3. **Write three personas** from what you observed, each with a prediction.
4. **Write five scenarios**, in the user's words, with no interface in them.
5. **Write usability goals** (Week 4 form) for the three most important scenarios.
6. **Sketch**, test, revise — Week 11.

Note

Steps 1–5 take about a week for a student project and are frequently skipped because they produce no visible interface. They are what makes the sketching productive: without them you are designing for an imagined user performing an imagined task, and the testing in Week 11 will have nothing to measure against.

Lab Activity

Lab 10 (2 hours)

1. **Contextual inquiry.** Observe one person performing your project's task as it is done today — on paper, in an office, or in an existing system. Watch first, ask afterwards, and ask specifically about exceptions.

2. **Interview versus observation.** Before observing, ask the person to describe how they do the task. Afterwards, compare the description with what you saw and list every discrepancy. The discrepancies are your findings.

3. **Task analysis.** Produce a hierarchical task analysis of your project's main goal, with plans. Check that no leaf contains the word "click".

4. **Personas.** Write three personas from your observations. Each must end with a sentence that predicts a behaviour and could settle a design argument.

5. **Scenarios.** Write five scenarios. Give them to another group and ask whether each could be handed to a test participant as written.

6. **Usability goals.** Write three goals in the Week 4 form, based on your scenarios.

7. **Project.** All of the above, assembled into a short design document. This is assessed, and it is the input to Week 11.

Summary

• **User-centred design** is a **cycle**: understand, specify, design, evaluate, repeat. The same work produces different value depending on when the evaluation happens.

• **Your intuition is unreliable in a predictable direction** — you know where everything is, understand the model, have time, and care. All four make you optimistic.

• You are also **not designing for people like you**: computer science students tolerate complexity that ordinary users will not.

• **Interviews tell you beliefs; observation tells you behaviour.** Where they disagree, the discrepancy is the finding.

• **Contextual inquiry** combines both: watch first, ask afterwards, and ask about exceptions.

• **Task analysis** decomposes a goal to observable actions and reveals steps nobody mentioned — especially loops and exceptions. Analyse the **task**, not the existing interface.

• A **persona** is built from observation, describes goals and context rather than demographics, and ends with a prediction. Three to five, or it constrains nothing.

• A **scenario** names a person, a situation and a goal, with **no interface in it** — because it becomes the task in next week's usability test.

Self-check

1. Why is user-centred design described as a cycle rather than a sequence?
2. Give four specific ways in which a designer differs from a user, and say which direction the bias runs.
3. What can an interview tell you that observation cannot, and vice versa?
4. What are the two rules of contextual inquiry?
5. What is wrong with a task analysis whose leaves contain the word "click"?
6. Give two features of a useless persona and two of a useful one.
7. Why must a scenario contain no reference to the interface?
8. What is the relationship between a scenario and a usability test task?

رومانی صبحی رزق جرجس مطر

(Full solutions are in the Week 10 question bank.)

رومانى صبحى رزق جرجس مطر

2025/2026 **2025/2026** **2025/2026**

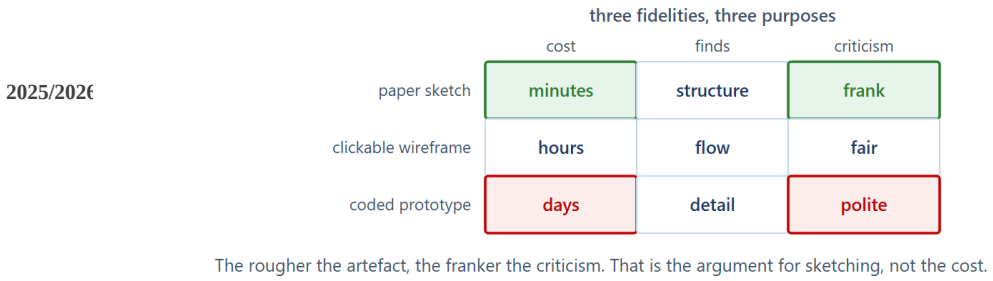
Week 11 — Interface Evaluation: Rapid Prototyping and Iterative Refinement

Learning Outcomes

By the end of this week you should be able to:

1. Choose a prototype fidelity appropriate to the question being asked.
2. Explain why low-fidelity prototypes produce better feedback.
3. Plan and run a usability test with five participants.
4. Facilitate without contaminating the result.
5. Conduct a heuristic evaluation and say how it differs from user testing.
6. Rate findings by severity and write a report that leads to action.
7. Iterate: decide what to change, change it, and re-test.

11.1 Prototyping



Three fidelities, three purposes.

Definition

A **prototype** is a representation of a design built to answer a question. Its fidelity should be the lowest that can answer the question being asked.

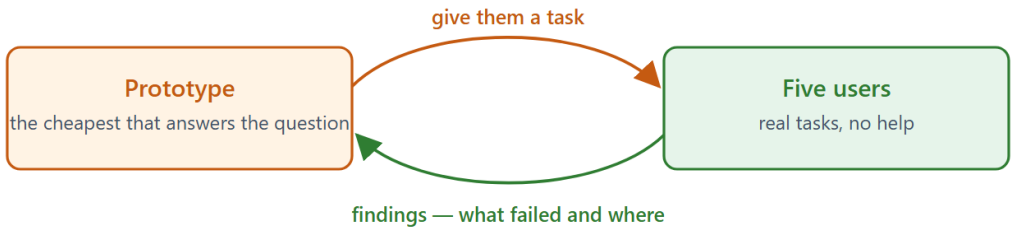
Fidelity	Cost	Answers	Does not answer
Paper sketch	minutes	is the structure right? are the steps in the right order?	does it feel responsive?
Clickable wireframe	hours	can people find things? does the flow work?	is the visual design right?
Coded prototype	days	does the whole thing work, with real data and timing?	— but it is expensive to change

Key Idea

> **The rougher the artefact, the franker the criticism.**
>
> Shown a polished screen, participants and colleagues assume the decisions are settled. They soften their language, attribute problems to their own confusion, and suggest small adjustments.
>
> Shown a pencil sketch, the same people say plainly that the order is wrong and that they would not know what to press.
>
> This is the strongest argument for low fidelity, and it is not the cost argument. A rough prototype produces **better information**, not merely cheaper information.

Paper prototyping in practice. Draw each screen on a separate sheet. One team member plays the computer, placing the next sheet in response to the participant's "clicks". It feels absurd for about ninety seconds, and then it works — and it routinely finds structural problems that would otherwise have surfaced weeks later.

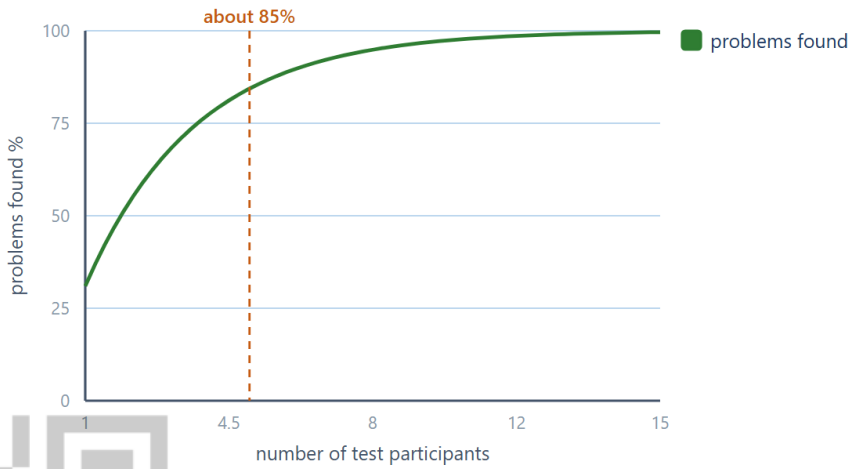
11.2 The iteration loop



20

Two or three small cycles beat one large study. The loop is the method; the prototype is disposable.

The loop is the method; the prototype is disposable.



Diminishing returns after five. Better to run three tests of five than one test of fifteen.

Diminishing returns after five participants.

> **Key Idea**
>
> **Five participants find roughly 85% of the problems**, on the standard model in which each participant independently has about a 31% chance of encountering any given problem.

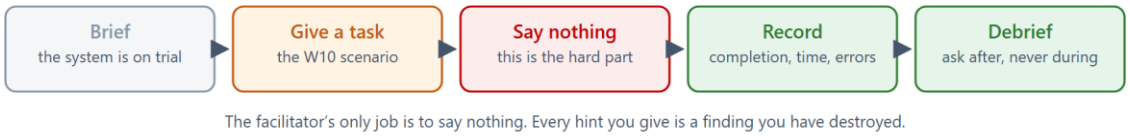
The consequence that matters is not "test with five". It is: **three tests of five beat one test of fifteen.** Fifteen participants on one design find slightly more problems in that design; five participants on three successive designs find problems in all three, and you have fixed two of them before the third test.

Note

A caution on the number. Five is right for a *formative* test — finding problems in order to fix them. It is **not** enough to measure a completion rate precisely, and you should never report "80%" from five participants. Say "four of five", which is honest about the precision.

Five is also per *user group*. If novices and experts differ substantially, you need five of each.

11.3 Running a usability test



Five stages, and the third is the hard one.

Before.

- **Tasks** — take them from your Week 10 scenarios, almost verbatim. They must have a definite end and contain no interface vocabulary.
- **Define "error" and the timer's start and end** in advance (Week 4).
- **Recruit** participants who resemble the intended users, and who have not seen the design.
- **Pilot** with one person first. You will find a broken task statement, and it is better to lose the pilot than a real participant.

During.

1. **Brief.** Explain that **the system is being tested, not the participant**, that there are no wrong answers, and that you will not help. Ask them to think aloud.
2. **Give one task at a time**, read aloud and also on paper.
3. **Say nothing.** Wait at least 60 seconds after they get stuck.
4. **Record** completion, time, errors and the point of failure.
5. **Debrief afterwards** — never during.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

	never say	say instead
they ask for help	"click the blue button"	"what would you try?"
they get stuck	"it is under Settings"	*silence, 60 seconds*
they apologise	"do not worry, it is hard"	"we are testing the system"
they succeed	"well done!"	"thank you — next task"

Encouragement changes behaviour as much as hints do. Stay neutral in both directions.

Stay neutral in both directions.

Common Mistake

The facilitator's only job is to say nothing, and it is genuinely difficult.

Every hint destroys the finding you were there to collect. So does *encouragement*: "well done!" tells the participant that the previous action was correct, which changes their next one. So does sympathy: "don't worry, everyone finds this hard" tells them the system is at fault and alters what they report.

Neutral responses: *"what would you try?"*, *"what do you expect to happen?"*, *"thank you — the next task is…"*

2025/2026 2025/2026 2025/2026

11.4 Heuristic evaluation

	finds	needs	cost
user testing	real failures	5 users	a day
heuristic evaluation	likely failures	3 experts	hours

They find different problems, so use both. Heuristic evaluation first, to avoid wasting users on obvious defects.

Two methods, different findings.

Definition

Heuristic evaluation is an inspection method in which several evaluators independently examine an interface against a list of principles, then combine their findings.

The usual heuristics are Nielsen's ten, which overlap heavily with Week 5: visibility of system status; match between system and the real world; user control and freedom; consistency and standards; error prevention; recognition rather than recall; flexibility and efficiency of use; aesthetic and minimalist design; help users recognise, diagnose and recover from errors; help and documentation.

Procedure.

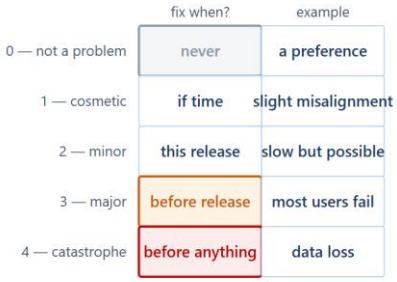
1. **Three to five evaluators**, working **independently** — this matters, since evaluators find different problems and discussion before recording suppresses disagreement.

رومانی صبحی رزق جرجس مطر

2. Each goes through the interface **twice**: once for flow, once against the heuristics.
3. Each records findings as **location + heuristic violated + severity**.
4. **Then** combine, merge duplicates and agree severities together.

> **Key Idea**
>
> **Use both methods, heuristic evaluation first.** They find different problems: inspection finds violations of known principles, including ones users tolerate silently; user testing finds real failures, including ones no principle predicts.
>
> Running the inspection first means you do not spend five participants discovering that the button is unlabelled.

11.5 Severity



A flat list of twenty findings gets filed. Three ranked findings get fixed. Ranking is part of the deliverable.

2025/2026

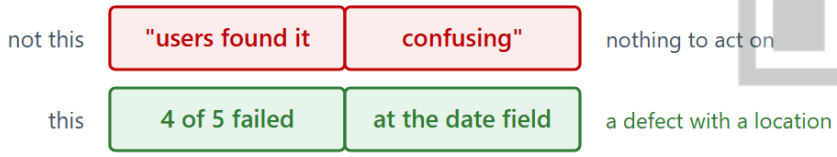
Ranking is part of the deliverable.

Rating	Meaning	Action
0	not a usability problem	none
1	cosmetic	fix if time permits
2	minor — slows users	fix in this release
3	major — most users fail or are seriously delayed	fix before release
4	catastrophic — data loss, task impossible, safety	fix before anything else

Severity combines three factors: **how many users hit it**, **how badly it affects them**, and **whether it persists** after they have encountered it once.

> **Common Mistake**
>
> **An unranked list of twenty findings gets filed and forgotten.** A team cannot act on twenty things, so it acts on none, and the report is remembered as "the usability people complained about everything".
>
> Three ranked findings, each with a proposed change, get fixed. **Prioritisation is part of your job, not the client's.**

11.6 Reporting



A finding names how many, what happened, and where. Anything less is an impression.

A finding names how many, what, and where.

A usability finding has four parts:

1. **How many participants** encountered it — "4 of 5", never "80%".
2. **What happened**, described behaviourally — what they did, not how they felt.
3. **Where** — the specific screen and control.
4. **The proposed change**, specific enough to implement.

> **4 of 5 participants failed at the date field.** All four typed `01/05/2026` and received "invalid format"; two tried twice more and abandoned. **Severity 3.** *Accept any common date format and normalise it; show the interpreted date back for confirmation.*

Compare: *"users found the form confusing"*, which names no location, no frequency and no change.

> **Tip**
>
> **Include one short video clip per major finding**, with permission. Thirty seconds of a real user failing ends an argument that a paragraph cannot. Pair it with the numbers: the clip establishes that the problem is real, the numbers establish that it is frequent.

11.7 Iterating

The loop closes here:

1. **Rank** the findings by severity.
2. **Fix the top three.** Not all twenty — the top three.
3. **Re-test with five new participants.**
4. **Check that the fixes worked** *and that nothing new broke*.

> **Key Idea**
>
> **A fix is a hypothesis.** You believed the date field was the problem and you changed it; whether that helped is a question, not a fact.
>
> Re-testing is what distinguishes iterative design from redesigning repeatedly on the basis of opinion — and it occasionally shows that the fix made things worse, which is precisely the information you needed.
>
> **Lab Activity**
>
> **Lab 11 (2 hours) — the central lab of the course.**

رومانی صبحی رزق جرجس مطر

1. **Paper prototype.** Produce the five main screens of your project on paper, one sheet each.
2. **Pilot.** Run one participant to check the task statements. Fix whatever breaks.
3. **Test with five.** One facilitator, one note-taker. Use your Week 10 scenarios as tasks. Record completion, time, errors and failure points. The facilitator says nothing.
4. **Heuristic evaluation.** Independently, three of you evaluate another group's prototype against Nielsen's ten. Combine afterwards.
5. **Rank.** Assign a severity 0–4 to every finding from both methods.
6. **Fix the top three** and re-test with three new participants. Report whether the fixes worked.
7. **Project deliverable:** the test plan, the results table, the ranked findings, the changes made, and the re-test result. This is the main assessed component of the practical mark.

Summary

• A **prototype answers a question**; use the lowest fidelity that can answer it.

• **Rough artefacts get franker criticism** — this is the main argument for low fidelity, ahead of cost.

• **Five participants find about 85% of problems**, so **three tests of five beat one test of fifteen**. Five is formative, not a basis for reporting percentages.

• **The facilitator's job is to say nothing.** Hints, encouragement and sympathy all contaminate the result.

• **Define "error" and the timer boundaries in advance**; **pilot** before the real sessions.

• **Heuristic evaluation** finds principle violations; **user testing** finds real failures. Use both, inspection first.

• **Severity 0–4**, combining frequency, impact and persistence. **Rank your findings** — an unranked list of twenty gets filed.

• A finding states **how many, what, where, and the proposed change** — "4 of 5", never "80%".

• **A fix is a hypothesis.** Re-test it.

Self-check

1. What determines the right fidelity for a prototype?
2. Give the two arguments for low fidelity, and say which is stronger.
3. Why do three tests of five beat one test of fifteen?
4. Why must a facilitator avoid praise as well as hints?
5. Why should "error" be defined before the session starts?
6. How does heuristic evaluation differ from user testing, and in what order should they be run?
7. What are the three components of a severity rating?
8. Rewrite as a proper finding: "several people had trouble with the login."

(Full solutions are in the Week 11 question bank.)

رومانى صبحى رزق جرجس مطر

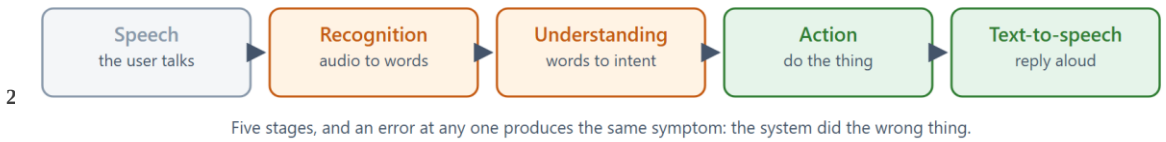
Week 12 — Natural Language and Voice Interfaces, and Text-to-Speech

Learning Outcomes

By the end of this week you should be able to:

1. Describe the stages of a voice interaction and where each can fail.
2. Distinguish a recognition error from an understanding error, and give the remedy for each.
3. Decide whether voice is the right choice for a given task.
4. Explain why a voice interface has no visibility, and how to compensate.
5. Choose a confirmation strategy proportionate to consequence.
6. Write output for the ear rather than for the eye.
7. Explain how text-to-speech serves accessibility, and what your markup determines.

12.1 The stages of a voice interaction



Five stages, and one symptom for every failure.

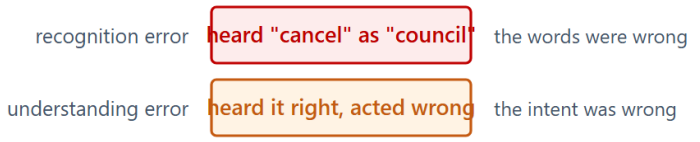
Stage	What happens	Fails when
Speech	the user talks	noise, accent, hesitation
Recognition	audio becomes words	similar-sounding words, unfamiliar accent
Understanding	words become an intent	ambiguity, unsupported phrasing
Action	the system does something	the intent maps to the wrong action
Text-to-speech	the system replies	the reply is too long, or unclear

Key Idea

Every one of these failures produces the same symptom: the system did something other than what the user wanted. From the user's side the cause is invisible, so they cannot adapt — they do not know whether to speak more clearly, use different words, or give up.

That single fact is the source of most of the difficulty in voice design.

12.2 Two kinds of error



Two different failures needing two different remedies — and the user cannot tell which one occurred.

Two failures needing two remedies.

> **Definition**
>
> A **recognition error** is a failure to convert audio into the correct words — “cancel” heard as “council”.
>
> An **understanding error** is a failure to convert correct words into the correct intent — the system heard exactly what was said and did the wrong thing.

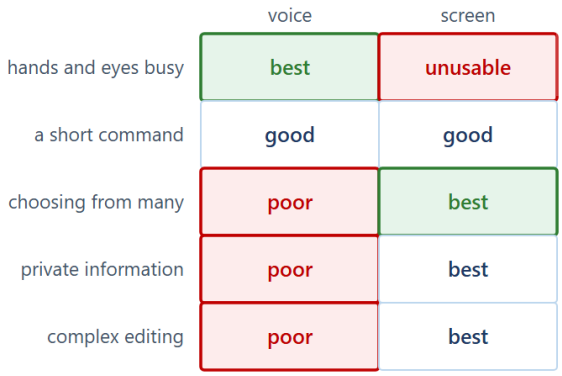
The remedies are different:

	Remedy
Recognition	better models; a constrained vocabulary; **showing what was heard** so the user can correct it
Understanding	support more phrasings; ask a clarifying question rather than guessing; confirm the intent before acting

> **Tip**
>
> **Display or speak back what was heard.** This single measure converts an invisible failure into a visible one: the user sees “council” and knows to repeat the word rather than rephrasing the whole request.
>
> It is the voice equivalent of the Week 6 rule — show the interpretation back — and it is the cheapest improvement available in a voice interface.

2025/2026 2025/2026 2025/2026

12.3 When voice is the right choice



Voice wins where a screen cannot be used at all. Elsewhere it usually loses, and loudly.

Voice wins where a screen cannot be used at all.

Situation	Voice	Why
Hands and eyes are busy — driving, cooking, surgery	**best**	the only available channel
A short, well-known command	good	faster than unlocking and navigating
Choosing among many options	poor	the options must be read aloud, serially
Private or sensitive information	poor	everyone nearby hears it
Complex editing or comparison	poor	no spatial arrangement, no scanning
Accessibility, for a user who cannot see the screen	**essential**	§12.7

> **Key Idea**
>
> Voice is not a better interface. It is a **different channel**, excellent where a screen is unavailable and weak almost everywhere else.
>
> The commonest mistake is adopting voice because it is available, for tasks where a screen is perfectly usable — which produces an interface that is slower, less private and less reliable than the buttons it replaced.

2025/2026 2025/2026 2025/2026

12.4 The invisibility problem



A voice interface has no visibility at all. Every principle of Week 5 must be rebuilt from nothing.

Every principle of Week 5 must be rebuilt from nothing.

A voice interface has **no visibility whatsoever**. There is no menu, no affordance, no display of the current state, and no way for the user to see what is possible.

Three specific consequences:

Nobody knows what to say. A screen shows its options; a microphone shows nothing. Users fall back on a small number of phrases they have found to work and never discover the rest — the same collapse as an unlabelled toolbar (Week 7), but total.

Nobody knows the current state. Is it listening? Did it hear? Is it working? The evaluation gulf (Week 1) is unbridged by default, because there is nothing to perceive.

2025/2026 2025/2026 2025/2026

Errors cannot be located. When something goes wrong the user cannot inspect anything — they can only try again, differently, with no information about what to change.

Compensations, none of them complete:

- **Suggest on first use and after failure** — "you can ask me about your timetable, your fees or your results."
- **Confirm what was heard** before acting.
- **Give audible state cues** — a sound for listening, a different one for processing, so the state is at least perceptible.
- **Offer a screen wherever one exists.** A voice assistant on a phone should display what it heard and what it will do.

رومانی صبحی رزق جرجس مطر

12.5 Confirmation

	confirm?	why
"what is the weather?"	no	harmless if wrong
"turn on the lamp"	implicit	visible, reversible
"send 500 pounds"	explicit	costly and hard to undo
"delete everything"	explicit + repeat	catastrophic

Confirm in proportion to consequence — the Week 8 rule, applied where the user cannot see the state.

Confirm in proportion to consequence.

The Week 8 severity rule applies with extra force, because the user cannot see the state and therefore cannot detect an error before it takes effect.

Request	Strategy
“What is the weather?”	**none** — harmless if misheard
“Turn on the lamp”	**implicit** — the lamp comes on; the result is visible and reversible
“Send 500 pounds to Ahmed”	**explicit** — repeat the amount and recipient, and wait
“Delete all my files”	**explicit, with a distinct response** — and an undo period

2025/2026 2025/2026 2025/2026

Definition

Implicit confirmation folds the understanding into the response: *“Turning on the lamp.”* It costs nothing and lets the user interrupt.

Explicit confirmation requires an affirmative before acting: *“Send 500 pounds to Ahmed Hassan — is that right?”*

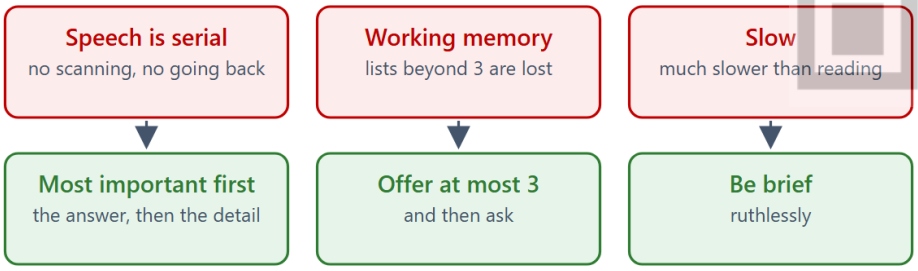
Common Mistake

Over-confirming destroys a voice interface faster than any other error. A system that asks “are you sure?” after every request is slower than the screen it replaced, and users answer “yes” reflexively — which removes the protection for the one case that mattered.

Same mechanism as Week 8’s dialogs, with less tolerance: a spoken confirmation costs two or three seconds of the user’s time, every time.

رومانى صبحى رزق جرجس مطر

12.6 Writing for the ear



Writing for the ear is not writing for the eye. Everything from Week 2 about scanning is unavailable.

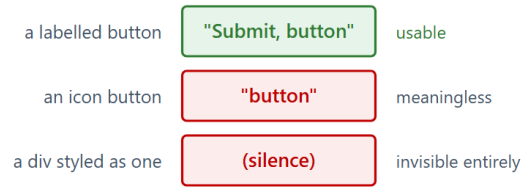
Speech is serial, slow, and gone.

Text-to-speech output obeys different constraints from text on a screen:

- **It is serial.** There is no scanning, no F-pattern (Week 2), no skipping ahead, and no glancing back.
- **It is transient.** Once spoken it is gone, so everything must be held in working memory — about four chunks (Week 3).
- **It is slow.** Speech is considerably slower than reading, so length costs the user directly.

> **Key Idea**
>
> **Three rules for spoken output.**
>
> 1. **Most important first.** "It will rain this afternoon" before the detail. The user should be able to stop listening after the first sentence.
>
> 2. **At most three options, then ask.** "I found three rooms: B2-114, B2-116 and B3-201. Which would you like?" Beyond three, the earlier items are gone.
>
> 3. **Be ruthlessly brief.** Every additional clause costs seconds and occupies memory. Pleasantries that read as polite on a screen are an obstruction when spoken.

12.7 Text-to-speech and accessibility



Text-to-speech is most used not by voice assistants but by screen readers — and your markup decides what it says.

What your markup determines.

Most text-to-speech in daily use is not a voice assistant. It is a **screen reader** — NVDA, JAWS, VoiceOver, TalkBack — used by people who cannot see the screen, and the quality of the experience is determined almost entirely by the markup a developer wrote.

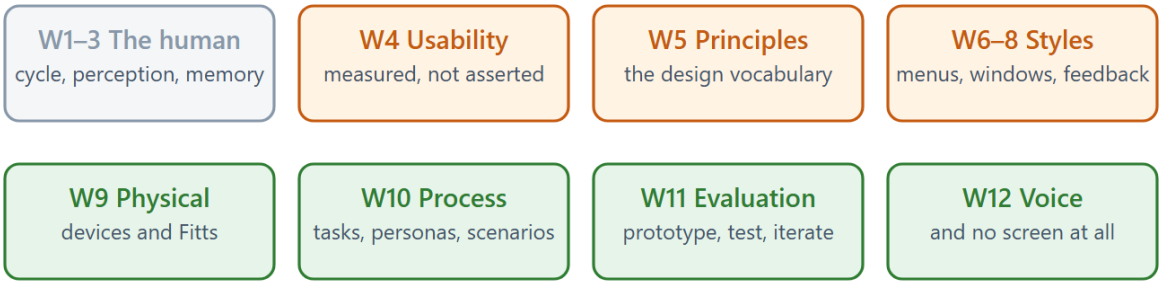
The developer wrote	The screen reader says
`<button>Submit</button>`	"Submit, button" — usable
`<button><img src="save.png"></button>`	"button" — meaningless
`<div class="btn" onclick="…">Save</div>`	*(nothing — it is not focusable)*

> **Key Idea**
>
> **The third row is the common defect**, and it is invisible in normal testing. A `<div>` styled to look like a button is, to a screen reader, not a control at all: it cannot be reached by keyboard, it is not announced, and the function is simply unavailable.
>
> You cannot detect this by looking at the screen. You detect it by using the keyboard alone, and by turning the screen reader on — which is the Week 4 lab.

Five rules that cover most of it:

1. **Use real controls** — `<button>`, `<a>`, `<input>` — not styled `<div>`s.
2. **Every control has an accessible name** — visible text, or `aria-label`.
3. **Every image has alt text**, or `alt=""` if it is decorative.
4. **Labels are associated with fields** (`<label for>`), so the field announces its purpose.
5. **Dynamic changes are announced** via a live region (`role="status"`), or the user never learns they happened.

12.8 The course, end to end



One argument: an interface is a claim about a person — and the claim can be tested.

Twelve weeks, one argument.

> **Key Idea**
>
> Three ideas have recurred every week, and they are what to carry forward.
>
> **1 — The human is the fixed part of the system.** Working memory will not get larger, attention will not become parallel, and the perceptual limits of Week 2 are the same as they were in 1970. Hardware changes; the constraint does not, and a design that respects it will outlive the technology it was built on.

رومانى صبحى رزق جرجس مطر

2 — Your intuition is not evidence. You know where everything is, you understand the model, and you cannot un-know either. Principles let you guess well; only testing tells you whether you guessed right.

3 — An interface is a claim about a person, and the claim can be tested. This is the sentence the whole course exists to establish. "This design is better" is not a matter of taste, and it is not settled by seniority — it is a hypothesis with a completion rate attached.

You should now be able to sit in front of an unfamiliar interface, say in specific language what is wrong with it and what change would fix it, and then find out whether you were right. That is the outcome, and it is the same skill whether the interface is a website, a ticket machine, a medical device or a voice assistant that does not exist yet.

Lab Activity

Lab 12 (2 hours)

1. **Voice failure log.** Use a voice assistant for ten real tasks. For each failure, classify it as recognition or understanding, and say how you could tell — or could not.

2. **Discoverability.** Ask three people who own a voice assistant to list everything it can do. Compare with the documented capability. Report the proportion.

3. **Writing for the ear.** Take a paragraph of screen text and rewrite it for speech: most important first, at most three options, ruthlessly brief. Have someone read both aloud and compare comprehension.

4. **Confirmation design.** For a banking voice interface, specify the confirmation strategy for six requests of varying consequence, and justify each.

5. **Screen reader.** Turn on a screen reader and attempt one task on your own project prototype with the monitor off. Record every point at which it is impossible.

6. **Markup audit.** Find three `<div>`-based controls in a real site and rewrite them as accessible controls.

7. **Project — final deliverable.** Your revised prototype, the evaluation report from Week 11, the changes made, the re-test result, and a one-page reflection on what you would do differently. Due this week.

Summary

• A voice interaction has five stages — speech, recognition, understanding, action, reply — and **every failure produces the same symptom**, so the user cannot adapt.

• **Recognition errors** (wrong words) and **understanding errors** (wrong intent) need different remedies. **Show what was heard** — the cheapest single improvement.

• **Voice is a different channel, not a better one**: best where a screen is unavailable, weak where one is not.

• A voice interface has **no visibility at all** — no menu, no affordance, no state. Suggest, confirm, give audible state cues, and offer a screen wherever one exists.

• **Confirm in proportion to consequence**; over-confirming destroys a voice interface faster than any other error.

رومانی صبحی رزق جرجس مطر



> • **Speech is serial, transient and slow**: most important first, at most three options, ruthlessly brief.
>
> • Most text-to-speech is **screen readers**, and your **markup determines what they say**. A styled `<div>` is not a control.
>
> • **The human is the fixed part of the system; your intuition is not evidence; an interface is a testable claim.**

Self-check

1. Name the five stages of a voice interaction and a failure mode for each.
2. Distinguish recognition from understanding errors, and give the remedy for each.
3. Give two situations where voice is the right choice and three where it is not.
4. Why does a voice interface have no visibility, and name three compensations.
5. When is implicit confirmation sufficient, and when is explicit required?
6. Give the three rules for writing spoken output, with the reason for each.
7. Why does a `<div class="btn">` fail for a screen-reader user?
8. State the three ideas that recur through the whole course.

(Full solutions are in the Week 12 question bank.)

رومانى صبحى رزق جرجس مطر

Appendix A — The Design Checklists

The working checklists from the twelve chapters, collected for use during design and review. Each is a compression of a chapter; the reasoning is in the chapter, and a rule applied without its reasoning will be applied where it does not belong.

A.1 Diagnosing a failure *(Ch. 1)*

Ask three questions, in this order:

1. **Where in the cycle did it fail?** goal → intention → **specify the action** → execute → **perceive** → interpret → evaluate
2. **Which gulf?** *Execution* ("I don't know what to do") → visibility, affordance, mapping, constraints. *Evaluation* ("I can't tell what happened") → feedback.
3. **Slip, mistake or violation?** *Slip* → separate and distinguish controls; make it reversible. *Mistake* → better labelling and feedback, so the correct model forms. *Violation* → fix the procedure.

Never stop at "user error". It names nobody who can act and nothing that can change.

A.2 Display and layout *(Ch. 2)*

- ☐ **One** accent colour for the primary action — pre-attentive coding works only while unique
- ☐ Group by **spacing** before boxes or lines; smaller gaps within, larger between
- ☐ **Front-load** the meaning; informative headings; lists for procedures
- ☐ Never centre body text
- ☐ **Redundant coding** — colour *and* shape/icon/word
- ☐ Contrast **≥ 4.5:1** for body text
- ☐ Interrupt only when it is worth interrupting a person mid-task
- ☐ Show changes **where the eye already is**, and make them persist

A.3 Memory *(Ch. 3)*

- ☐ **Show, do not store** — keep needed information on screen
- ☐ **Chunk** displayed codes, numbers and procedures; accept unchunked input
- ☐ **Offer, do not ask** — lists, suggestions, recent items
- ☐ Persistent labels; placeholders only for format examples
- ☐ **Carry context forward** — never ask twice for the same data
- ☐ Modes: **remove**, else **display unmissably** (change the cursor), else **spring-load**
- ☐ Display state, so the correct mental model forms

A.4 Usability requirements *(Ch. 4)*

Every requirement needs four parts:

Part	Test
User	specific enough to recruit five of them?
Task	complete, with a definite end?
Measure	a number someone can collect on Tuesday?
Target	a pass mark, as a proportion?

- ☐ **Baseline before setting a target**
- ☐ **Completion rate first**, then time among completers
- ☐ Define "error" and the timer boundaries **in advance**
- ☐ Where satisfaction and performance disagree, **believe performance**
- ☐ State which usability components dominate for this product

A.5 The principles *(Ch. 5)*

Principle	Question
Visibility	can the user see what is possible and what state it is in?
Feedback	does every action produce a perceptible, located, specific response?
Affordance / signifier	does it show what can be done?
Mapping	does the control relate naturally to its effect?
Constraints	is the wrong action prevented rather than reported?
Consistency	does the same thing mean the same thing — internally and externally?

Handling an invalid action, strongest first: **make it impossible** > **allow with undo** > **confirm** > **report afterwards**

A critique = observation + principle + specific change.

A.6 Menus and forms *(Ch. 6)*

Menus

- ☐ 5–10 items per menu, 2–3 levels — **broad beats deep**
- ☐ Order by **frequency or task sequence**, not alphabetically
- ☐ Destructive items **last, separated, styled differently**
- ☐ Group at about seven items

Forms

- ☐ **Delete every field you cannot justify**
- ☐ One column; labels above or left; group by spacing
- ☐ **Match the order of the source document**
- ☐ Accept any reasonable format and **show the interpretation back**
- ☐ Validate **on leaving the field**
- ☐ Field width signals expected length

A.7 Windows, icons, direct manipulation *(Ch. 7)*

- ☐ Window arrangement follows the task: overlapping (browse), tiled (compare), full screen (focus)
- ☐ **Icon + label** unless the icon is universally conventional
- ☐ Use the **conventional** icon, even when anachronistic
- ☐ **Object then action** — no mode, changeable mind, only valid actions offered
- ☐ **Drag and drop is never the only route** — always a menu or keyboard equivalent
- ☐ Offer a command or bulk route wherever a manipulation repeats

A.8 Feedback and errors *(Ch. 8)*

2025/2026 2025/2026 2025/2026

Response time

Delay	Show
< 0.1 s	nothing
0.1–1 s	cursor change, pressed state
1–10 s	busy indicator
> 10 s	progress **with an estimate**, or named phases

- ☐ Never freeze, never go backwards, never sit at 100%

Messages — four parts: **what happened · why · what to do · at the cause**

- ☐ Never blame; never bury the instruction in an apology; never use the implementation's vocabulary
- ☐ **Prevent** → **accept** → **report.** Every message is an opportunity not taken
- ☐ Interrupt in proportion to consequence
- ☐ Confirm **success** and **state**, not only failure

A.9 Physical interaction *(Ch. 9)*

$$MT = a + b\log_2(2D/W)$$

- ☐ **Enlarge the target before moving it** — same effect, lower cost

- ☐ Edges and corners are the fastest targets
- ☐ **Never move a target** as the user reaches for it
- ☐ Distance is a **safety feature** for destructive controls

Target	Minimum
mouse pointer	24 px
finger, at rest	**44 px**
finger, in motion	60 px
spacing between targets	≥ 8 px

- ☐ Primary actions in the **lower half** on a phone; destructive ones out of easy reach
- ☐ Only **tap** is discoverable — every other gesture duplicates a visible control
- ☐ **Keyboard route for everything** — accessibility and ergonomics at once

A.10 Design process *(Ch. 10)*

- ☐ **Observe** three to five real users before designing
- ☐ Watch **and** ask; where they disagree, **the discrepancy is the finding**
- ☐ Task-analyse the **goal**, not the existing interface — no "click" in any leaf
- ☐ Record the **loops and exceptions**; they are where the requirements are
- ☐ **Three to five personas**, each ending in a **prediction** 2025/2026 2025/2026 2025/2026
- ☐ **Scenarios with no interface in them** — they become the test tasks

A.11 Evaluation *(Ch. 11)*

- ☐ Lowest fidelity that answers the question — **rough gets franker criticism**
- ☐ **Heuristic evaluation first** (3–5 evaluators, independently), then user testing
- ☐ **Five users**, per group; **three tests of five beat one of fifteen**
- ☐ **Pilot** before the real sessions
- ☐ The facilitator **says nothing** — no hints, no praise, no sympathy; wait 60 seconds
- ☐ **Severity 0–4**; rank the findings; lead with three
- ☐ Report **how many, what, where, and the change** — "four of five", never "80%"
- ☐ **A fix is a hypothesis. Re-test it.**

A.12 Voice and accessibility *(Ch. 12)*

Voice

- ☐ Use it where a screen is **unavailable**; avoid it where one works
- ☐ **Show or speak back what was heard**
- ☐ Suggest capabilities on first use and after failure
- ☐ Confirm in proportion to consequence — implicit where reversible

رومانی صبحی رزق جرجس مطر

- ☐ Most important **first**; at most **three** options; ruthlessly brief

Accessibility *(applies to every chapter)*

- ☐ Real controls — `<button>`, `<a>`, `<input>` — not styled `<div>`s
- ☐ An accessible name on every control
- ☐ `alt` on every image (`alt=""` if decorative)
- ☐ `<label for>` on every field
- ☐ Dynamic changes announced (`role="status"` / `role="alert"`)
- ☐ The whole task completable **by keyboard alone**
- ☐ Contrast ≥ 4.5:1; no meaning in colour alone; no timing requirement

رومانى صبحى رزق جرجس مطر

2025/2026 2025/2026 2025/2026

Appendix B — Numbers, Formulas and Templates

B.1 The numbers worth knowing

Quantity	Value	Chapter
Working memory capacity	about **4 chunks** (traditionally quoted as 7)	3
Working memory duration	**seconds** without rehearsal	3
Pre-attentive detection	under **200 ms**, independent of set size	2
Proportion of words read on a screen	roughly **20–30%**	2
Colour-vision deficiency	about **8%** of men, **0.5%** of women	2
Minimum contrast, body text	**4.5:1** (3:1 for large text)	2
Instantaneous response	**0.1 s**	8
Flow of thought unbroken	**1 s**	8
Limit of attention	**10 s**	8
Useful animation duration	**200–300 ms**	8
Minimum target, mouse	**24 px**	9
Minimum target, finger at rest	**44 px**	9
Minimum target, finger in motion	**60 px**	9
Minimum spacing between touch targets	**8 px**	9
Menu items before grouping	**5–10**	6
Menu depth	**2–3 levels**	6
Spoken options before asking	**3**	12
Usability test participants	**5 per user group**	11
Problems found by 5 participants	about **85%**	11

2025/2026 2025/2026 2025/2026

B.2 Fitts's law

$$MT = a + b\log_2\left(\frac{2D}{W}\right)$$

- MT — movement time

- D — distance to the target
- W — target width **along the direction of movement**
- a — intercept, typically 150–250 ms for a mouse
- b — slope, typically 100–170 ms per bit for a mouse

Index of difficulty: $ID = \log_2(2D/W)$, measured in bits.

The two consequences. The relationship is **logarithmic**, so distance matters less than intuition suggests; and D and W appear in the same ratio, so **doubling the width equals halving the distance** — and is usually the cheaper change.

Worked: $D = 400$, $W = 16$, $a = 200$, $b = 150$:

$$MT = 200 + 150\log_2(50) = 200 + 150(5.64) = 1047 \text{ ms}$$

Doubling W to 32, or halving D to 200, both give $D/W = 12.5$ and $MT = 897$ ms.

B.3 Problems found by *n* participants

$$\text{found}(n) = 1 - (1 - L)^n$$

with $L \approx 0.31$, the probability that one participant encounters a given problem.

n	Found
1	31%
3	67%
5	**85%**
10	98%

2025/2026 2025/2026 2025/2026

The conclusion is about allocation, not about the number 5. With fifteen participants available, three tests of five beat one test of fifteen — because two rounds of fixes are made and verified before the last test.

Five is formative (finding problems), **not summative** (measuring performance). Never report a percentage from five participants; say "four of five".

B.4 The usability requirement template

> **[proportion] of [specific user group]** shall **[complete a specific task]** within **[measure]**, measured from **[start]** to **[end]**, **unaided**, with no more than **[error allowance]**.

Example:

> **Eight of ten** students who have never registered before shall **complete registration for five courses** within **six minutes**, measured from **login** to the **confirmation screen**, **unaided**, with no more than **two errors**.

Checks: is the user recruitable? does the task have a definite end? is the measure a number someone can collect? is the target a proportion rather than "all users"? has it been **baselined**?

B.5 The critique template

> **Observation:** what specifically happens, stated neutrally. **Principle:** which principle it violates, named. **Change:** a specific alteration, implementable as written. **Severity:** 0–4.

Example:

> After pressing Save, nothing on the screen changes. **Feedback** — the evaluation gulf is unbridged. Show "Saved 14:32" beside the button, persisting until the next edit. **Severity 3.**

B.6 The usability finding template

> **[n] of [N] participants [did what], at [where].** [What they tried, and what happened.] **Severity [0–4].** *[The proposed change.]*

Example:

> **4 of 5 participants failed at the date field.** All four typed `01/05/2026` and received "invalid format"; two retried twice and abandoned. **Severity 3.** *Accept common date formats and normalise; show the interpreted date back for confirmation.*

B.7 Severity scale

	Meaning	Action
0	not a usability problem	none
1	cosmetic	fix if time permits
2	minor — slows users	fix this release
3	major — most users fail or are seriously delayed	fix before release
4	catastrophic — data loss, task impossible, safety	fix before anything else

Severity combines **how many** users hit it, **how badly** it affects them, and **whether it persists** after the first encounter.

B.8 Nielsen's ten heuristics

1. Visibility of system status
2. Match between the system and the real world
3. User control and freedom
4. Consistency and standards
5. Error prevention
6. Recognition rather than recall
7. Flexibility and efficiency of use
8. Aesthetic and minimalist design

9. Help users recognise, diagnose and recover from errors
10. Help and documentation

B.9 Shneiderman's eight golden rules

1. Strive for consistency
2. Offer shortcuts for frequent users
3. Offer informative feedback
4. Design dialogues to yield closure
5. Prevent errors and offer simple error handling
6. Permit easy reversal of actions
7. Support internal locus of control
8. Reduce short-term memory load

B.10 Norman's principles and stages

Six principles: visibility · feedback · affordance (and signifier) · mapping · constraints · consistency.

Seven stages: form the goal → form the intention → specify the action → execute → perceive the state → interpret it → evaluate against the goal.

Two gulfs: execution (stages 2–4, bridged by visibility, affordance, mapping, constraints) and evaluation (stages 5–7, bridged by feedback).

B.11 Error types and remedies

Type	Definition	Remedy
Slip	right plan, wrong execution	separate and distinguish controls; make it reversible
Mistake	wrong plan, executed perfectly	better labelling and feedback, so the correct model forms
Violation	deliberate departure from an obstructive procedure	fix the procedure
Capture	a frequent habit takes over a similar action	do not place similar sequences adjacent
Description	right action, wrong object	make targets visually and spatially distinct
Loss of activation	forgot why you came here	keep the goal visible
Mode	right action for the other mode	remove the mode; else display it unmissably; else spring-load it

B.12 Choosing an interaction technique

The choice	Use	Avoid
one of 2–5	radio buttons — all visible	a dropdown
one of 6–20	a dropdown	radio buttons
one of hundreds	search with suggestions	any list
several of a few	checkboxes	a multi-select list
a value in a range	slider **and** number field	slider alone
a date	picker **and** typed field	picker alone
free text	a text field	voice, at a desk
an ordering	up/down buttons **and** drag	drag alone

Note how often the answer is **both** — an approximate method that is fast, and a precise method that is reliable and provides the keyboard route.

B.13 Response-time and waiting indicators

Delay	Show	Because
< 0.1 s	nothing	it is already done
0.1–1 s	cursor change, pressed state	acknowledge the action
1–10 s	busy indicator	something is happening
> 10 s	progress with an estimate, or named phases	so the user can leave and return

Never: freeze the interface · let a progress bar go backwards · hold a bar at 100%.

B.14 Diagnostic quick-reference

Symptom	Likely diagnosis	First action
user cannot find the function	stage 3, execution gulf	visibility — put it where they look
user repeats an action	stages 5–6, evaluation gulf	feedback — located, specific, persistent
user acts confidently and wrongly	a **mistake**: wrong mental model	display the state the model gets wrong
user picks the adjacent control	a **slip**	separate, distinguish, make reversible
everyone bypasses the procedure	a **violation**	fix the procedure

2025/2026 2025/2026 2025/2026

Symptom	Likely diagnosis	First action
“it randomly does the wrong thing”	a **mode error**	change the cursor; remove or spring-load the mode
users abandon a form	too many fields; rejected input	delete fields; accept any reasonable format
duplicate records in the database	evaluation gulf	confirmation that is located and persists; idempotent server
dashboard misses are common	no unique pre-attentive code	one colour for one distinction; sort by severity
low completion, high satisfaction	aesthetic-usability effect	believe the completion rate
test found nothing	the facilitator helped	say nothing; wait 60 seconds

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

Appendix C — Glossary

Accessibility. The extent to which a system can be used by people with disabilities of vision, hearing, motor control or cognition. Not an addition to usability but usability without unstated assumptions about the user. (Ch. 4)

Aesthetic-usability effect. People perceive attractive interfaces as easier to use and report higher satisfaction, independently of measured performance. It buys tolerance, not performance — and it means a beautiful bad design passes a satisfaction-based test. (Ch. 5)

Affordance. A relationship between an object and a user determining what actions are possible. It exists whether or not anyone perceives it. Contrast **signifier**. (Ch. 5)

Alarm fatigue. The condition in which frequent non-actionable alarms train staff to silence them without reading, so the one that matters is silenced too. The general mechanism behind over-notification. (Ch. 1, 2, 8)

Attention. The allocation of processing to one stream of information. Largely single-channel; what is called multitasking is rapid switching, and switching is costly. (Ch. 2)

Capture error. A frequent, well-practised habit taking over a similar but less frequent action. The reason a rare destructive action must not sit beside a routine one. (Ch. 3)

Change blindness. The normal failure to notice a substantial change to a display when it coincides with a visual interruption such as a reload, scroll or blink. (Ch. 2)

2025/2026 2025/2026 2025/2026

Chunk. Whatever a person has learned to treat as a single unit. Working-memory capacity is measured in chunks, not items, which is why chunking a displayed code improves accuracy at no cost. (Ch. 3)

Command language. An interaction style in which the user types instructions. Poor learnability; excellent for expert speed, precision, **composition** and **automation** — the last two making the interface programmable. (Ch. 6)

Consistency. *Internal* — the system behaves the same way throughout itself. *External* — it behaves like other systems the user knows. *Real-world* — it matches the domain it models. Where internal and external conflict, external usually wins. (Ch. 5)

Constraint. A limit on the actions that are possible, so that an incorrect action cannot be performed. Stronger than a warning, because it removes the error rather than reporting it. Physical, logical, cultural or semantic. (Ch. 5)

Contextual inquiry. Observing work in its real setting and asking about it there, as it happens. Watch first, ask afterwards, and ask about the exceptions. (Ch. 10)

Description error. Performing the right action on the wrong object, because the two were insufficiently distinct. (Ch. 3)

Direct manipulation. An interaction style with continuous representation of objects, physical actions rather than named commands, and **rapid, incremental, reversible operations with immediately visible effects**. Fails for bulk, repetition, unseen objects and abstract operations. (Ch. 7)

Fitts's law. $MT = a + b\log_2(2D/W)$ — movement time grows with the logarithm of the ratio of distance to target width. Doubling the width equals halving the distance. (Ch. 9)

Formative evaluation. Testing in order to find problems and fix them. Five participants suffice. Contrast **summative evaluation**, which measures performance and needs a larger sample. (Ch. 11)

Gestalt laws. Principles by which the visual system groups a scene before it is read: proximity, similarity, enclosure, continuity, closure, common fate. **Proximity is the strongest**, and it overrides labels. (Ch. 2)

Gulf of evaluation. The gap between the state the system is in and the user's ability to perceive and interpret it. Bridged by **feedback**. Produces quiet failures — retries and duplicate records — rather than complaints. (Ch. 1)

Gulf of execution. The gap between what the user wants to do and what the system lets them do. Bridged by **visibility, affordance, mapping and constraints**. (Ch. 1)

Heuristic evaluation. An inspection method in which three to five evaluators independently examine an interface against a list of principles, then combine findings. Finds different problems from user testing; run it first. (Ch. 11)

Hierarchical task analysis. Decomposing a goal into sub-tasks until each leaf is a single observable action, with a **plan** at each level stating order and conditions. Analyse the task, not the existing interface. (Ch. 10)

Icon. A graphical control understood by **resemblance** or by **convention** — never by novelty. Use icon plus label unless the icon is universally conventional. (Ch. 7)

Implicit confirmation. Folding the system's understanding into its response — "Turning on the lamp" — so the user can interrupt at no time cost. Contrast **explicit confirmation**, which requires an affirmative before acting. (Ch. 12)

Iteration. Prototyping, testing and revising repeatedly. Not repetition caused by failure but the method itself, since the necessary information is in the users rather than the designer. (Ch. 10, 11)

Learnability. How easy a system is to use the first time. One of five usability components; conflicts with **efficiency**. (Ch. 4)

Mapping. The relationship between a control and its effect. A **natural mapping** exploits spatial or cultural analogy so that nothing must be learned — knobs arranged as the burners are. (Ch. 5)

Memorability. How easy a system is to use on return after a period away. The component teams most often forget; it governs the long tail of rarely-used functions. (Ch. 4)

Mental model. A person's internal explanation of how something works. Built entirely from the **system image**, and acted on confidently whether or not it is correct. (Ch. 3)

Mistake. A failure of the plan — the wrong intention, executed perfectly. Caused by a wrong mental model; remedied by better labelling and feedback, never by a confirmation dialog. (Ch. 1)

Mode. A state in which the same action produces a different result. Remedies, in order: remove it; display it unmissably (change the cursor); make it spring-loaded. (Ch. 3)

Object-action order. Selecting the object first and then choosing the action. Avoids a mode, lets the user change their mind, and lets the system offer only valid actions. (Ch. 7)

Persona. A concrete description of a representative user — goals, context, skills, constraints — built from observation and ending in a **prediction** that can settle a design argument. Three to five, or it constrains nothing. (Ch. 10)

Pre-attentive feature. A visual property detected in under about 200 ms, in a time independent of the number of distractors. Works only while **unique**: emphasis is a budget. (Ch. 2)

Prototype. A representation of a design built to answer a question. Use the lowest fidelity that can answer it — rough artefacts attract franker criticism. (Ch. 11)

Recall. Retrieving an item from memory unaided. Slow and unreliable. Contrast **recognition**. (Ch. 3)

Recognition. Judging whether a presented item is the one wanted. Fast and comparatively accurate — the asymmetry that explains why menus replaced command lines. (Ch. 3)

Recognition error *(voice)*. A failure to convert audio into the correct words. Contrast **understanding error**, where the words were right and the intent was wrong. The user cannot tell them apart unless the system shows what it heard. (Ch. 12)

Response-time thresholds. 0.1 s — feels instantaneous; 1 s — flow of thought unbroken; 10 s — the limit of attention. Properties of people, unchanged since the 1960s. (Ch. 8)

Scenario. A short narrative describing a specific person, in a specific situation, achieving a specific goal, **with no reference to the interface** — because it becomes the task in a usability test. (Ch. 10)

2025/2026 2025/2026 2025/2026

Screen reader. Software that speaks an interface aloud, used by blind and severely low-vision users. What it says is determined almost entirely by markup; a styled `<div>` is not a control. (Ch. 12)

Severity. A 0–4 rating combining how many users encounter a problem, how badly it affects them, and whether it persists. Ranking findings is part of the evaluator's job. (Ch. 11)

Signifier. The perceptible indication that an action is possible — a handle, a border, a label. What the designer actually controls. Contrast **affordance**. (Ch. 5)

Slip. A failure to execute a correct plan — the right intention carried out wrongly. Remedied by separating and distinguishing controls and by making actions reversible. (Ch. 1)

System image. Everything a system presents: screen, labels, behaviour, documentation. The only channel between the designer's model and the user's, and therefore the only place a wrong mental model can be corrected. (Ch. 3)

Task allocation. The decision about which parts of a task a person performs and which the machine performs. The commonest error is giving the human the machine's job — accuracy without attention. (Ch. 1)

Text-to-speech. Converting text to spoken audio. Most of its daily use is in screen readers rather than voice assistants. Speech is serial, transient and slow, so spoken output puts the most important information first and offers at most three options. (Ch. 12)

رومانی صبحی رزق جرجس مطر

Usability. The extent to which a **specified user** can achieve a **specified goal** in a **specified context** with effectiveness, efficiency and satisfaction. A property of a system, user, goal and context together — never of a system alone. (Ch. 4)

Usability requirement. A statement with four parts — user, task, measure, target — that a build can fail. “The system shall be easy to use” is not one. (Ch. 4)

User-centred design. An approach in which users, tasks and context are studied first and repeatedly, requirements are stated testably, and designs are evaluated with real users until the requirements are met. (Ch. 10)

Violation. A deliberate departure from a prescribed procedure. When widespread, it is evidence that the procedure obstructs the work — a design finding, not a discipline problem. (Ch. 1)

Visibility. The principle that the possible actions and the current state should be apparent without searching, remembering or experimenting. On screen at the point of use > in a menu > behind a gesture. (Ch. 5)

WIMP. Windows, Icons, Menus, Pointer — the interface paradigm from Xerox PARC in the early 1970s. It survives because each element converts a memory task into a perception task. (Ch. 7)

Working memory. The store holding information currently in use. About four chunks, decaying in seconds, destroyed by interruption. The bottleneck of every interface. (Ch. 3)

رومانی صبحی رزق جرجس مطر

Course Sheet — Applied Questions

CS331 Human Computer Interaction · Computer Science, FCI Tanta University

Student name: _____

Student ID: _____

Section / Lab group: _____

Project group: _____

Date submitted: _____

> **Note**
>
> **Instructions.**
>
> 1. This sheet covers the whole course (Chapters 1–12) and counts towards the **Assignments / Laboratory Activities** mark. Total: **50 marks**.
>
> 2. Answer **in your own words**, in the space provided. Where a question asks for a principle, *name it and apply it to the case* — a list of definitions earns no credit.
>
> 3. Question 14 requires you to observe real interfaces and real people. Answers that are clearly invented will receive zero.
>
> 4. Hand in the completed sheet, detached from the book, by **Week 13**.

Part A — People and Machines (Chapters 1–3)

Q1. *(4 marks)* A student opens the university portal to print their exam timetable. They find the timetable page easily, but cannot find any print option. They take a screenshot instead, then discover the screenshot cuts off half the table.

(a) At which of Norman's seven stages did the first failure occur? Which gulf is it?

(b) Propose **one** design change, and name the principle it applies.

رومانى صبحى رزق جرجس مطر

Q2. *(4 marks)* Classify each of the following as a **slip**, a **mistake** or a **violation**, and give the remedy that follows from your classification.

(a) A cashier presses *Void* instead of *Total*; the two keys are identical and adjacent.

(b) A user deletes "Old Files" thinking it is a shortcut, not realising it holds the only copy of their work.

(c) Lab staff write the shared admin password on a sticky note because the system logs them out every five minutes.

(d) A student types their ID into the *Name* field because both fields look the same.

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

Q3. *(3 marks)* A system shows a one-time code on screen and asks the user to type it into a field on the **next** page.

(a) Which human limitation does this design ignore?

(b) Rewrite the step so that the task allocation is correct.

Part B — Usability and Principles (Chapters 4–5)

Q4. *(3 marks)* "The new registration system must be easy to use." Rewrite this as **two** usability requirements that could fail a test. Name the usability component each one measures.

Q5. *(4 marks)* For each problem below, **name the principle violated** and **propose a specific fix**:

(a) A form's *Submit* button does nothing visible for four seconds; users click it again.
(b) Four cooker knobs in a row control four burners arranged in a square.
(c) An app uses a trash-can icon for *Archive* on one screen and for *Delete* on another.
(d) A "Delete account" option sits immediately next to "Edit profile", in the same style.

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

Part C — Interaction Styles, Errors and Response Time (Chapters 6–8)

Q6. *(3 marks)* A faculty website menu has 2 top-level items, each opening 2 sub-items, each opening 2 more, five levels deep. Explain what is wrong in terms of **breadth versus depth**, and propose a better structure.

Q7. *(3 marks)* Rewrite this error message so that it lets the user recover. State the three things your version contains.

> **Error 0x80045: Invalid input.**

Q8. *(3 marks)* Match each operation to the feedback it needs, using the **0.1 s / 1 s / 10 s** thresholds:

(a) a button changes colour when pressed — the system takes 50 ms;

(b) a search returns results in 3 s;

(c) a report export takes 40 s.

2025/2026 2025/2026 2025/2026

Part D — Physical Interaction (Chapter 9)

Q9. *(4 marks)* Use Fitts's law, $MT = a + b\log_2(2D/W)$, with $a = 200$ ms and $b = 150$ ms/bit.

A *Save* button 20 px wide sits 600 px from where the pointer usually starts.

(a) Compute the index of difficulty and the movement time.

(b) The designer can either make the button 60 px wide or move it to 200 px away. Compute MT for each option, and say which is the cheaper change.

رومانی صبحی رزق جرجس مطر

Q10. *(2 marks)* A mobile banking app uses 30 px buttons with 2 px spacing for "Confirm" and "Cancel". Identify the problem and give the corrected values.

Part E — Design Process and Evaluation (Chapters 10–11)

Q11. *(3 marks)* A team writes this persona: *"Ahmed, 21, likes football and coffee, uses an iPhone."* Explain why it is not useful, and rewrite it so that it could guide design decisions for a course-registration system.

Q12. *(4 marks)* Your group tested its prototype with **3** participants and found 6 problems.

(a) Using found$(n) = 1 - (1 - L)^n$ with $L = 0.31$, roughly what proportion of the problems would 3 participants be expected to find? What about 5?

(b) Write one finding in the standard form (*n of N participants … ; severity; proposed change*).

(c) Why should the facilitator stay silent during the test?

2025/2026 2025/2026 2025/2026

رومانى صبحى رزق جرجس مطر

Part F — Voice Interfaces (Chapter 12)

Q13. *(2 marks)* A university phone line says: *"For admissions press 1, for exams press 2, for fees press 3, for transcripts press 4, for timetables press 5, for housing press 6, for library press 7, for anything else press 8."* Explain the problem using what you know about memory and voice, and propose a fix.

2025/2026 2025/2026 2025/2026

Part G — Field Work

Q14. *(8 marks)* Record **three** entries from your failure log — real interfaces you used during the term. Use the format from Chapter 1: what you wanted, what happened, the **stage** and **gulf**, the **principle** violated, and the **fix**.

Example of the expected form: **Ticket machine, Tanta station.** I wanted a return ticket. I pressed "Tickets", then looked for "Return" and could not find it — it was a checkbox inside "Single". **Stage 3, execution gulf. Visibility.** *Fix:* show Return as a first-class option beside Single.

Entry 1.

Entry 2.

Entry 3.

رومانى صبحى رزق جرجس مطر

Question	1	2	3	4	5	6	7	8	9	10	11	12	13	14	Total
Marks	4	4	3	3	4	3	3	3	4	2	3	4	2	8	**50**
Awarded															